Инструкция по установке ПО

Школа синтеза цифровых схем · basics-graphics-music · версия 0.2 от 8 октября 2026 года

1. Введение

Эта инструкция описывает установку программного обеспечения, которое нужно для лабораторных работ Школы синтеза цифровых схем, и для работы с репозиторием примеров `basics-graphics-music` (далее BGM).

Инструкция заменяет предыдущую редакцию, `Инструкция_по_установке_ПО.pdf` от 3 октября 2025 года. Фотографии и снимки экрана из той редакции сохранены. Там, где сведения устарели, рядом стоит пометка «Было верно раньше» с объяснением, что изменилось. Полный перечень расхождений собран в отдельном файле.

1.1. Школа синтеза цифровых схем

Школа синтеза цифровых схем (School of Digital Circuit Synthesis) — бесплатная образовательная программа компании **YADRO**, посвящённая схемотехнике и цифровому дизайну: разработке на уровне RTL, функциональной верификации и основам проектирования микросхем.

По данным сайта школы, программа объединяет **более 2000 участников и 24 вуза** России и Белоруссии. Занятия сезона 2026/2027 проходят по субботам с 12:00 до 15:00 по московскому времени — в университетских кластерах очно и онлайн с записью. Участие бесплатное; YADRO предлагает проектную работу со своими инженерами, стажировки и кадровый резерв.

Актуальная информация и регистрация: edu.yadro.com/chip-design-school.

1.2. Репозиторий basics-graphics-music

BGM — это набор переносимых примеров на SystemVerilog для ПЛИС и для заказных микросхем. Над ним работали **более 60 человек** (69 авторов по данным `git log` на момент подготовки этой редакции); ведущий разработчик — **Юрий Панчул**.

Репозиторий используется в Школе синтеза цифровых схем и на семинарах в разных странах: Бишкек (2022), Тбилиси (2023), Баку и Hacker Dojo в Кремниевой долине (2024), Тихуана и Ереван (2025).

Существуют три копии репозитория, и выбор между ними имеет значение:

НАЗНАЧЕНИЕ	АДРЕС
Экспериментальная версия разработки	github.com/yuri-panchul/basics-graphics-music
Стабильная версия для Школы синтеза	github.com/chipdesignschool/basics-graphics-music
Стабильная версия для международных семинаров	github.com/verilog-meetup/basics-graphics-music

Для занятий Школы синтеза берите версию `chipdesignschool` : она стабильна в течение семестра. Версия `yuri-panchul` — рабочая, в ней ведётся разработка, и поведение может измениться в любой день.

Чем BGM отличается от других наборов примеров

- **Один и тот же скрипт работает со всеми САПР.** В зависимости от выбранной платы `03_synthesize_for_fpga.bash` запускает Altera Quartus, AMD Vivado, Gowin EDA или открытый маршрут проектирования на основе Yosys. Учащемуся не нужно изучать интерфейс каждой из этих программ, чтобы начать.
- **Графика и звук с самого начала.** Примеры сразу работают с HDMI, VGA, ЖК-экранами и светодиодными матрицами, с распознаванием нот через микрофон и синтезом звука через ЦАП и усилитель, — а не с абстрактными мигающими светодиодами.
- **Микроархитектура, как в индустрии.** Есть примеры с конвейерами, очередями FIFO и управлением потоком по кредитам — то есть ровно те темы, которые спрашивают на собеседованиях.
- **54 платы** четырёх производителей ПЛИС поддерживаются одним и тем же кодом примеров (см. раздел 3).

Подробнее о структуре курса — в разделе 8 и в статье на Хабре: habr.com/ru/articles/1071736.

1.3. Репозиторий systemverilog-homework

`systemverilog-homework` (SVH) — набор небольших задач по SystemVerilog с автоматической проверкой. Он дополняет BGM: в BGM примеры запускаются на плате, а в SVH отрабатываются языковые конструкции и микроархитектурные приёмы в симуляторе.

НАЗНАЧЕНИЕ	АДРЕС
Версия Школы синтеза	github.com/chipdesignschool/systemverilog-homework
Версия разработки	github.com/yuri-panchul/systemverilog-homework
Версия для международных семинаров	github.com/verilog-meetup/systemverilog-homework

Для SVH достаточно Icarus Verilog и Git — платы и САПР не нужны. Подробнее в разделе 9.

1.4. Синтез примеров в заказную микросхему

Примеры BGM можно не только запускать на ПЛИС, но и превратить в настоящую микросхему.

- **Tiny Tapeout** — сервис, который собирает проекты многих участников на один кристалл (MPW, «шаттл») и изготавливает их на фабрике. Для BGM есть шаблон, который берёт модуль `lab_top` из примера. Подробнее в разделе 10 и в статье habr.com/ru/articles/1084844.
- **LibreLane** — открытый маршрут RTL-to-GDSII, который можно запустить у себя локально. В BGM за это отвечает скрипт `07_synthesize_for_asic.bash`, а посмотреть топологию можно скриптами `08_` и `09_`. Установка — в разделе 4.1.11.

> БЫЛО ВЕРНО РАНЬШЕ
>
> Прежняя редакция инструкции описывала установку **OpenLane** через Docker. В репозитории теперь используется **LibreLane** — развитие того же проекта, устанавливаемое через Nix. Старый скрипт `00_setup_open_lane.source_bash` в репозитории ещё лежит, но основным стал `00_setup_libre_lane.source_bash`.

2. Что изменилось по сравнению с прежней инструкцией

Короткий обзор; технические подробности — в соответствующих разделах, а построчный разбор ошибок прежней редакции — в файле исправлений.

ТЕМА	ЧТО НОВОГО
Altera вместо Intel	Intel снова стала Altera. Quartus Lite 25.1 ставится в каталог `altera_lite`, Quartus Pro — в `altera_pro`. Скрипты знают все восемь вариантов имени каталога.
Старые версии Quartus	Для плат на Cyclone II (DE1, DE2) нужен Quartus II **13.0sp1**; для Cyclone III (DE0, Марсоход MCY316) — **13.1** или 13.0sp1. Новые версии эти микросхемы уже не поддерживают.
Quartus Pro и новая плата	Terasic **DE23-Lite** (Agilex 3) требует редакции **Pro** с бесплатной лицензией. Ни одна редакция Lite эту микросхему не собирает.
Поиск инструментов	Задавать `PATH` не обязательно — скрипты находят Quartus, Vivado, Gowin EDA, Icarus и остальное самостоятельно.
USB Blaster III	Добавлено правило udev для USB Blaster III (`idProduct 6022`), нужное для DE23-Lite.
Icarus Verilog	Рекомендуется версия **13**, лучше **14**. Прежняя редакция описывала версию 12.

ТЕМА	ЧТО НОВОГО
OSS CAD Suite	Открытый маршрут для плат Gowin и Lattice; содержит и свой Icarus, который скрипты тоже умеют использовать.
Surfer	Просмотрщик временных диаграмм, альтернатива GTKWave: и как программа, и как расширение VS Code. Если `surfer` найден, скрипты используют его.
LibreLane вместо OpenLane	Весь раздел про OpenLane переписан.
XPACK RISC-V GCC	Компилятор C и ассемблера для программ, исполняемых ядрами `aps` (МИЭТ), `yrv` и `picorv32` внутри ПЛИС.
Упражнения с UART	В `labs/8_unfinished/8_1_uart` появились три упражнения, в которых плата разговаривает с компьютером по UART: вывод принятого байта на индикатор, эхо и калькулятор с обратной польской записью. Проверить их можно только через терминальную программу, поэтому установка терминала стала отдельным разделом — 4.1.11 для Windows и 4.2.11 для Linux.
Round-Robin Threads	Пример многопоточности на синтезированном процессоре переименован: прежнее название *Femto Threads*, нынешнее — **Round-Robin Threads** (RR Threads, RRT), `labs/5_cpu/program/10_rr_threads` .
Инфраструктура упражнений с процессорными ядрами	Сборка и загрузка программ для ядер RISC-V вынесена в `labs/5_cpu/program` с общими утилитами (`elf2hex` , `bin2hex`) и отдельной инструкцией по установке компилятора. Как это устроено — в статье habr.com/ru/articles/1091540.
Терминалы	Скрипт `14_run_terminal_program.bash` работает с minicom, picocom и putty; установка каждого описана в разделах 4.1.11 и 4.2.11.
Больше плат	54 платы вместо 39 в прежнем перечне. В скриптах появилась также поддержка САПР Efinity для ПЛИС Efinix, но действующих плат на Efinix пока нет — см. раздел 11.
Имя файла настройки	`00_setup_intel_fpga.source_bash` переименован в `00_setup_altera.source_bash` .
Устаревшие адреса	Репозитории на `gitflic.ru` больше не используются; `valid-ready-etc` устарел. Всё на GitHub.

3. Поддерживаемые платы и требования к версиям САПР

Репозиторий поддерживает **54 плат** на ПЛИС четырёх производителей. Таблица ниже — копия файла `boards/README.md` из репозитория; там же лежат варианты на русском, в форматах `.html` и `.csv`. Данные для этой таблицы взяты из файла `boards/README.csv` того же репозитория, поэтому она не может разойтись с ним.

ИЗГОТОВИТЕЛЬ ПЛИС	ИЗГОТОВИТЕЛЬ ПЛАТЫ	ПЛАТА	СЕМЕЙСТВО ПЛИС	САПР И ПОДХОД
Altera	ALINX	`alinx_ax301`	Cyclone IV E	Quartus с 13.0sp
Altera	ALINX	`alinx_ax4010`	Cyclone IV E	Quartus с 13.0sp
Altera	Altera	`dk_dev_3c120n`	Cyclone III	Quartus 13.1 и с
Altera	Marsohod	`marsohod_mcy112`	Cyclone	Quartus 9.1 SP2; нужна лицензи
Altera	Marsohod	`marsohod_mcy316`	Cyclone III	Quartus 13.1 и с
Altera	OMDAZZ	`omdazz`	Cyclone IV E	Quartus с 13.0sp
Altera	OMDAZZ	`omdazz_epm570`	MAX II	Quartus с 13.0sp
Altera	Piswords	`piswords6`	Cyclone IV E	Quartus с 13.0sp
Altera	Terasic	`c5gx`	Cyclone V	Quartus с 13.0sp
Altera	Terasic	`de0`	Cyclone III	Quartus 13.1 и с
Altera	Terasic	`de0_cv`	Cyclone V	Quartus с 13.0sp
Altera	Terasic	`de0_nano`	Cyclone IV E	Quartus с 13.0sp
Altera	Terasic	`de0_nano_soc`	Cyclone V	Quartus с 13.0sp
Altera	Terasic	`de1`	Cyclone II	Quartus 13.0sp1
Altera	Terasic	`de10_lite`	MAX 10	Quartus 14.0.2 и
Altera	Terasic	`de10_nano`	Cyclone V	Quartus с 13.0sp
Altera	Terasic	`de1_soc`	Cyclone V	Quartus с 13.0sp
Altera	Terasic	`de2`	Cyclone II	Quartus 13.0sp1

ИЗГОТОВИТЕЛЬ ПЛИС	ИЗГОТОВИТЕЛЬ ПЛАТЫ	ПЛАТА	СЕМЕЙСТВО ПЛИС	САПР И ПОДХОД
Altera	Terasic	de23_lite	Agilex 3	Quartus Pro 26.1
Altera	Terasic	de2_115	Cyclone IV E	Quartus с 13.0sp
Altera	Terasic	terasic_sockit	Cyclone V	Quartus с 13.0sp
Altera	ZEOWAA	zeowaa	Cyclone IV E	Quartus с 13.0sp
Altera	unknown	emooc_cc	Cyclone IV E	Quartus с 13.0sp
Altera	unknown	rzrd	Cyclone IV E	Quartus с 13.0sp
Altera	unknown	saylinx	Cyclone IV E	Quartus с 13.0sp
Gowin	Marsohod	marsohod3gw2	GW1NR-9	Gowin EDA, люба
Gowin	Sipeed	tang_mega_138k	GW5AST	Gowin EDA, 1.9.9
Gowin	Sipeed	tang_mega_138k_pro	GW5AST-138	Gowin EDA, люба
Gowin	Sipeed	tang_nano_20k	GW2AR-18	Gowin EDA, люба
Gowin	Sipeed	tang_nano_4k	GW1NSR-4	Gowin EDA, люба
Gowin	Sipeed	tang_nano_9k	GW1NR-9	Gowin EDA, люба Yosys/OSS †
Gowin	Sipeed	tang_primer_20k_dock	GW2A-18C	Gowin EDA, люба Yosys/OSS †
Gowin	Sipeed	tang_primer_20k_lite	GW2A-18	Gowin EDA, люба
Gowin	Sipeed	tang_primer_25k	GW5A	Gowin EDA, 1.9.9
Gowin	Xunlong	orangepi_msoc	GW5AT-138B	Gowin EDA, люба
Lattice	1BitSquared	icebreaker	iCE40	САПР не назнач Yosys/OSS †
Lattice	Colorlight	colorlight75b	ECP5	Yosys/OSS †
Lattice	Colorlight	colorlightI5	ECP5	Yosys/OSS †
Lattice	Fabmicro	karnix	ECP5	Yosys/OSS †

ИЗГОТОВИТЕЛЬ ПЛИС	ИЗГОТОВИТЕЛЬ ПЛАТЫ	ПЛАТА	СЕМЕЙСТВО ПЛИС	САПР И ПОДХОД
Lattice	Greg Davill	`orangecrab`	ECP5	Yosys/OSS †
Lattice	Olimex	`ice40hx8k_evb`	iCE40	Yosys/OSS †
Xilinx	ALINX	`alinx_ax7035b`	Artix-7	Vivado, любая в
Xilinx	Digilent	`arty_a7_100`	Artix-7	Vivado, любая в
Xilinx	Digilent	`arty_a7_35`	Artix-7	Vivado, любая в
Xilinx	Digilent	`basys3`	Artix-7	Vivado, любая в
Xilinx	Digilent	`cmod_s7`	Spartan-7	Vivado, любая в
Xilinx	Digilent	`eclypse_z7`	Zynq-7000	Vivado, любая в
Xilinx	Digilent	`nexys4`	Artix-7	Vivado, любая в
Xilinx	Digilent	`nexys4_ddr`	Artix-7	Vivado, любая в
Xilinx	Digilent	`nexys_a7_100`	Artix-7	Vivado, любая в
Xilinx	Digilent	`nexys_a7_50`	Artix-7	Vivado, любая в
Xilinx	Digilent	`zybo_z7`	Zynq-7000	Vivado, любая в
Xilinx	QMTech	`qmtech_kintex_7`	Kintex-7	Vivado, любая в
Xilinx	unknown	`a7_lite_35t`	Artix-7	Vivado, любая в

Пояснения к столбцам:

- **САПР и подходящие версии.** Диапазоны для Quartus *измерены*: у каждой установленной версии спрашивали, умеет ли она собирать проект под конкретную микросхему платы. Крестиком (†) помечены диапазоны, у которых одна граница не проверена — например, нижняя граница 14.0.2 для MAX 10. Для Vivado, Gowin EDA и открытого маршрута в репозитории требований к версии не записано.
- **TM1638.** «Всегда» — обёртка платы всегда использует модуль клавиатуры и индикатора TM1638; «нет» — не использует; «на выбор» — в репозитории есть сборки и с ним, и без него.
- **Графика.** То, что подключает обёртка платы, а не то, на что способна сама плата.

Одна и та же плата встречается в репозитории в нескольких конфигурациях — с модулем TM1638 и без него, с HDMI или с одним из ЖК-экранов, со штатным САПР или с открытым маршрутом. Всего 112 каталогов в `boards/` на 54 платы.

3.1. Три случая, когда версия Quartus критична

Для большинства плат годится любой современный Quartus Lite. Но есть три исключения, из-за которых нельзя просто поставить самую новую версию:

ПЛАТЫ	МИКРОСХЕМА	ЧТО СТАВИТЬ	ПОЧЕМУ
Terasic DE1, DE2	Cyclone II	**Quartus II 13.0sp1**	Версия 13.1 уже не поддерживает Cyclone II
Terasic DE0, Марсоход MCY316, DK-DEV-3C120N	Cyclone III	**Quartus II 13.1** (или 13.0sp1)	Версия 21.1 и новее уже не поддерживает Cyclone III
Terasic DE23-Lite	Agilex 3	**Quartus Prime Pro** + бесплатная лицензия	Ни одна редакция Lite эту микросхему не собирает

> ПРИМЕЧАНИЕ
>
> Несколько версий Quartus могут стоять на одной машине одновременно и не мешают друг другу. Скрипты сами выберут подходящую для выбранной платы — см. раздел 4.1.1.4. Именно поэтому для старых плат удобно поставить 13.0sp1 или 13.1 *рядом* с новым Quartus Lite, а не вместо него.

> ВНИМАНИЕ
>
> Плата **Марсоход MCY112** построена на микросхеме Cyclone первого поколения (EP1C12). Ни одна из проверенных версий Quartus — от 13.0sp1 до 25.1std и Pro 26.1.1 — не собирает для неё проект: все выдают ошибку `Error (20005)` о необходимости лицензии. Файл проекта самой платы в репозитории был создан редакцией Quartus II 9.1 SP2 Web, то есть бесплатной. Для этой платы нужен Quartus того же поколения либо лицензия.

4. Установка САПР и сопутствующего ПО

Порядок разделов: Windows (4.1), Linux (4.2), macOS (4.3). Внутри каждого — одни и те же инструменты в одном и том же порядке.

Что обязательно, а что нет. Обязательны Git, Icarus Verilog с просмотрщиком диаграмм, VS Code и RARS — без них не выполнить ни одной лабораторной работы. САПР для ПЛИС нужен только тот, который соответствует вашей плате: для платы на Altera — Quartus, на Xilinx — Vivado, на Gowin — Gowin EDA или OSS CAD Suite, на Lattice — OSS CAD Suite. Если платы нет вообще, ставить САПР не нужно: примеры можно моделировать в Icarus Verilog, а упражнения SVH выполнять целиком без платы.

> ПРИМЕЧАНИЕ

> Во всех случаях в пути к каталогу установки **не должно быть русских букв и пробелов**. Это касается всех перечисленных программ и является самой частой причиной необъяснимых сбоев.

4.1. Windows 10 и 11

4.1.1. Установка Quartus (для плат на ПЛИС Altera)

Quartus нужен только для плат с ПЛИС Altera (прежнее название — Intel). Если у вас плата на Xilinx, Gowin или Lattice, пропустите разделы 4.1.1–4.1.3.

> НОВОЕ В ЭТОЙ РЕДАКЦИИ
>
> Intel снова стала **Altera**. Новые версии ставятся в каталоги `altera_lite` (редакция Lite) и `altera_pro` (редакция Pro), тогда как версии времён Intel ставились в `intelFPGA_lite` и `intelFPGA`. Скрипты репозитория знают все эти имена, поэтому выбор версии ни на что не влияет — ставьте в предлагаемый установщиком каталог.

Какую версию выбрать

- **Quartus Prime Lite 21.1** — версия, на которую рассчитаны лабораторные работы Школы и которая описана в этой инструкции пошагово. Годится для всех плат на Cyclone IV, Cyclone V, MAX II и MAX 10.
- **Quartus Prime Lite 25.1** и другие свежие версии — тоже подходят для тех же плат. Ставятся в `altera_lite`.
- **Quartus II 13.0sp1 или 13.1** — *дополнительно*, если у вас плата на Cyclone II или Cyclone III (см. 3.1).
- **Quartus Prime Pro** — только для DE23-Lite.

4.1.1.1. Пошаговая установка Quartus Prime Lite 21.1

1. Убедитесь, что скачаны оба файла: установщик (`QuartusLiteSetup-21.1.exe`) и архив поддержки вашей микросхемы с расширением `.qdz`. Оба файла должны лежать **в одной папке**.

Какой именно `.qdz` нужен, определяется микросхемой на плате:

ПЛАТА	МИКРОСХЕМА	ФАЙЛ ПОДДЕРЖКИ
DE10-Lite	MAX 10	`max10-21.1.1.850.qdz`
DE10-Nano, DE1-SoC, DE0-CV	Cyclone V	`cyclonev-21.1.1.850.qdz`
OMDAZZ, RzRd, ZEOWAA, DE2-115, DE0-Nano	Cyclone IV	`cyclone-21.1.1.850.qdz`

Модель микросхемы написана на самом корпусе микросхемы на плате; её же можно найти в столбце «Семейство ПЛИС» таблицы раздела 3.

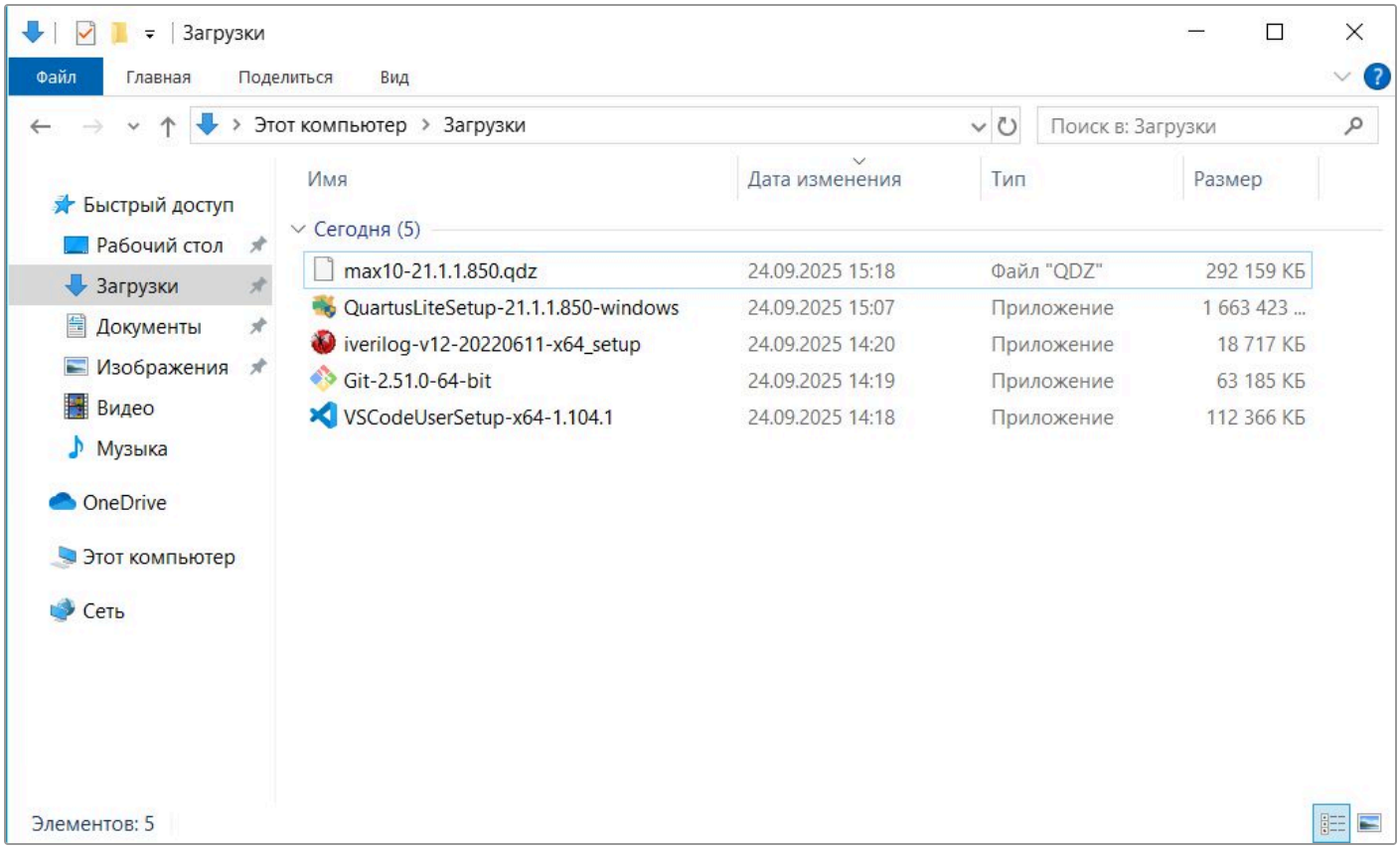


Рис. 1. Установщик Quartus и файл .qdz в одной папке

2. Запустите установщик и примите условия лицензионного соглашения.

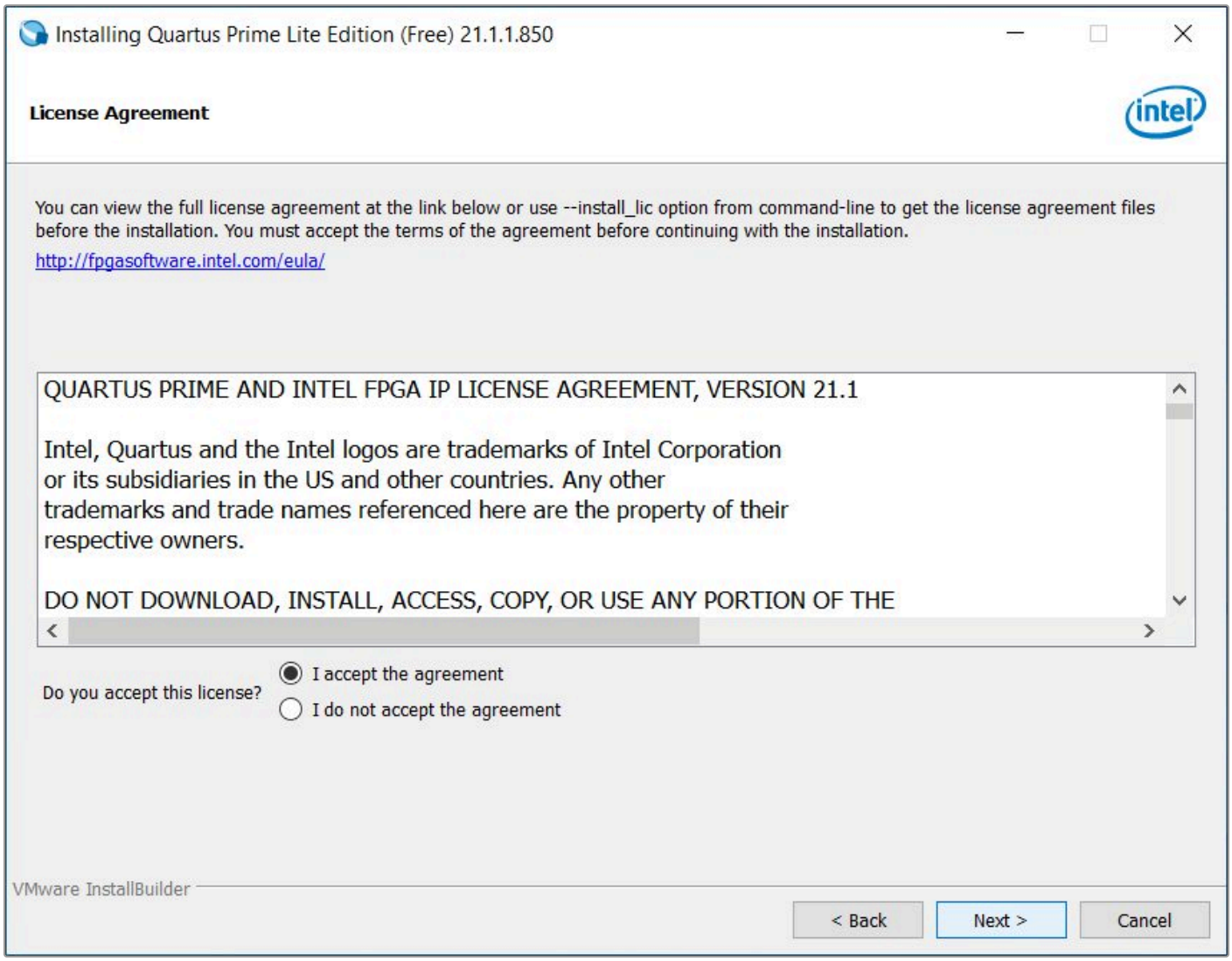


Рис. 2

3. Путь установки оставьте предложенный по умолчанию.

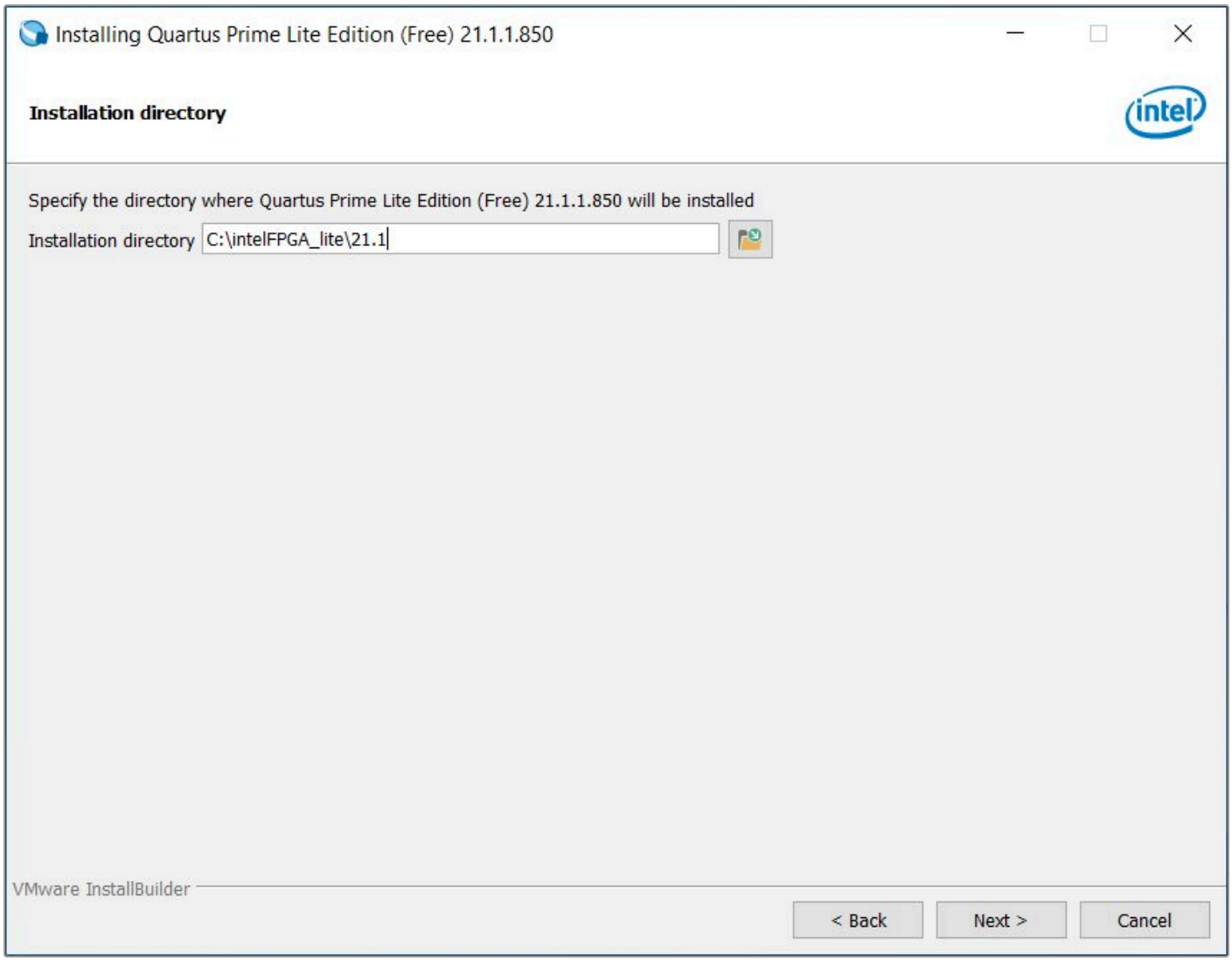


Рис. 3

4. Если файл `.qdz` лежал рядом с установщиком, на этом шаге в списке *Devices* появится ваша микросхема. Если файлов несколько, отметьте галочкой только нужную серию — остальные занимают место без пользы.

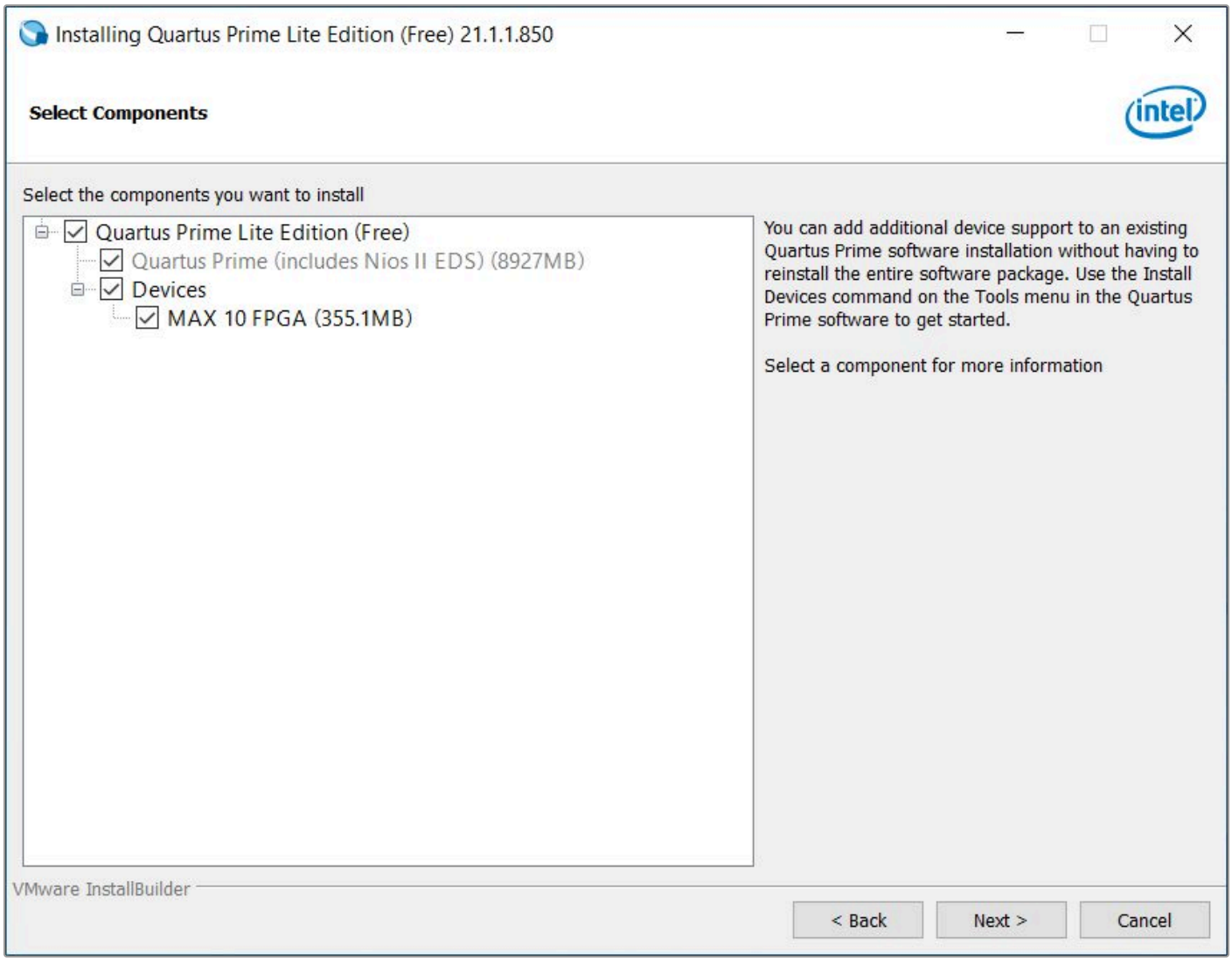


Рис. 4

5. Нажмите *Next* и дождитесь окончания установки.

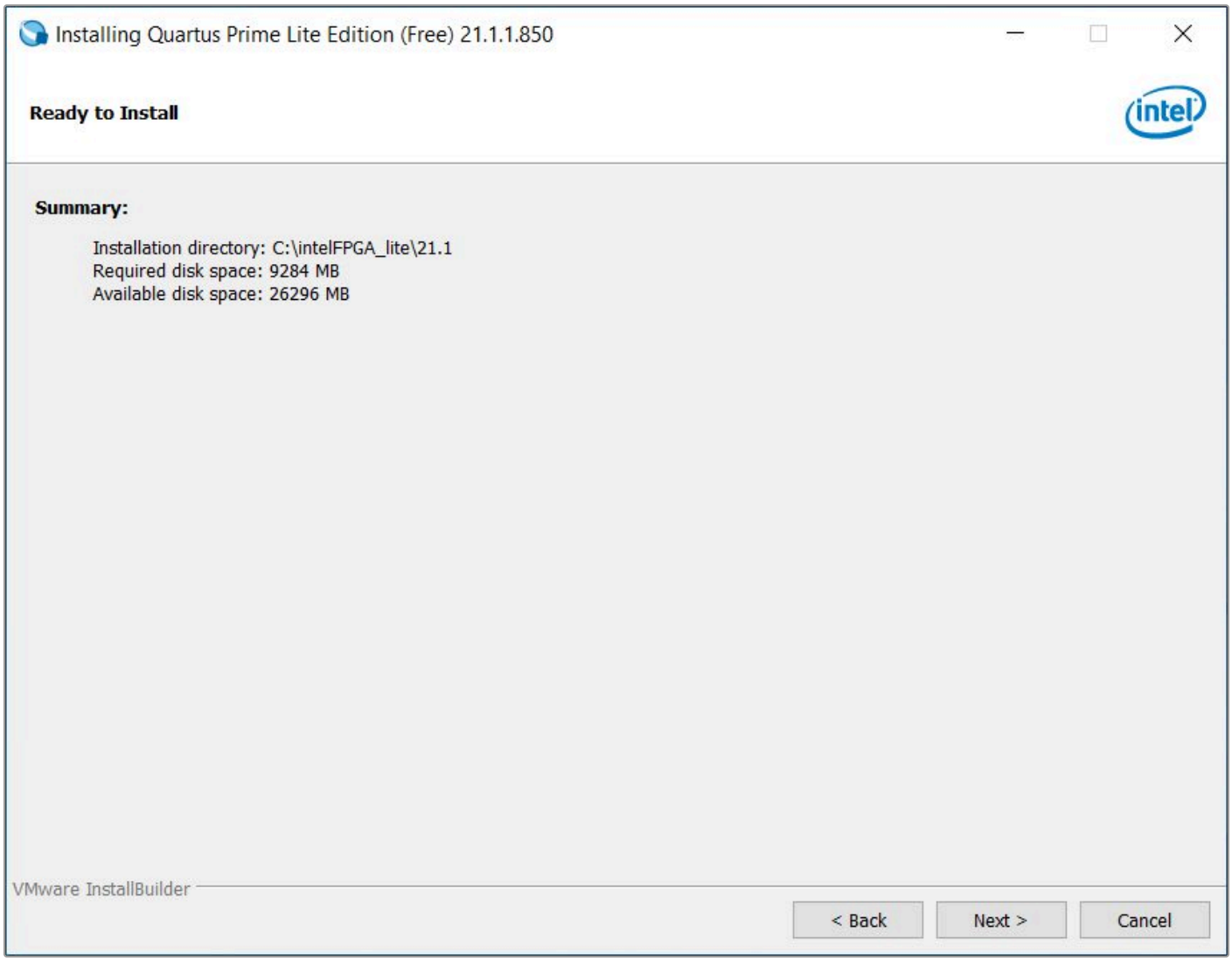


Рис. 5

6. На последнем экране обязательно поставьте галочку *Launch USB Blaster II driver installation* и нажмите *Finish*.



Рис. 6

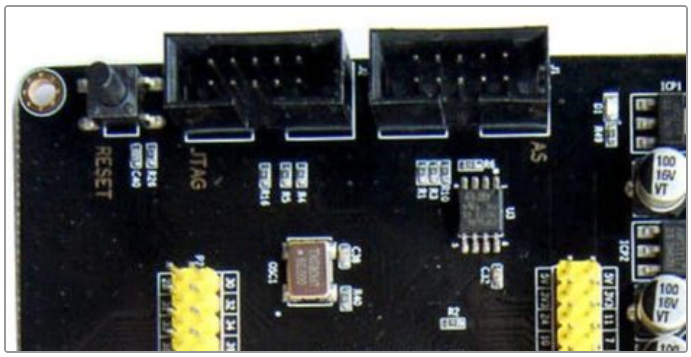


Рис. 7

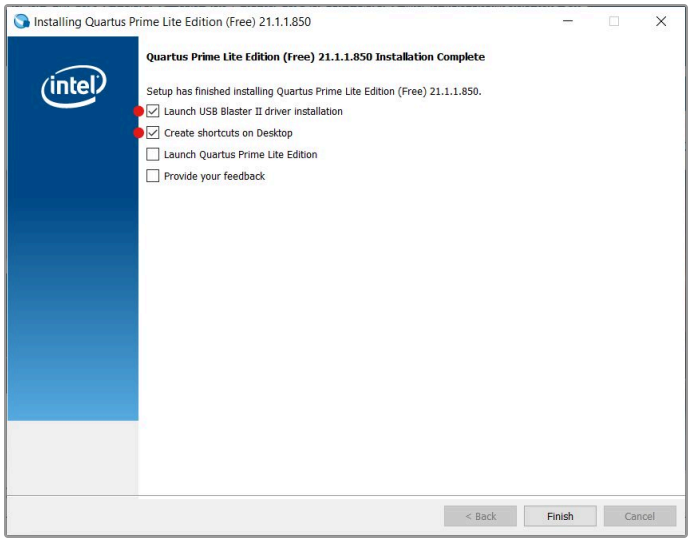


Рис. 8

> ПРИМЕЧАНИЕ
>
> Прописывать переменные окружения и `PATH` после установки **не нужно**. Как скрипты находят Quartus — в разделе 4.1.1.4.

4.1.1.2. Установка драйвера USB Blaster

Для прошивки платы нужен драйвер программатора. Дальше в качестве примера взята плата OMDAZZ/RzRd с программатором DDS2022-24.

1. Подключите программатор. Шлейф JTAG одним концом — к разъёму **JTAG** на плате (не к AS!), другим — к программатору. Обратите внимание на выемку в разъёме: шлейф вставляется только одной стороной.

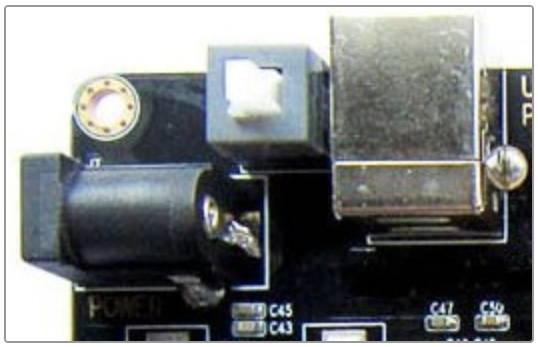


Рис. 9

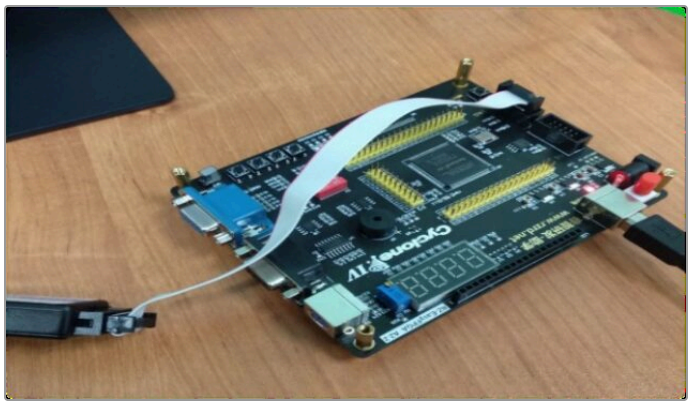


Рис. 10

2. Подключите питание платы кабелем USB B-type и включите её кнопкой рядом с разъёмом. Готовая к работе плата выглядит так:



Рис. 11

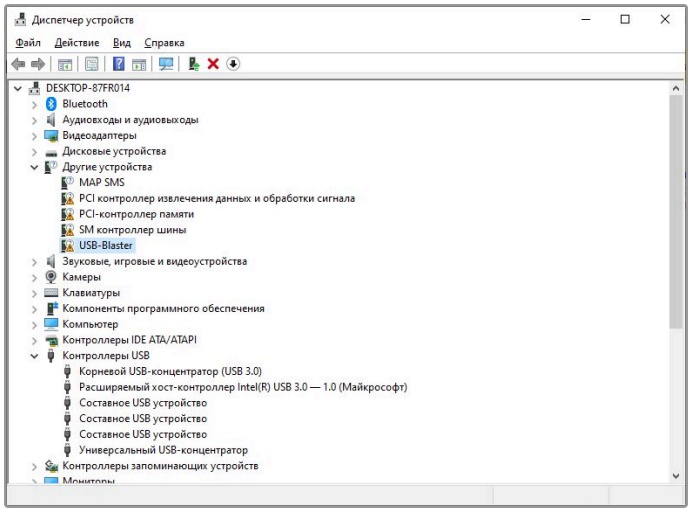


Рис. 12

3. Откройте *Диспетчер устройств* (быстрее всего — через поиск у кнопки «Пуск»). После подключения программатора в списке появится новое устройство.

4. Нажмите на нём правой кнопкой → *Обновить драйвер* → *Найти драйверы на этом компьютере*.

5. В качестве папки поиска укажите каталог установки Quartus. Драйвер лежит внутри него, в подкаталоге `quartus/drivers/usb-blaster`.

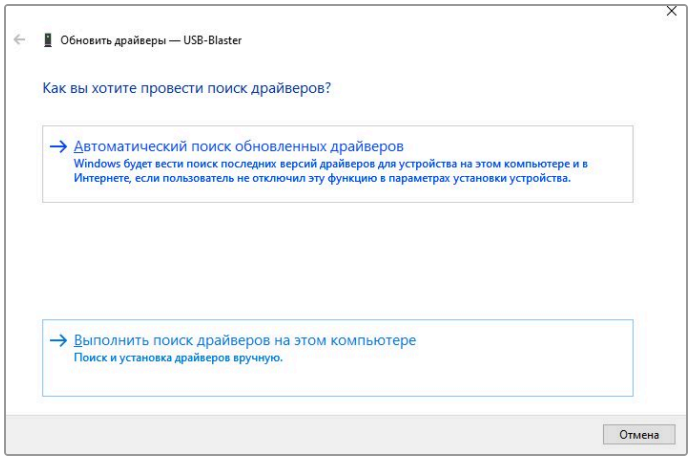


Рис. 13

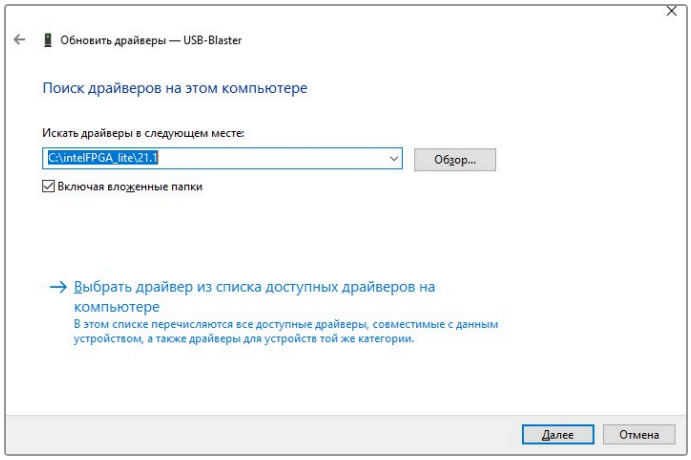


Рис. 14

ПРИМЕЧАНИЕ

Если устройство в диспетчере не появилось: попробуйте другой разъём USB (в том числе USB 2.0 вместо 3.0 — работают не все), проверьте подключение JTAG и то, что на программаторе горит светодиод. Если на семисегментном индикаторе OMDAZZ разом горят все сегменты, плата неисправна. Если ничего не помогло, напишите модераторам Школы.

НОВОЕ В ЭТОЙ РЕДАКЦИИ

USB Blaster III. У программаторов разных поколений разные идентификаторы USB, и драйвер нужен свой. У плат с USB Blaster III — к ним относится DE23-Lite — идентификатор продукта `6022`. Под Windows драйвер ставится так же, из каталога Quartus; для Linux нужно правило udev, см. раздел 4.2.1.2.

4.1.1.3. Проверка установки Quartus

1. Запустите Quartus. Должно появиться окно выбора; нажмите *Run the Quartus Prime Software*.

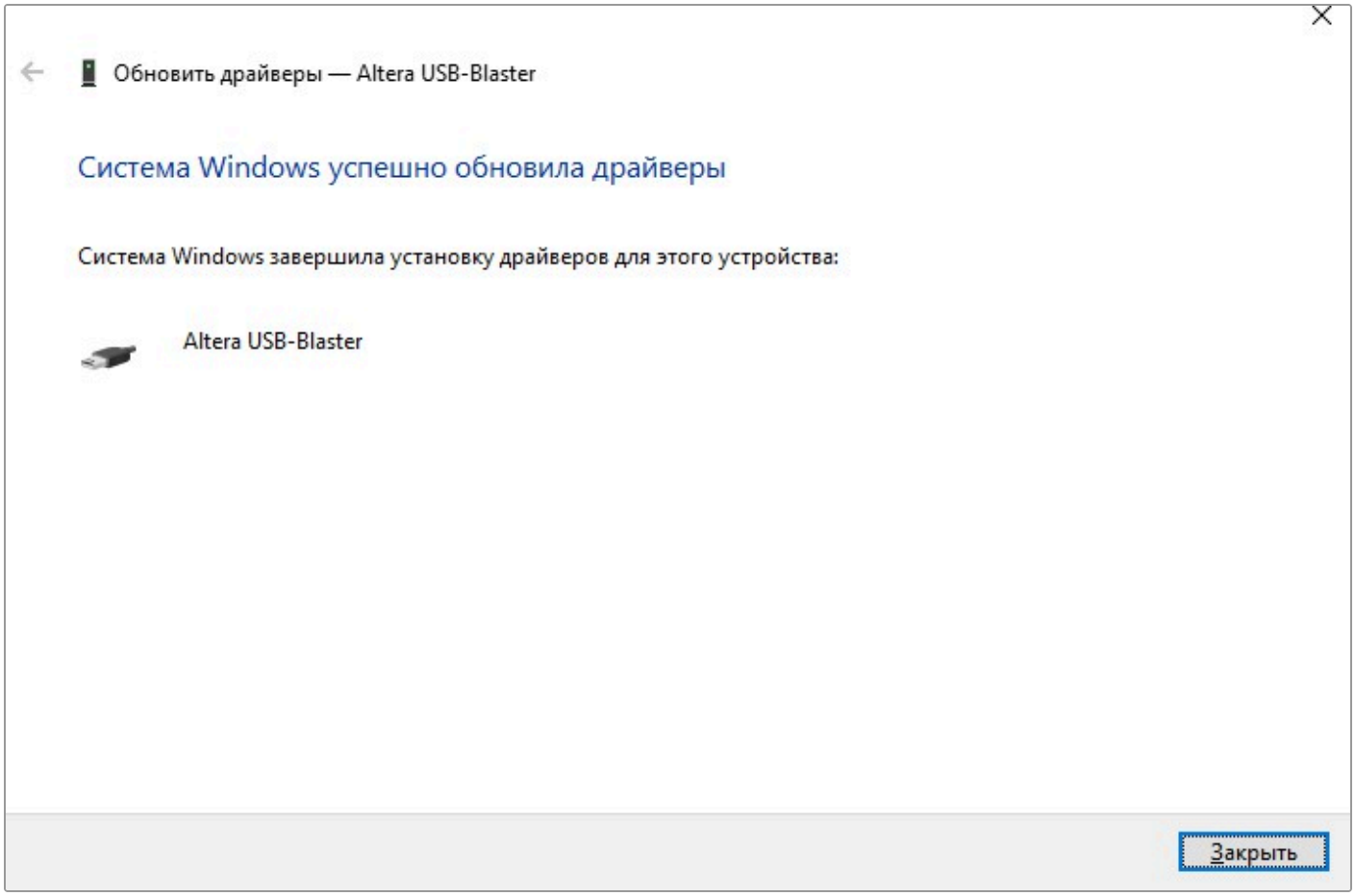


Рис. 15

2. Если плата есть, откройте окно *Programmer*.

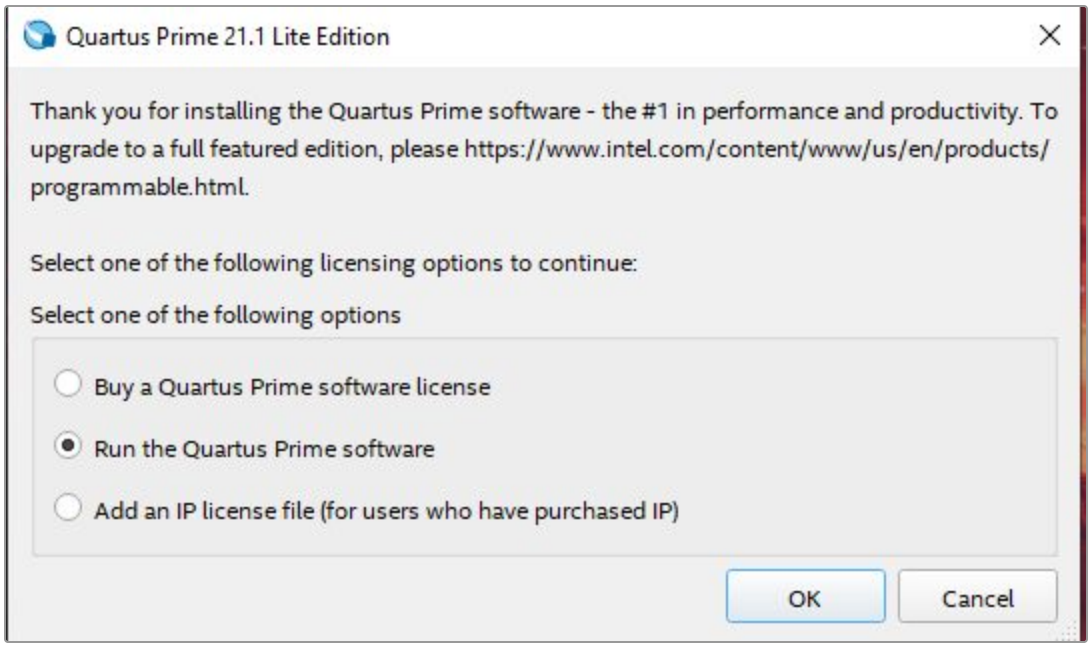


Рис. 16

3. Нажмите *Hardware Setup*.

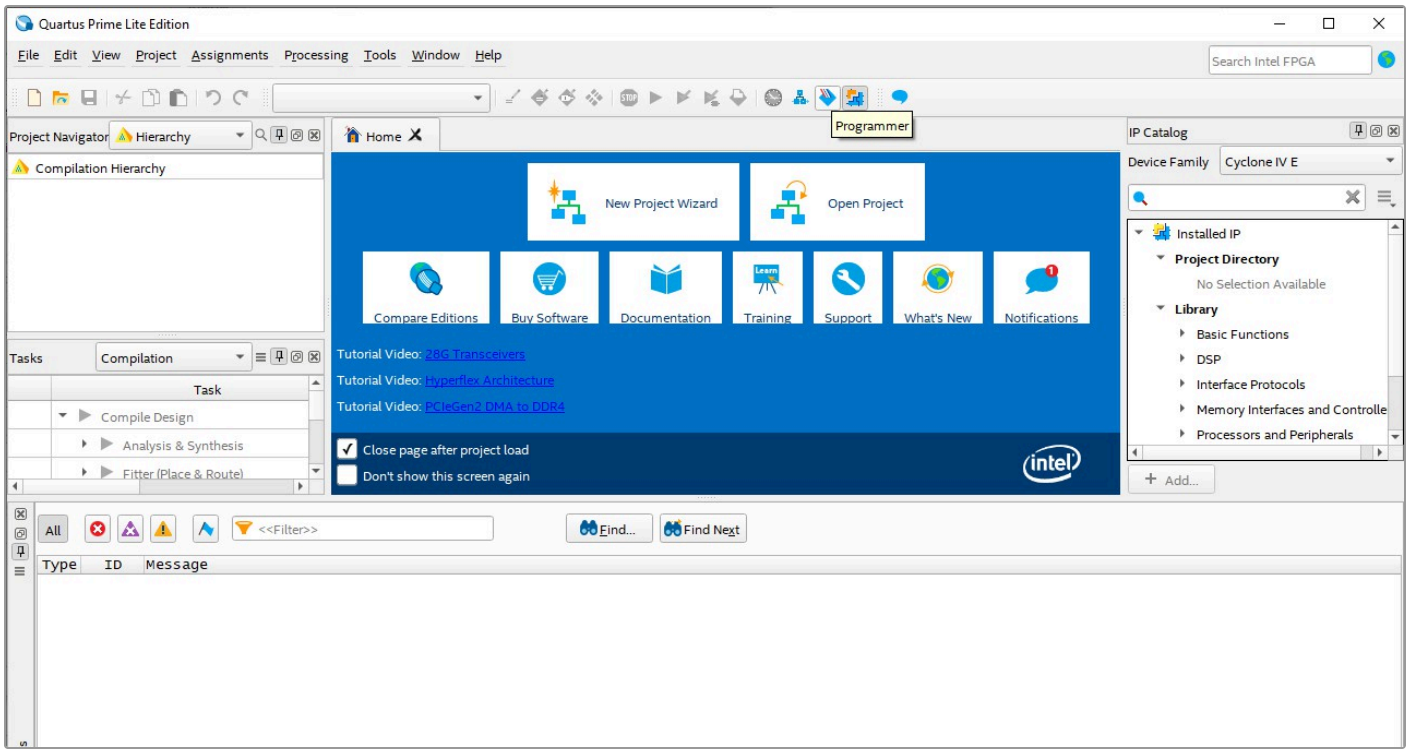


Рис. 17

4. Если в выпадающем списке есть *USB-Blaster*, драйвер установлен правильно.

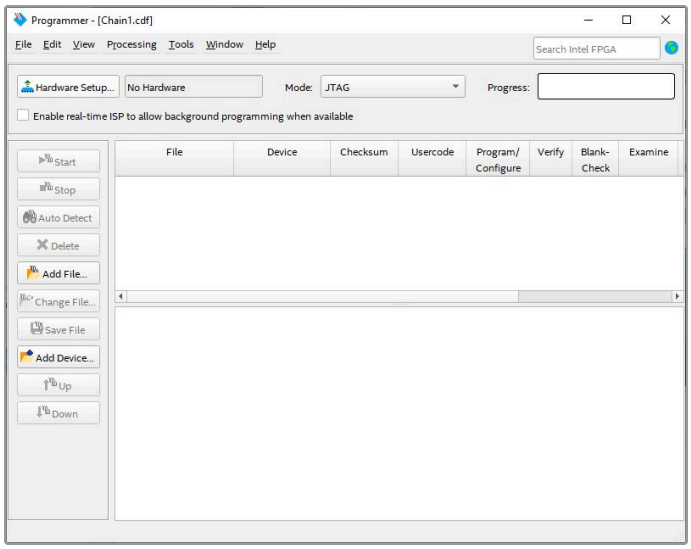


Рис. 18

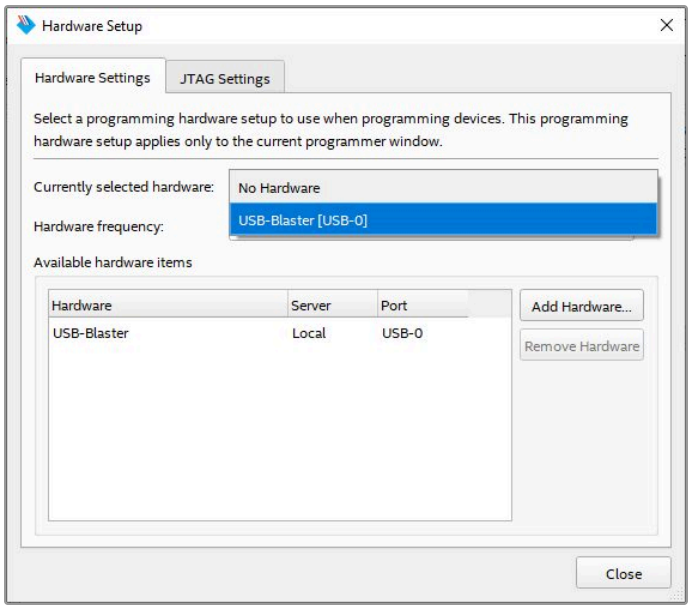


Рис. 19

5. Добавьте через *Add File* готовый файл прошивки `.sof` и нажмите *Start*. Надпись *Successful* означает, что Quartus и программатор работают.

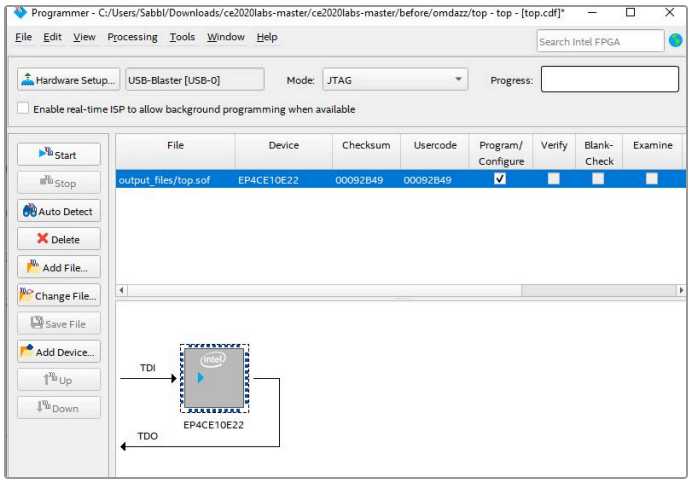


Рис. 20



Рис. 21

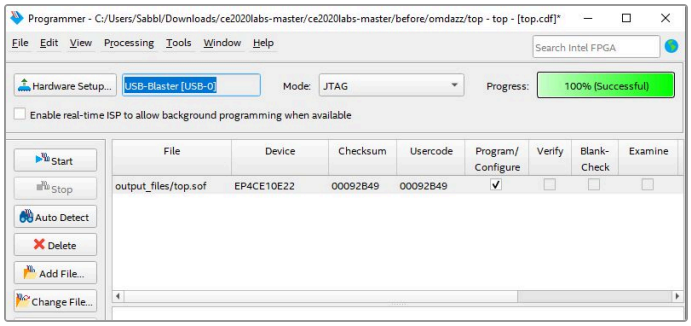


Рис. 22

> **ПРИМЕЧАНИЕ**
>
> Отдельный файл прошивки для проверки скачивать не обязательно: достаточно собрать любой пример репозитория скриптом `03_synthesize_for_fpga.bash` — он и соберёт, и прошьёт

> плату. Прежняя редакция инструкции предлагала скачать `top.sof` с Яндекс.Диска; этот путь остаётся рабочим, но необязателен.

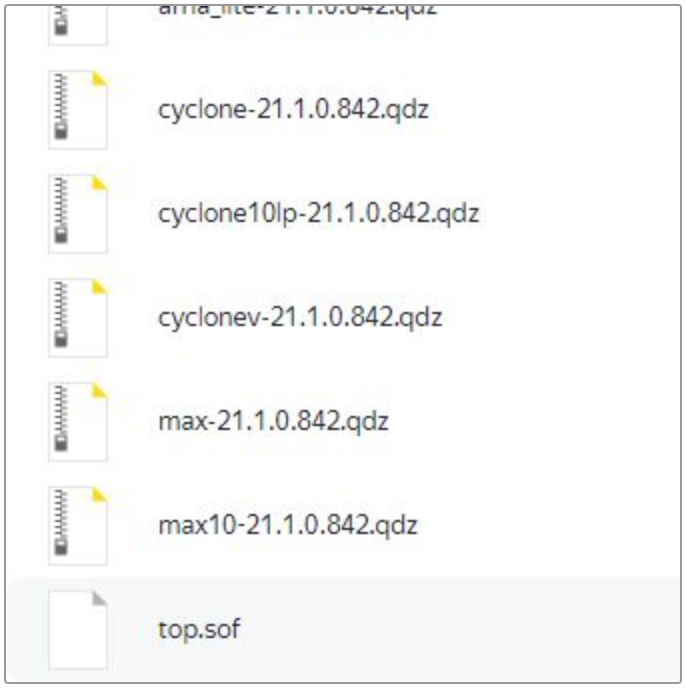


Рис. 23
Окно Programmer после успешной прошивки

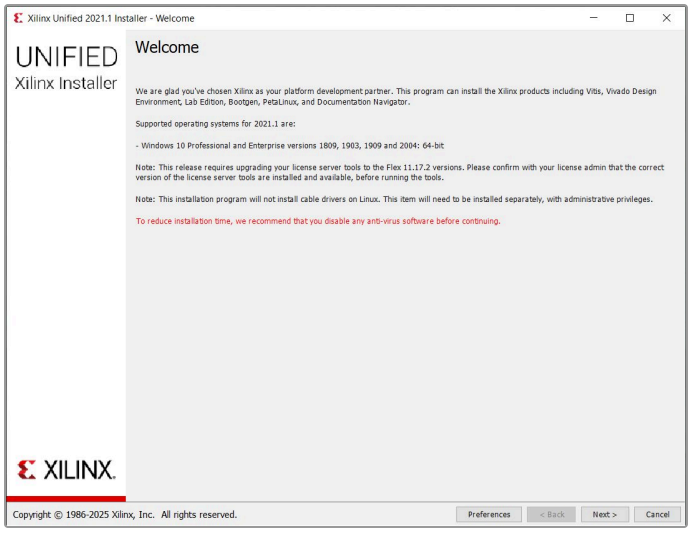


Рис. 24

4.1.1.4. Как скрипты BGM находят Quartus

В репозитории BGM синтез, прошивка и запуск графического интерфейса выполняются одними и теми же скриптами для всех САПР:

```
./03_synthesize_for_fpga.bash          # синтез и прошивка
./04_configure_fpga.bash               # только прошивка
./05_run_gui_for_fpga_synthesis.bash   # графический интерфейс
```

За поиск Quartus отвечает файл `scripts/steps/00_setup_altera.source_bash`. Порядок поиска такой, и первый успешный шаг прекращает дальнейший поиск:

1. `QUARTUS_ROOTDIR` — если переменная задана и указывает на каталог `quartus` внутри установки, берётся именно он, без всякого поиска.
2. `quartus` в `PATH` — если он уже доступен и подходит для вашей платы, скрипт ничего не меняет.
3. `INTEL_FPGA_HOME`, затем `ALTERA_HOME`, затем `QUARTUS_HOME` — каталог, внутри которого лежат каталоги установок.
4. Стандартные места: домашний каталог, `/opt`, `/tools` (под Windows — диски `/c`, `/d`, `/e`).

В каждом из этих мест проверяются все имена каталогов, которыми пользовались установщики Altera и Intel за годы:

```
altera  altera_lite  altera_std  altera_pro
intelFPGA  intelFPGA_lite  intelFPGA_std  intelFPGA_pro
```

> ПРИМЕЧАНИЕ
>
> **Прописывать `PATH` не обязательно.** Прежняя редакция инструкции требовала добавить `QUARTUS_ROOTDIR`, `QSYS_ROOTDIR` и `PATH` в `~/.bashrc`. Сейчас это необязательно: при установке в стандартное место скрипты находят Quartus сами. Переменные остаются полезными в двух случаях — когда установка лежит в нестандартном каталоге и когда на машине несколько версий и вы хотите принудительно выбрать одну.

Имя каталога **ничего не говорит** ни о версии, ни о редакции. На машине разработчика `altera_lite/25.1std` содержит редакцию Lite, а `altera/13.1` — редакцию Web (так раньше называлась Lite). Поэтому скрипт не доверяет именам: он запускает `quartus_sh` и спрашивает у самого инструмента его версию, редакцию и список микросхем, которые тот умеет собирать.

Дальше скрипт берёт из файла проекта вашей платы (`boards/<плата>/board_specific.qsf`) конкретную микросхему и оставляет только те установки, которые её поддерживают. Среди оставшихся выбор идёт так:

1. бесплатная редакция (Lite или Web) — впереди платной, даже если она старше;
2. затем более новая версия;
3. Standard впереди Pro — но только при равных версиях.

> ВНИМАНИЕ
>
> Опрос занимает от одной до пяти секунд на установку, и он выполняется при каждом запуске: в ветке `main` результат **нигде не сохраняется**. Если на машине стоят несколько версий Quartus и эта задержка мешает, задайте `QUARTUS_ROOTDIR` — тогда скрипт берёт указанную установку и не опрашивает остальные.

Если ни одна установка не умеет собирать проект под микросхему вашей платы, скрипт сообщает об этом сразу, называя микросхему и перечисляя найденные версии, — вместо того чтобы выбрать неподходящую и упасть позже посреди синтеза. Если при этом вы сами указали установку через `QUARTUS_ROOTDIR` или через `INTEL_FPGA_HOME` / `ALTERA_HOME` / `QUARTUS_HOME`, скрипт вам доверяет: он выдаёт предупреждение и продолжает работу.

4.1.2. Установка AMD Vivado (для плат на ПЛИС Xilinx)

Vivado нужен только для плат с ПЛИС AMD Xilinx. Для плат на Altera, Gowin или Lattice он не требуется.

> ВНИМАНИЕ

> Образ установщика Vivado занимает более 80 ГБ, а установленная программа — более 50 ГБ. Проверьте свободное место заранее.

Лабораторные работы рассчитаны на версию **2022.2**. Сообщество Школы проверяло, что работы выполняются и на версиях 2018, 2021 и 2023, однако рекомендуется именно 2022.2.

1. На приветственном экране установщика нажмите *Next*.

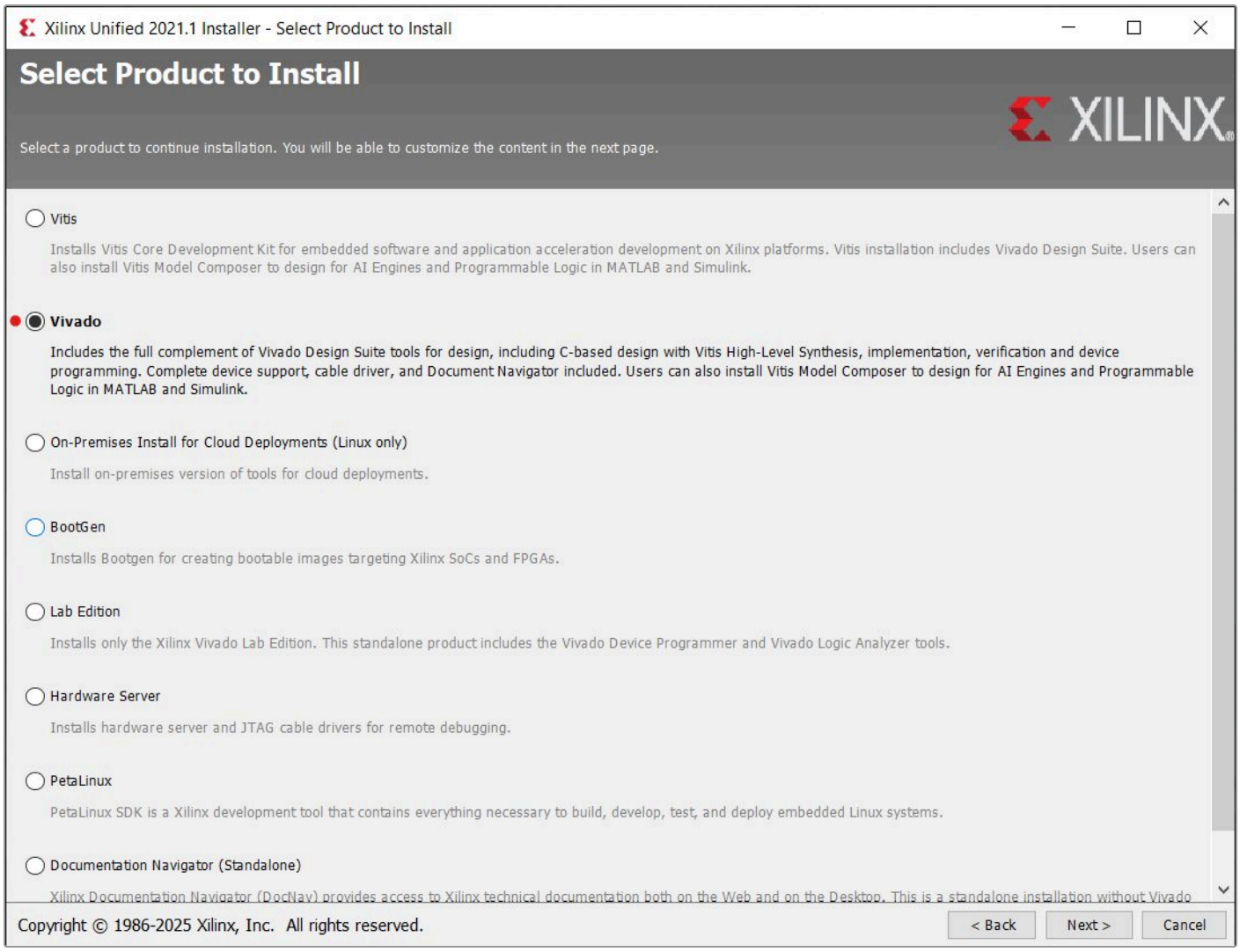


Рис. 25

2. Поставьте флажок напротив *Vivado*.

3. На шаге *Product Devices* отметьте серии микросхем, которые поддерживаются в репозитории: **Zynq-7000**, **Artix-7**, **Kintex-7** и **Spartan-7**. Остальные серии занимают десятки гигабайт без пользы.

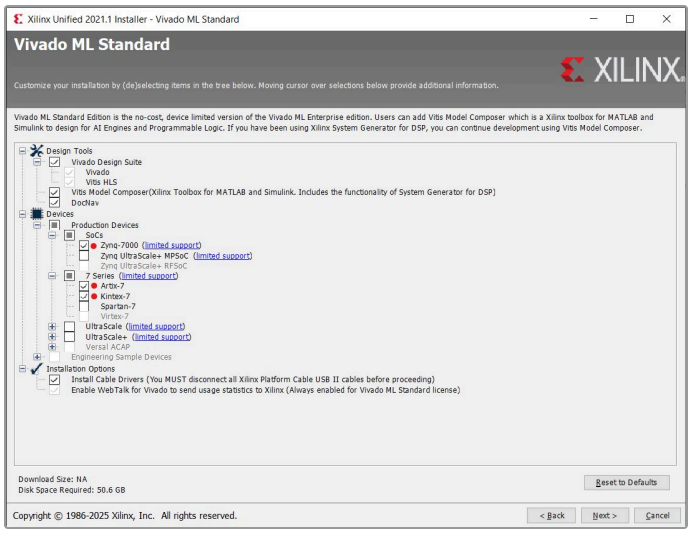


Рис. 26

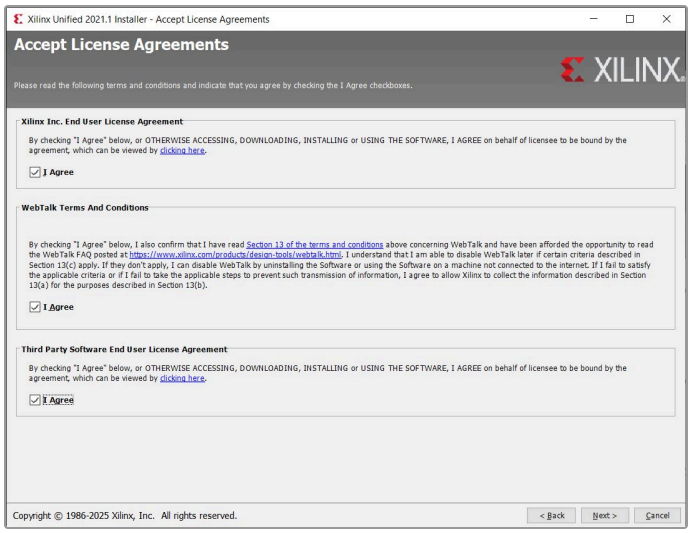


Рис. 27

4. Примите условия лицензионного соглашения, оставьте путь установки по умолчанию и нажмите *Install*.

> БЫЛО ВЕРНО РАНЬШЕ
>
> Прежняя редакция называла только Zynq-7000, Artix-7 и Kintex-7. В репозитории сейчас есть и платы на **Spartan-7** (например, Digilent Cmod S7), поэтому эту серию тоже стоит отметить.

Как скрипты BGM находят Vivado

За поиск отвечает `scripts/steps/00_setup_xilinx.source_bash`. Порядок такой же по смыслу, как у Quartus:

1. `XILINX_VIVADO` — прямое указание на каталог конкретной версии;
2. `vivado` в `PATH`;
3. `XILINX_HOME` — каталог, внутри которого лежат установки;
4. стандартные места: домашний каталог, `/opt`, `/tools`; под Windows — `/c`, `/d`, `/e`.

Внутри каждого места проверяются каталоги `Xilinx`, `AMD` и `AMDDesignTools`, причём в двух раскладках:

```
<место>/<вендор>/Vivado/<версия>    — Vivado 2024.1 и старше
<место>/<вендор>/<версия>/Vivado    — Vivado 2024.2 и новее
```

> НОВОЕ В ЭТОЙ РЕДАКЦИИ
>
> Начиная с выпуска 2024.2 AMD поменяла раскладку каталогов: номер версии теперь стоит *выше* названия продукта, а каталог вендора называется уже не обязательно `Xilinx`. Прежние версии скриптов из-за этого не находили Vivado; теперь поддерживаются обе раскладки.

Если установок несколько, выбирается самая новая версия. Прописывать `PATH` не нужно.

Отдельное руководство по установке Vivado с иллюстрациями есть в самом репозитории: docs/vivado_installation_guide.

4.1.3. Установка Gowin EDA (для плат на ПЛИС Gowin)

Почему платы Gowin удобны для учёбы

- **Скорость синтеза.** Пример собирается за секунды, а не за минуты, — на занятии это принципиально: учащийся успевает сделать несколько попыток.
- **Все три операционные системы.** Gowin EDA работает под Windows, Linux и macOS. Для сравнения: Quartus и Vivado под macOS не выпускают вовсе.
- **Цена и комплектация.** Tang Nano 9K с ЖК-экраном и звуковыми модулями обходится примерно в 60 долларов на учащегося — дешевле любой платы Altera или Xilinx сравнимых возможностей.
- **Возможность вырасти.** Код примера переносится на Tang Primer 25K и Tang Mega 138K практически без изменений.

Подробное сравнение: verilog-meetup.com — Can Gowin beat Xilinx and Altera in the educational market?

Установка

Для скачивания нужно зарегистрироваться на сайте производителя. Школа рассчитывает на версию **V1.9.9 Education**.

> ПРИМЕЧАНИЕ
>
> Для плат Tang Primer 25K и Tang Mega 138K (семейства GW5A и GW5AST) версии 1.9.9 недостаточно — эти микросхемы появились позже. Используйте более новую Gowin EDA.

1–2. На приветственном экране нажмите *Next* и примите лицензионное соглашение.

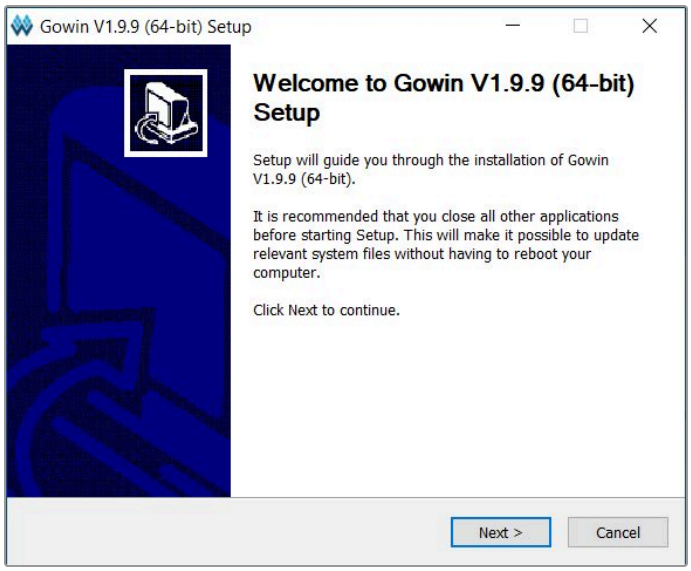


Рис. 28

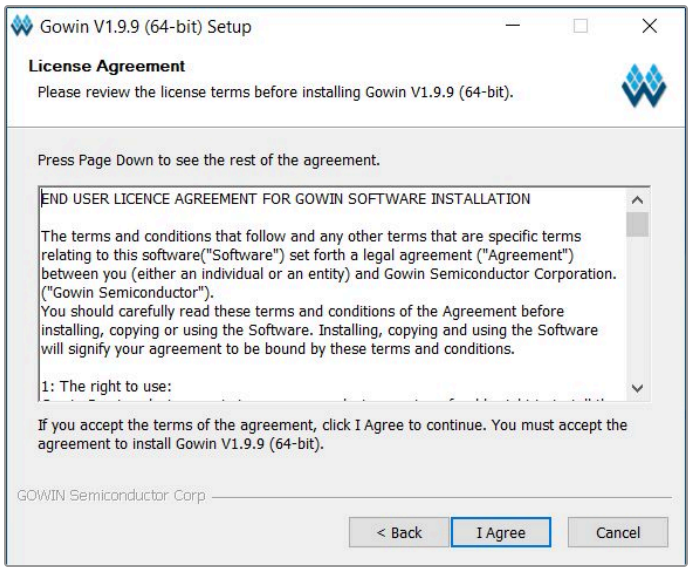


Рис. 29

3–4. Поставьте все галочки, чтобы установить все компоненты. Путь оставьте по умолчанию и нажмите *Install*.

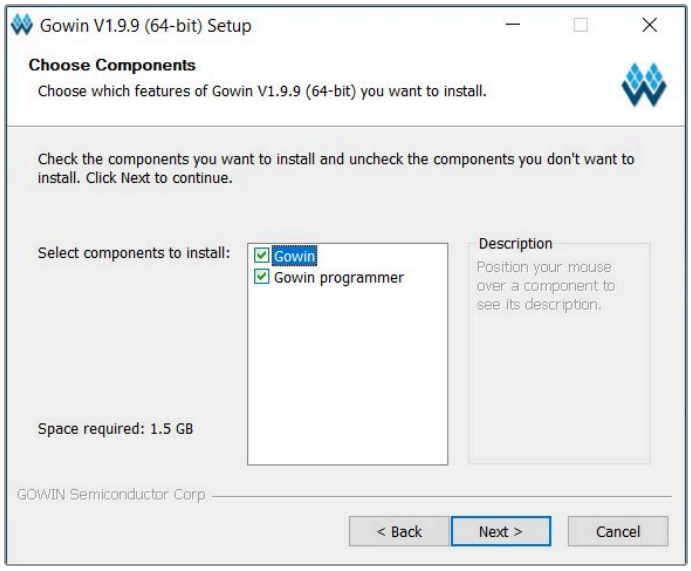


Рис. 30

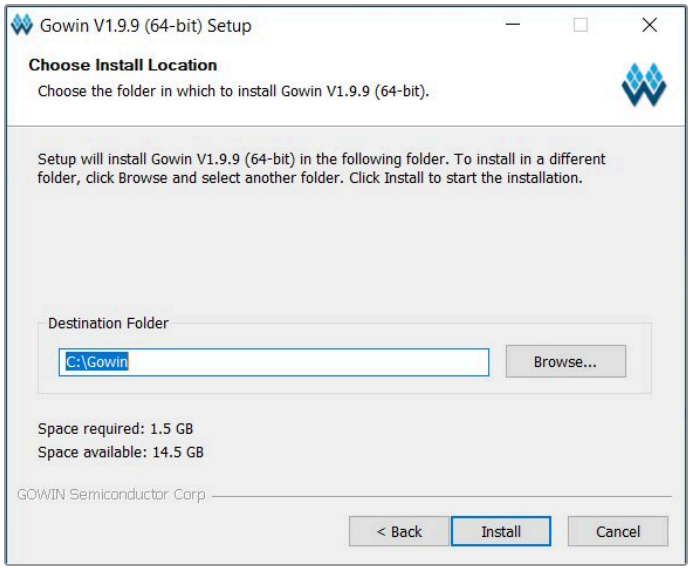


Рис. 31

5–6. По завершении поставьте галочки установки драйверов программатора и нажмите *Finish*. В окне *FTDI CDM Drivers* нажмите *Extract*.

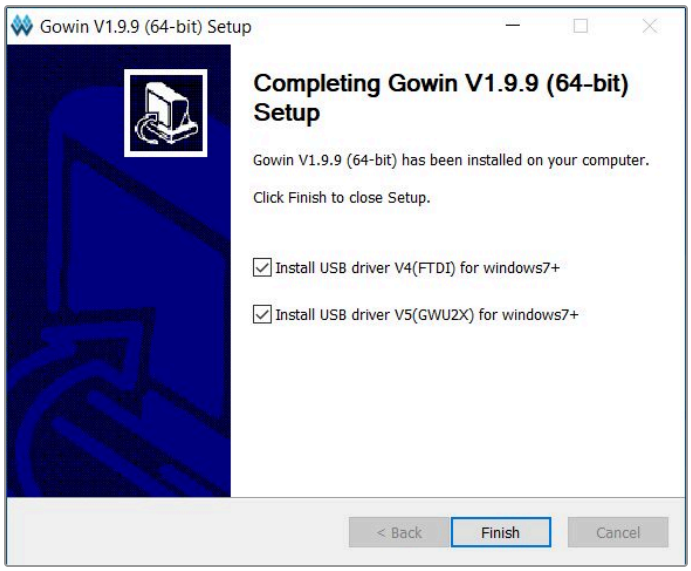


Рис. 32

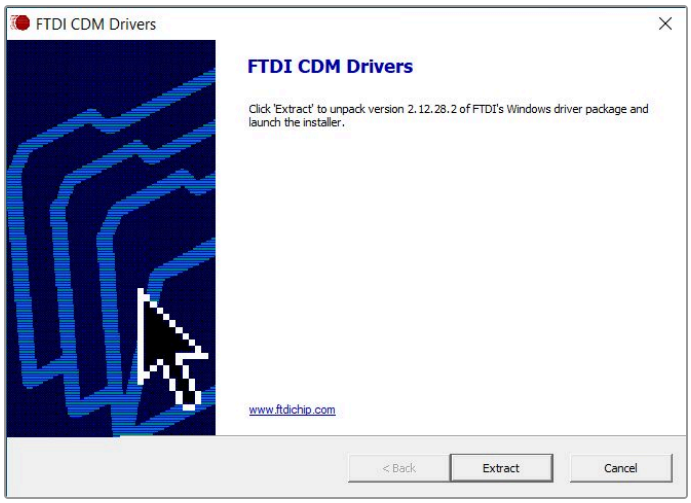


Рис. 33

7–9. Далее → принять соглашение → дождаться окончания → *Готово*.

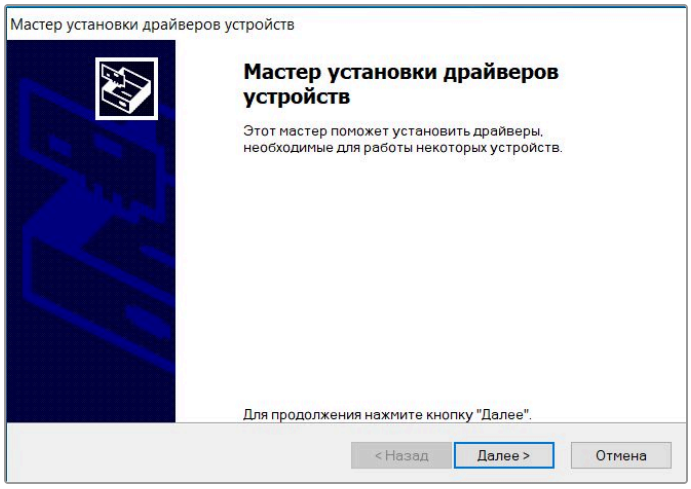


Рис. 34

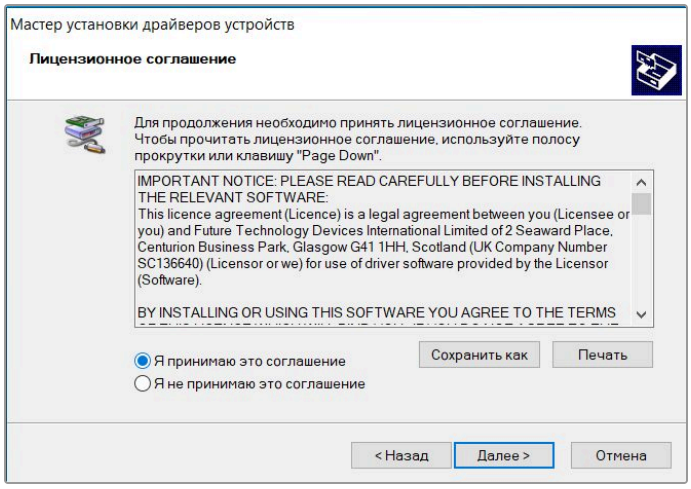


Рис. 35

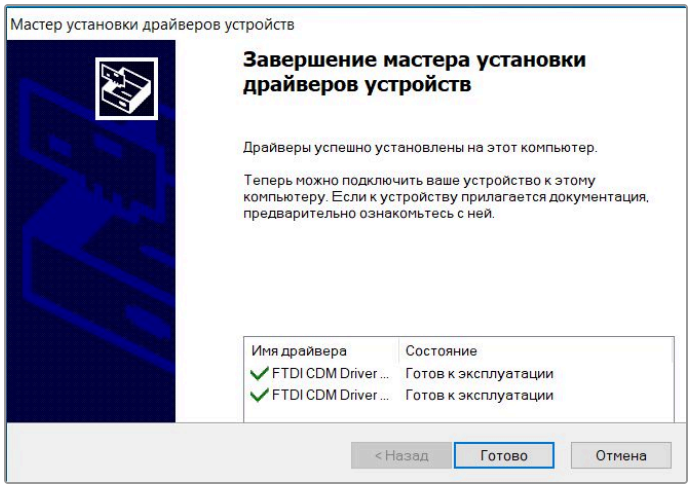


Рис. 36

10–12. В области уведомлений замигает значок щита — нажмите на него, оставьте путь по умолчанию, *Install*, затем *Close*.

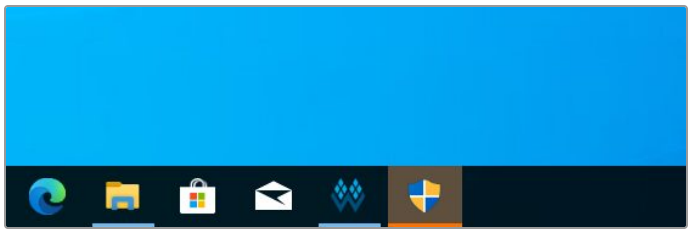


Рис. 37

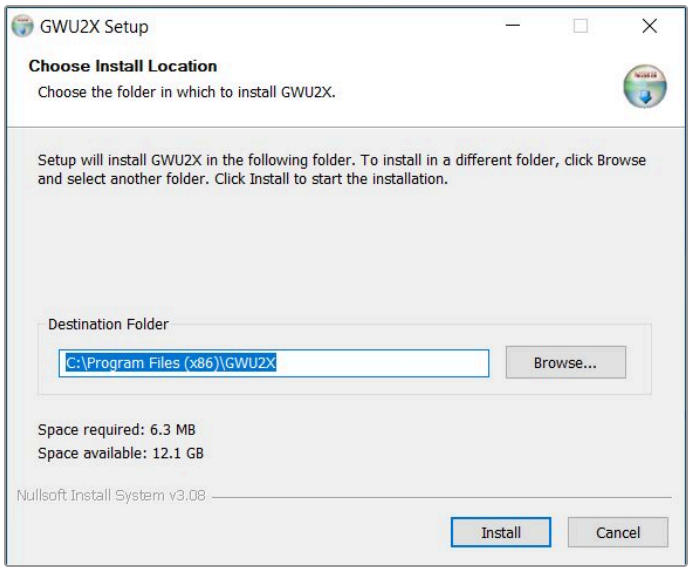


Рис. 38

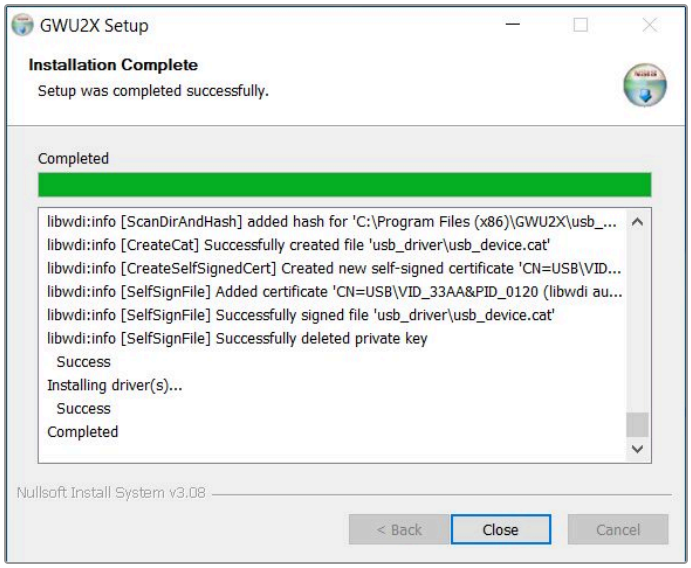


Рис. 39

Как скрипты BGM находят Gowin EDA

Файл `scripts/steps/00_setup_gowin.source_bash` проверяет:

1. `GOWIN_VERSION_DIR` — каталог конкретной версии (внутри него должны быть подкаталоги `IDE` и `Programmer`);
2. `GOWIN_HOME` — каталог, внутри которого лежат версии;
3. домашний каталог, `/opt`, `/tools`, а в них подкаталоги `Gowin` или `gowin`.

Если версий несколько, берётся самая новая (каталоги вида `Gowin_V1.9.9` сортируются, выбирается последний), и выдаётся предупреждение со списком найденного.

4.1.4. OSS CAD Suite — открытый маршрут для Gowin и Lattice

OSS CAD Suite — один архив, в котором собран весь открытый маршрут проектирования: Yosys (синтез), nextpnr (размещение и трассировка), openFPGALoader (прошивка), а также свои сборки Icarus Verilog, GTKWave и Verilator. Это единственный способ работать с платами на ПЛИС **Lattice** (ECP5, iCE40), и альтернативный способ работать с платами **Gowin**.

Установка

1. Скачайте архив для своей системы со страницы выпусков: github.com/YosysHQ/oss-cad-suite-build/releases.
2. Распакуйте его так, чтобы получился каталог `~/oss-cad-suite` — то есть прямо в домашнем каталоге. Скрипты BGM ищут именно по этому пути, а также в `~/Downloads/oss-cad-suite`.

Больше ничего делать не нужно: скрипт `scripts/steps/00_setup_yosys.source_bash` сам найдёт каталог и подключит его командой `source ~/oss-cad-suite/environment`. Прописывать `PATH` вручную не требуется.

Если каталога нет, скрипт ищет `yosys` в системе и, не найдя, выдаёт сообщение со ссылкой на страницу выпусков.

> ПРИМЕЧАНИЕ
>
> Открытый маршрут развивается быстро, и часть примеров репозитория на нём пока работает не полностью — особенно для плат Lattice. Если вы столкнётесь с таким примером, это хорошая тема для самостоятельной работы: Школа рассчитывает на помощь учащихся в отладке таких случаев. Сообщайте о найденном через issues в репозитории.

4.1.5. Установка Icarus Verilog и просмотрщика диаграмм (обязательно)

Icarus Verilog — открытый симулятор, которого достаточно для моделирования всех примеров репозитория. **GTKWave** или **Surfer** — программы для просмотра временных диаграмм в формате VCD.

> НОВОЕ В ЭТОЙ РЕДАКЦИИ
>
> **Нужна версия 13, лучше 14.** Прежняя редакция инструкции описывала версию 12. Скрипты репозитория проверяют версию при запуске: ниже 12 выдаётся ошибка, на 12 — предупреждение с рекомендацией перейти на 13 или 14. Часть примеров использует конструкции SystemVerilog, которые понимает только версия 13 и новее.

Где скачать: bleyer.org/icarus — сборки для Windows. Берите версию **14**.

1–2. Запустите установщик и примите лицензионное соглашение.

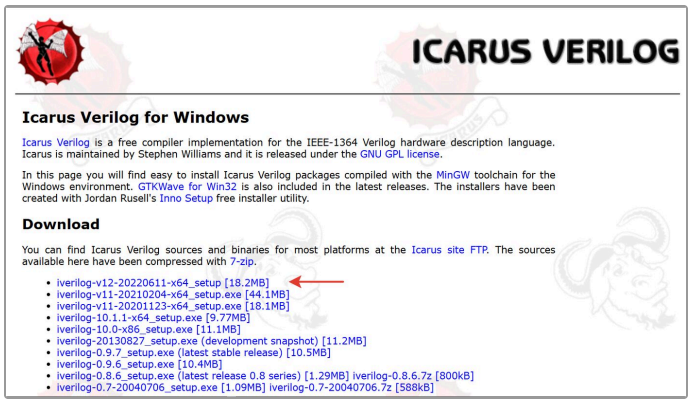


Рис. 40

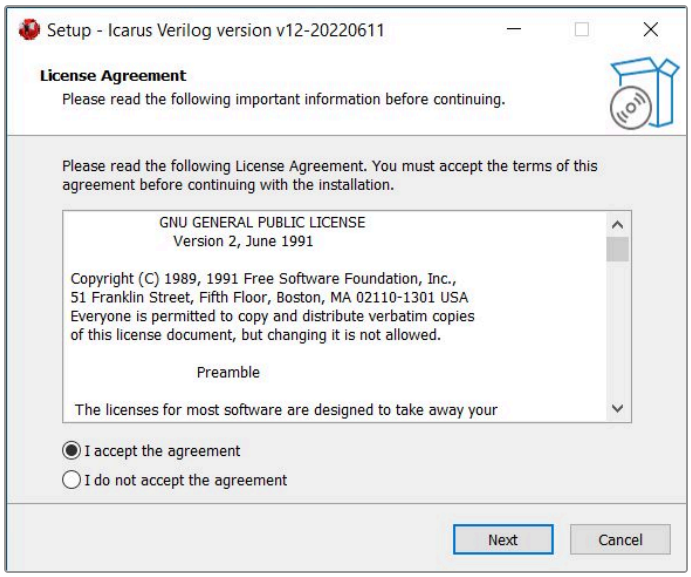


Рис. 41

3–4. Установщик предупреждает, что в пути установки не должно быть пробелов. Путь оставьте по умолчанию (`C:\iverilog`).

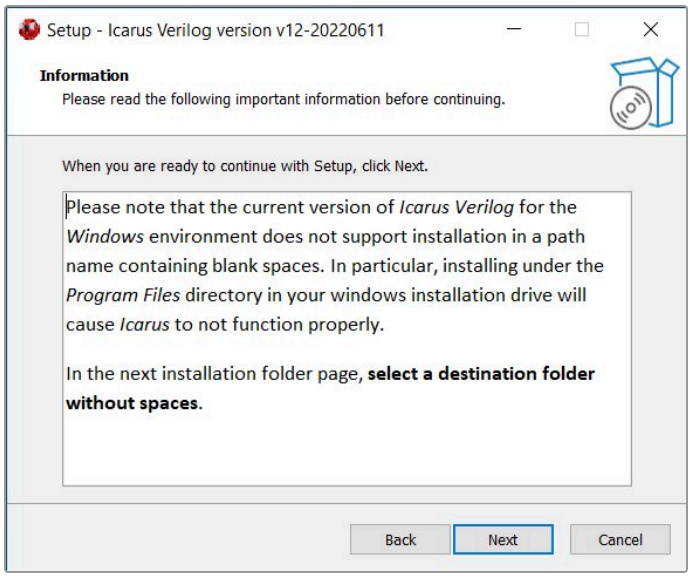


Рис. 42

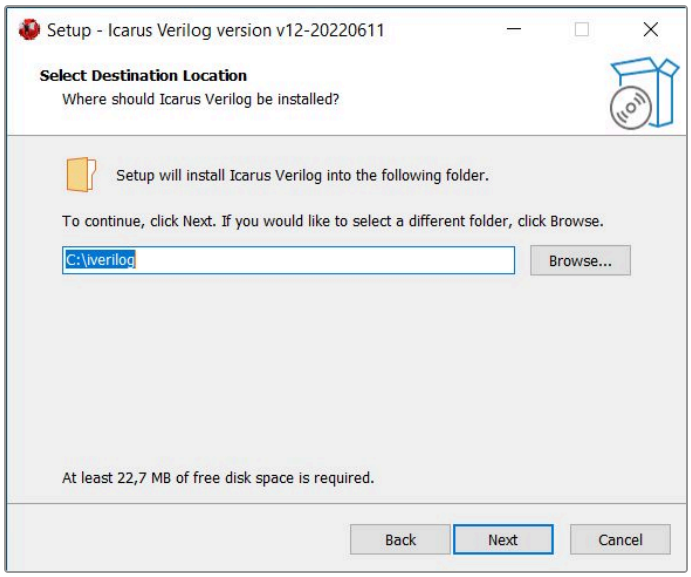


Рис. 43

5–6. Поставьте обе галочки для полной установки — она включает GTKWave. Имя папки в меню «Пуск» оставьте по умолчанию.

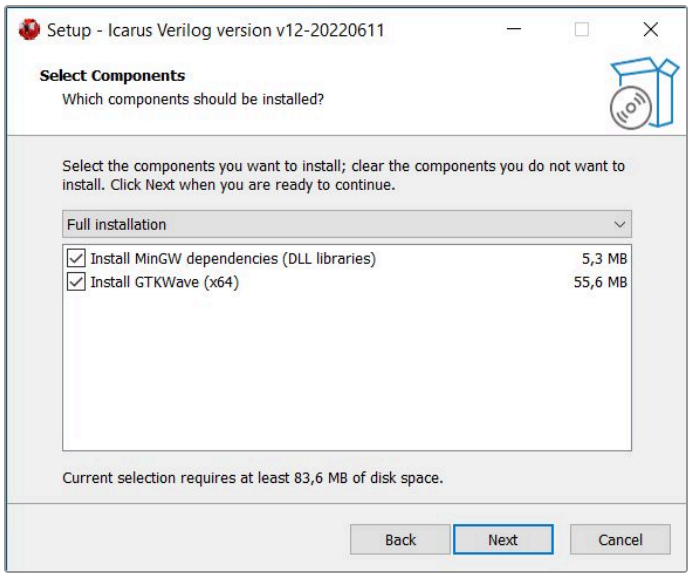


Рис. 44

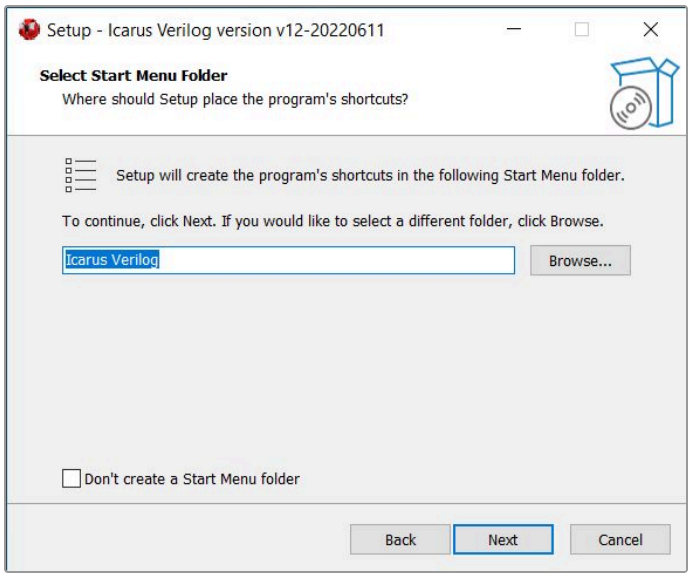


Рис. 45

7–8. Поставьте галочку *Add executable folder(s) to the user PATH* и нажмите *Install*.

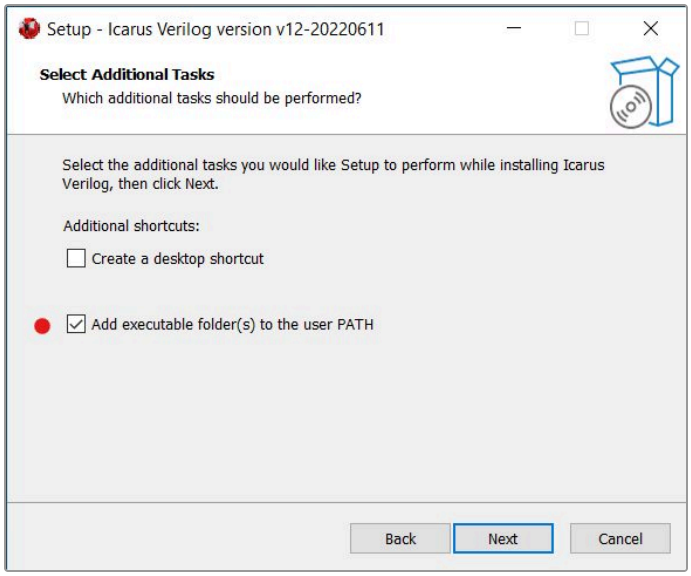


Рис. 46

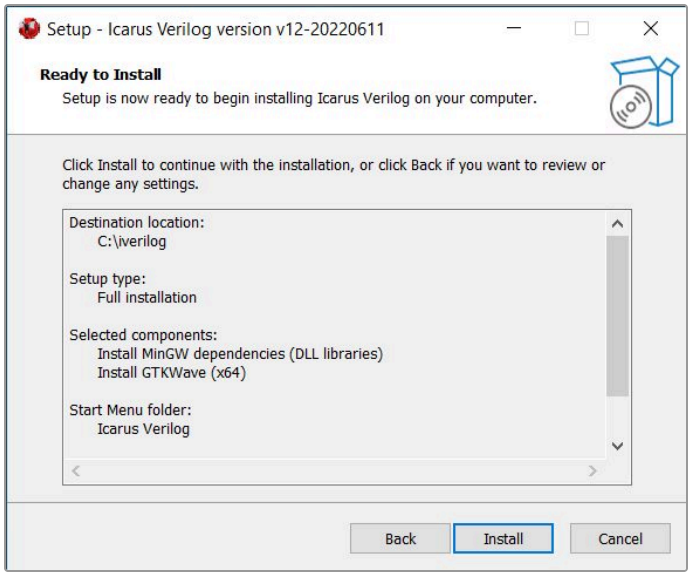


Рис. 47

Как скрипты BGM находят Icarus

Файл `scripts/steps/00_setup_icarus.source_bash` :

1. если `iverilog` доступен в `PATH` — используется он;
2. иначе под Windows проверяется `/c/iverilog` (то есть `C:\iverilog`), под Linux и macOS — `~/install/iverilog` ;
3. затем скрипт запускает `iverilog -V` , разбирает номер версии и предупреждает, если она слишком старая;
4. дополнительно проверяется поддержка `-g2023` — она нужна части примеров.

> НОВОЕ В ЭТОЙ РЕДАКЦИИ

> **Icarus из OSS CAD Suite тоже поддерживается.** Если вы поставили OSS CAD Suite (раздел 4.1.4), отдельная установка Icarus не обязательна: в архиве есть и симулятор, и GTKWave, и скрипты их используют.

Если вместо GTKWave в системе найдена программа `surfer`, скрипты используют её и берут файл настроек `surfer.scr` вместо `gtkwave.tcl`. См. раздел 4.1.7.

4.1.6. Установка Visual Studio Code (обязательно)

1–3. Скачайте установщик с code.visualstudio.com, запустите его и примите лицензионное соглашение.

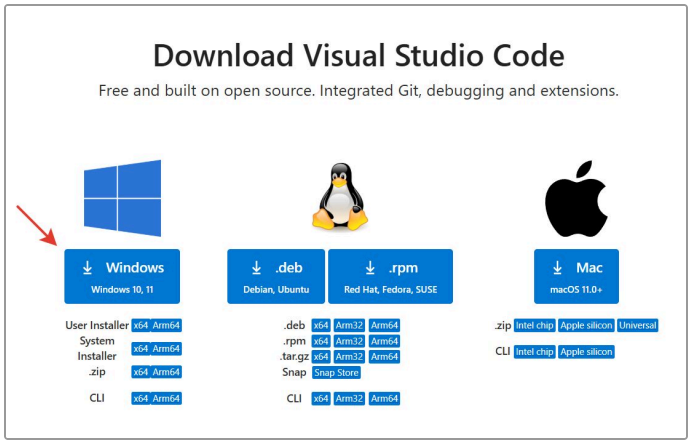


Рис. 48

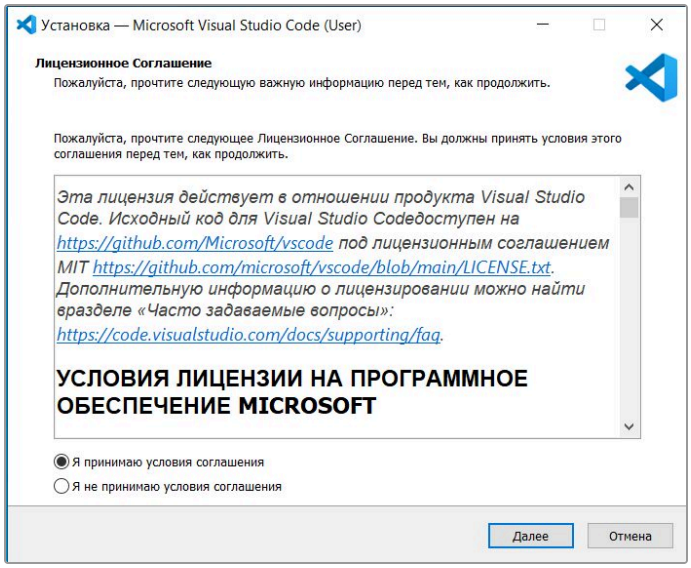


Рис. 49

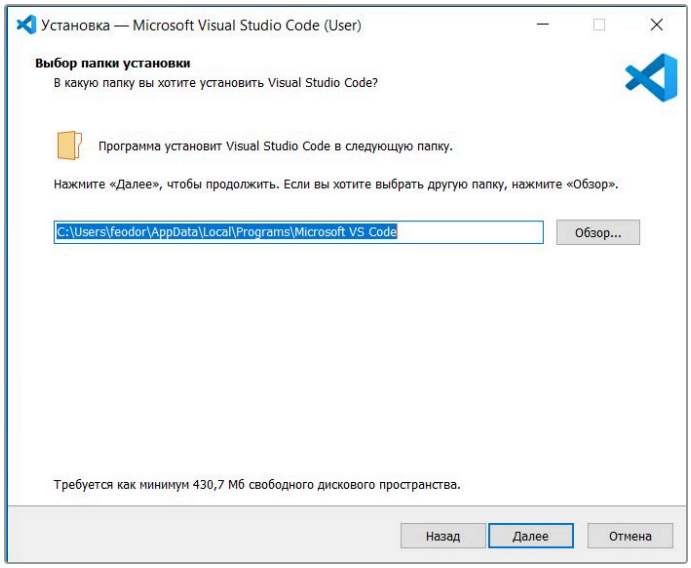


Рис. 50

4–5. Путь установки оставьте по умолчанию. В пути **не должно быть русских букв**. Имя папки в меню «Пуск» оставьте как есть.

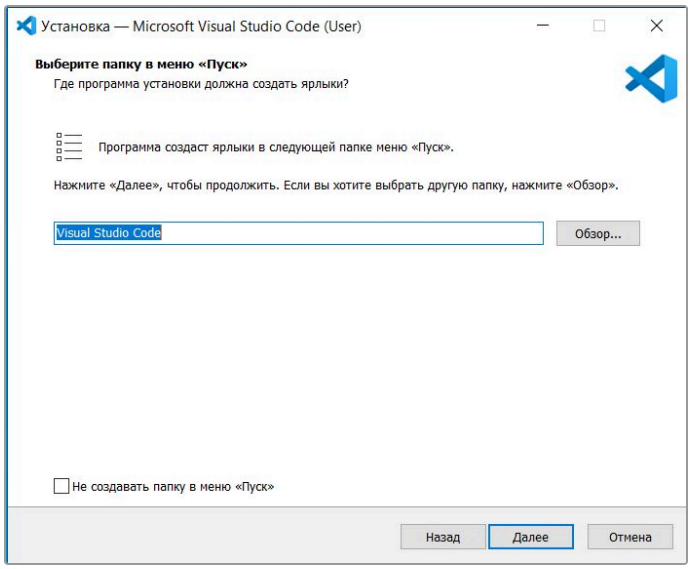


Рис. 51

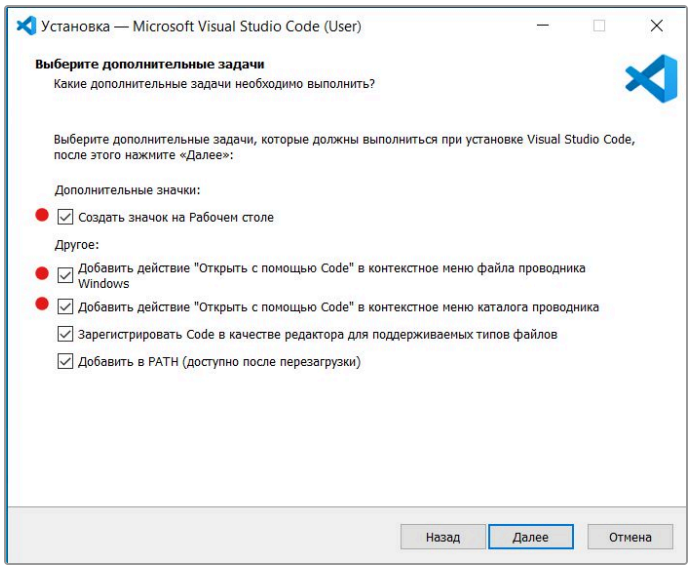


Рис. 52

6–7. Полезно поставить галочки «Создать значок на Рабочем столе» и оба пункта «Открыть с помощью Code» — они добавляют команду в контекстное меню проводника.

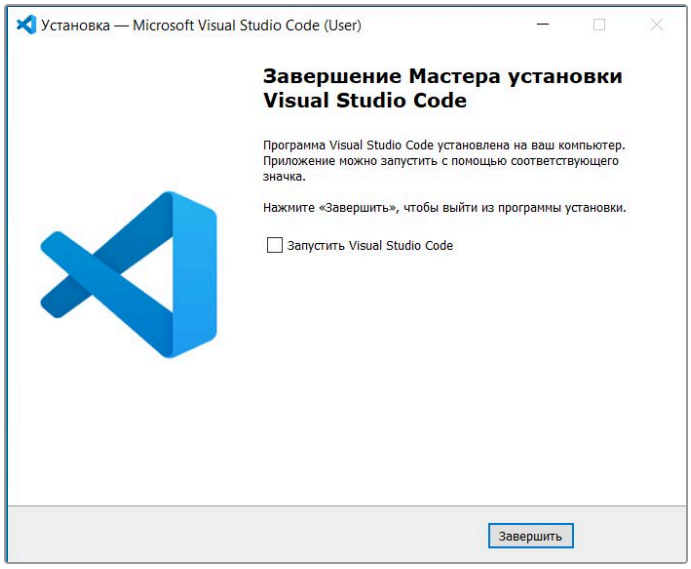


Рис. 53

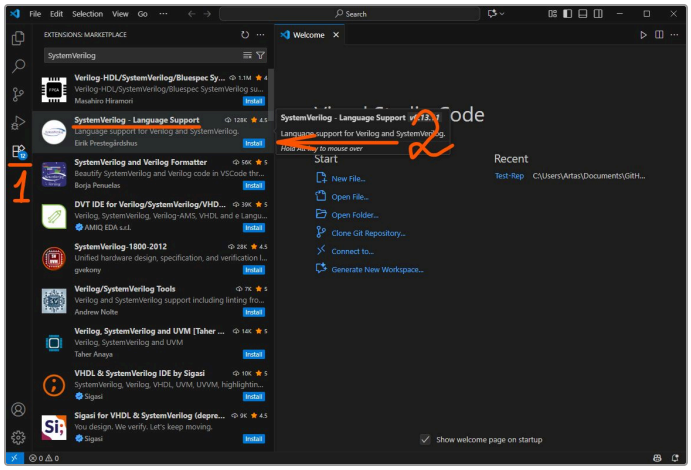


Рис. 54

8. Установите расширение для подсветки синтаксиса: вкладка расширений → в поиске *SystemVerilog* → *SystemVerilog - Language Support* → *Install*.

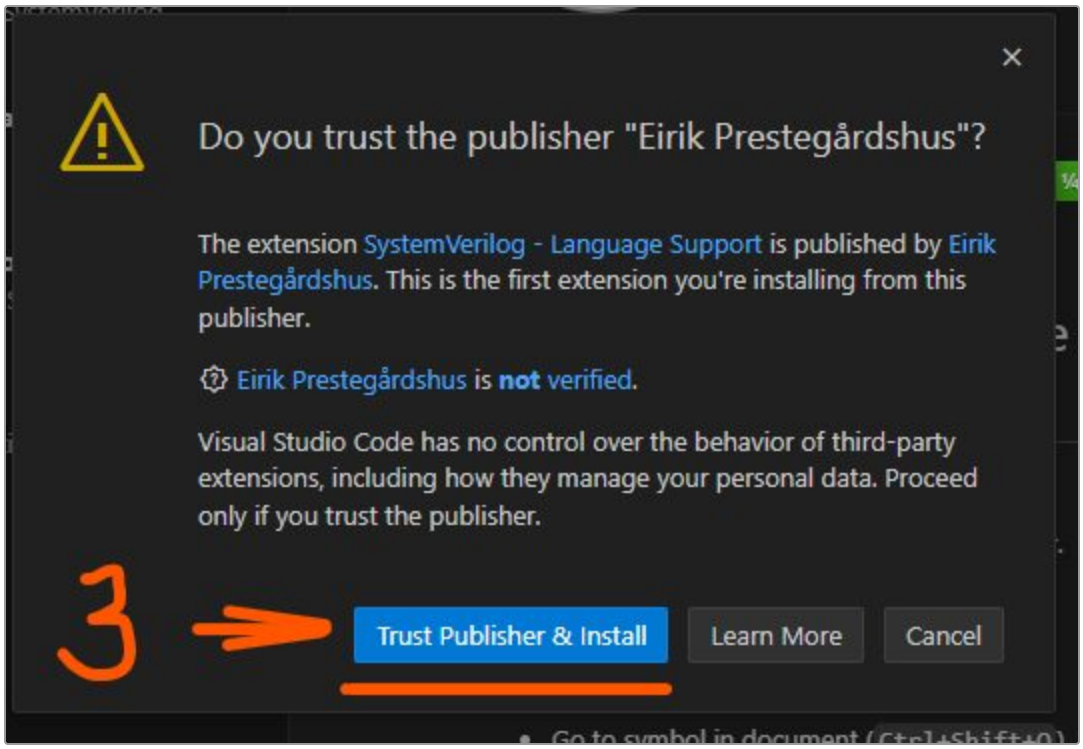


Рис. 55

4.1.7. Установка Surfer — просмотрщика диаграмм

> НОВОЕ В ЭТОЙ РЕДАКЦИИ
>
> Этого раздела в прежней редакции не было. **Surfer** — современная альтернатива GTKWave для просмотра временных диаграмм. Существует в двух видах, и полезны оба.

Расширение для VS Code

Откройте вкладку расширений, найдите *Surfer* и установите. После этого файлы `.vcd` открываются прямо в редакторе, рядом с кодом — не нужно переключаться в отдельное окно.

Отдельная программа

Скачайте сборку со страницы выпусков: gitlab.com/surfer-project/surfer/-/releases, и поместите исполняемый файл в каталог, который есть в `PATH`.

Скрипты репозитория проверяют наличие команды `surfer` и, если она есть, используют Surfer вместо GTKWave автоматически. Для каждого примера в репозитории рядом с `gtkwave.tcl` лежит файл `surfer.scr` с таким же набором отображаемых сигналов.

4.1.8. Установка Git (обязательно)

Git нужен, чтобы скачать репозиторий с примерами и получать обновления. Кроме того, вместе с ним под Windows устанавливается **Git Bash** — оболочка, в которой запускаются все скрипты репозитория.

> ВНИМАНИЕ
>
> Под Windows скрипты BGM запускаются именно из **Git Bash**, а не из `cmd.exe` и не из PowerShell. Это обязательное условие.

1–3. Скачайте установщик с git-scm.com/downloads/win, примите соглашение, следующие три окна оставьте по умолчанию.

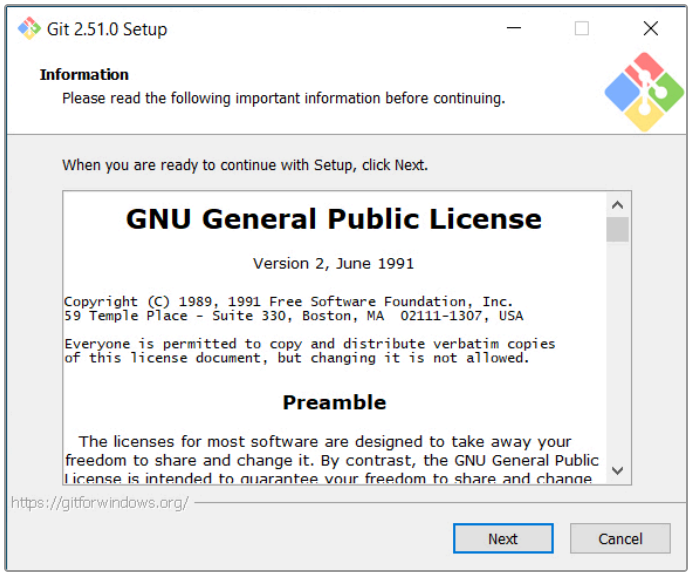


Рис. 56

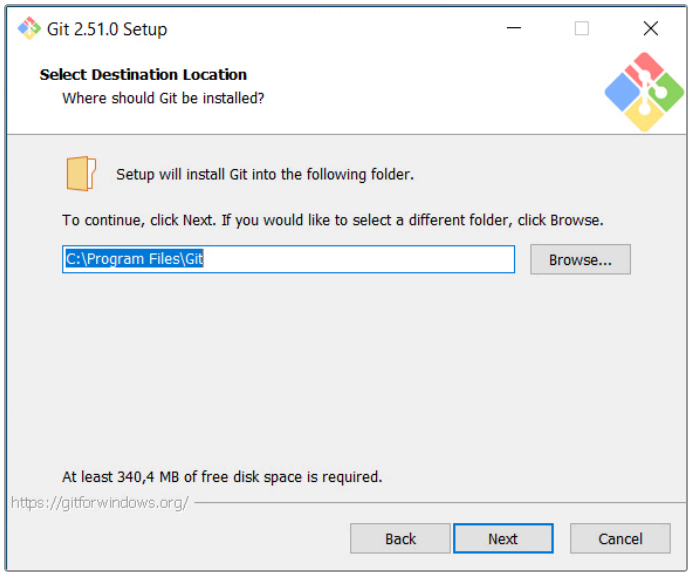


Рис. 57

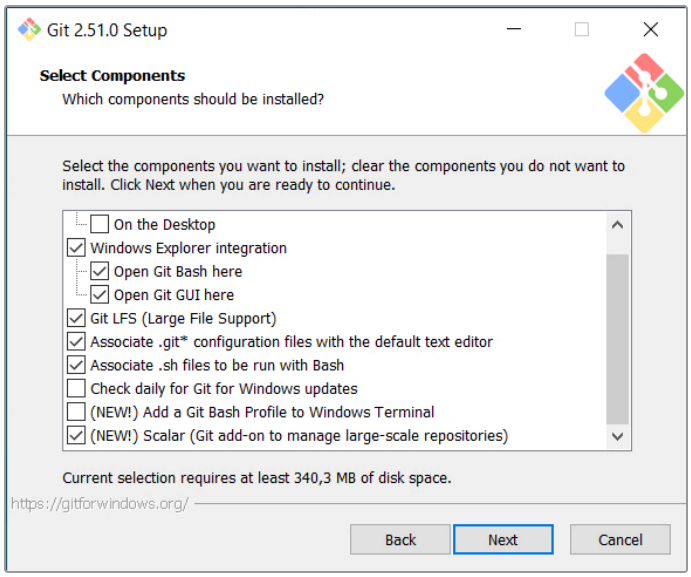


Рис. 58

4. В качестве редактора по умолчанию удобно выбрать из списка *Visual Studio Code*.

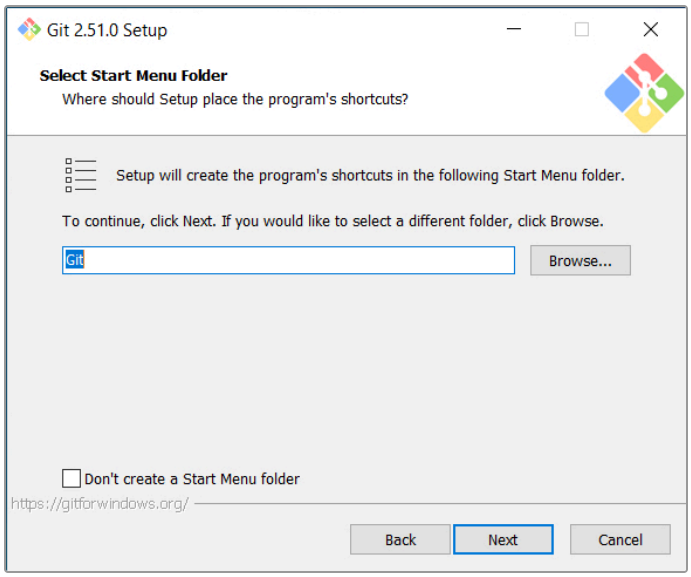


Рис. 59

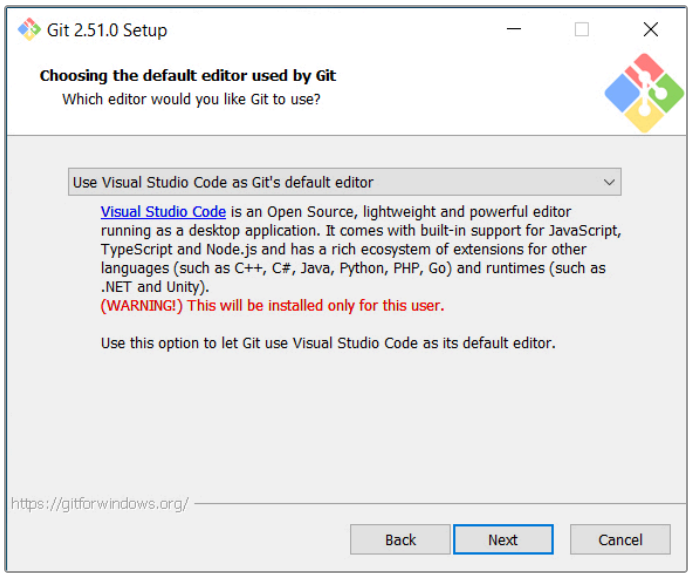


Рис. 60

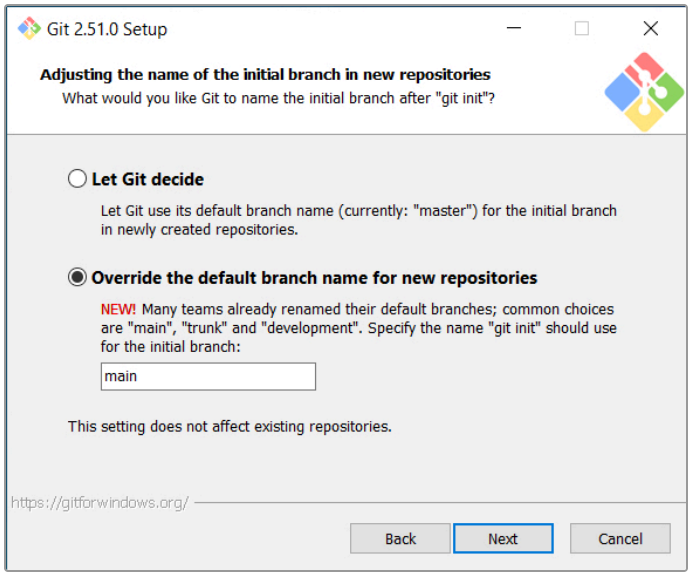


Рис. 61

5. Поставьте флажок *Override the default branch name for new repositories* и оставьте имя `main`.

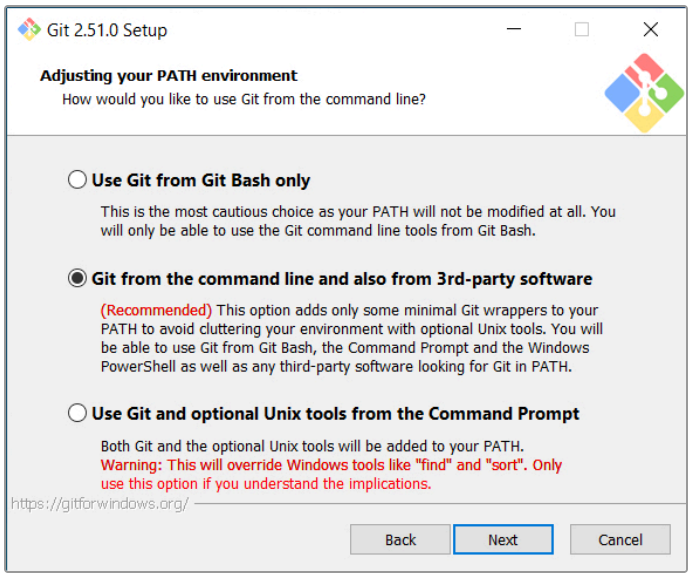


Рис. 62

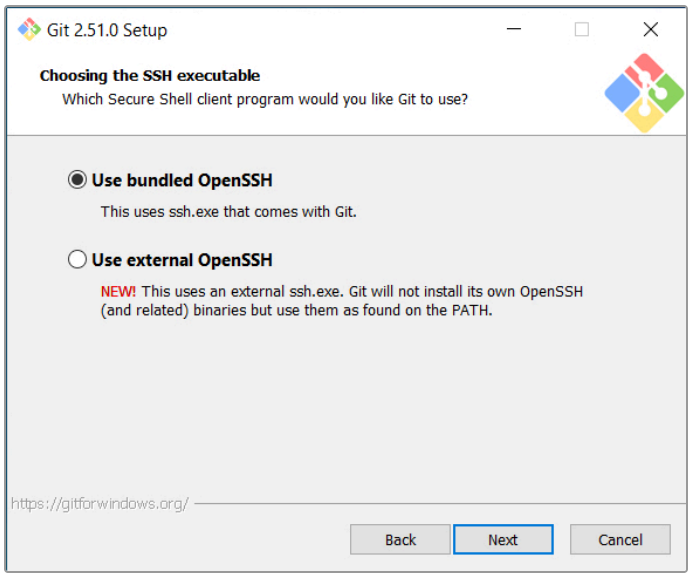


Рис. 63

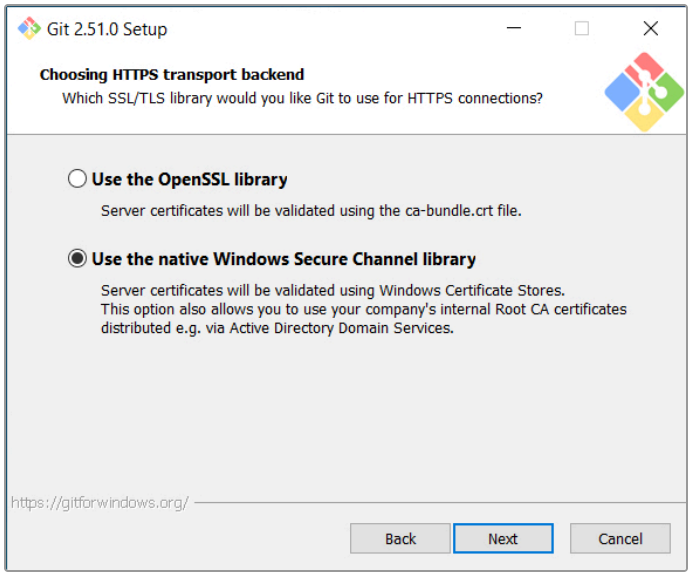


Рис. 64

6. Остальные страницы оставьте по умолчанию и завершите установку.

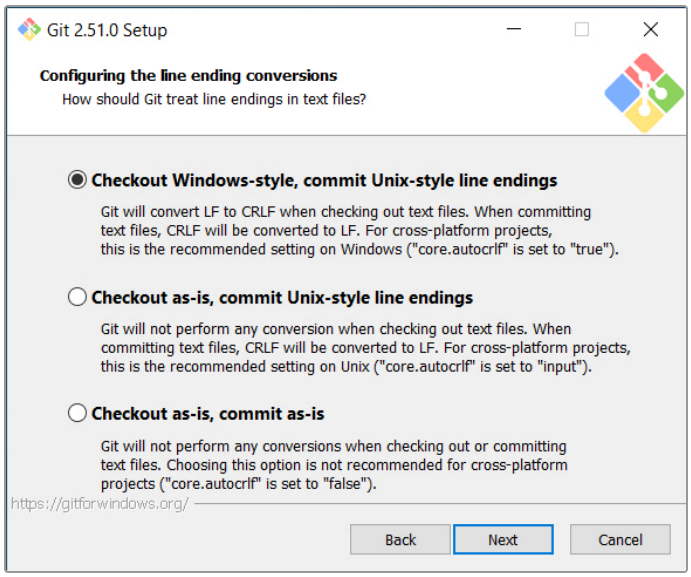


Рис. 65

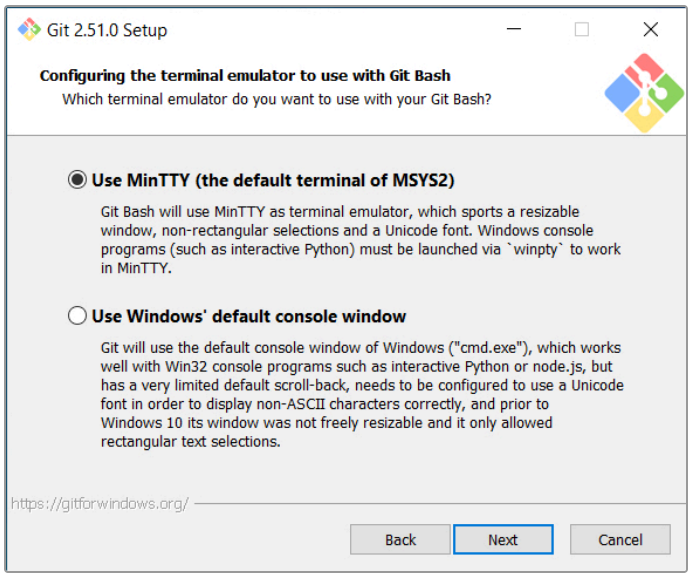


Рис. 66

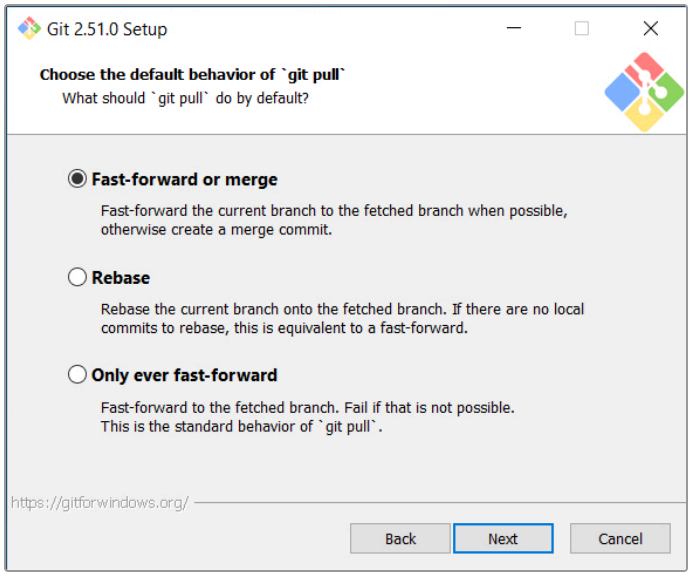


Рис. 67

7. После установки в системе появятся *Git Bash* и *Git CMD*, а в контекстном меню проводника — пункты *Open Git GUI here* и *Open Git Bash here*.

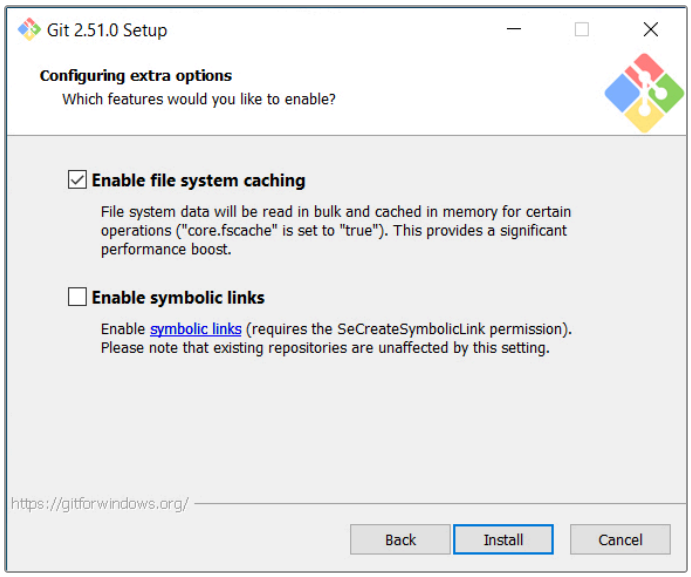


Рис. 68

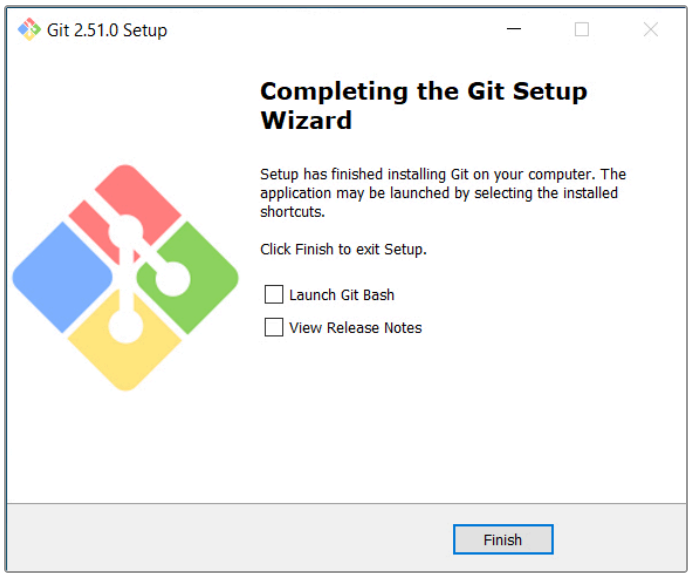


Рис. 69

Получение репозиториев

```
mkdir -p ~/projects
cd ~/projects
git clone https://github.com/chipdesignschool/basics-graphics-music.git
git clone https://github.com/chipdesignschool/systemverilog-homework.git
```

> БЫЛО ВЕРНО РАНЬШЕ
>
> Прежняя редакция предлагала клонировать репозитории с `gitflic.ru`, в том числе `valid-ready-etc`. Эти адреса больше не используются: `valid-ready-etc` устарел, а актуальные репозитории живут на GitHub (см. разделы 1.2 и 1.3). Пункт про `main` вместо `master` в настройках Git теперь существенен именно поэтому — на GitHub основная ветка называется `main`.

4.1.9. Установка RARS (обязательно)

RARS — симулятор системы команд RISC-V. В нём выполняются упражнения по ассемблеру из раздела курса про архитектуру процессора, и его запускает скрипт `10_run_instruction_set_simulator.bash`.

RARS распространяется одним исполняемым jar-файлом, поэтому нужна Java 8 или новее.

1. Скачайте сборку OpenJDK с jdk.java.net и распакуйте, например, в `C:\Program Files\OpenJDK`.

2. Создайте системную переменную `JAVA_HOME` с путём к распакованному каталогу и добавьте в `PATH` значение `%JAVA_HOME%\bin`.

3. Проверьте установку командой в `cmd`:

```
java --version
```

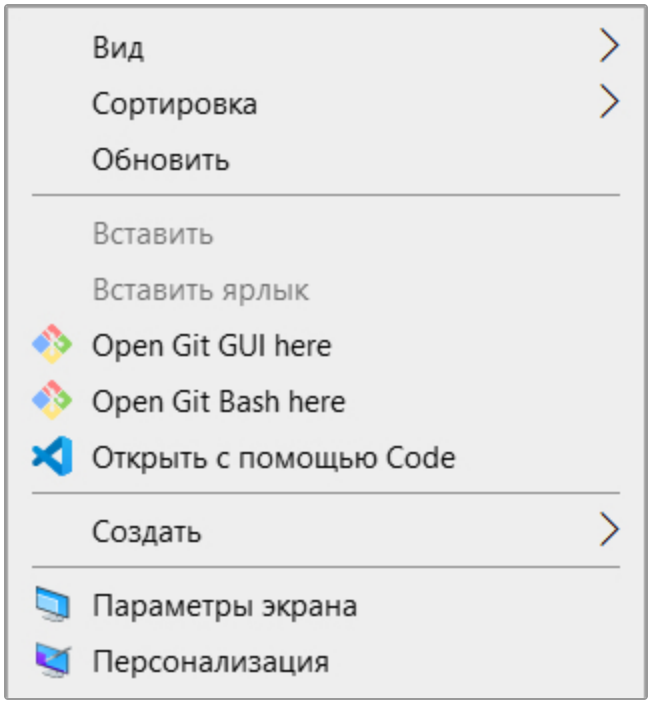


Рис. 70

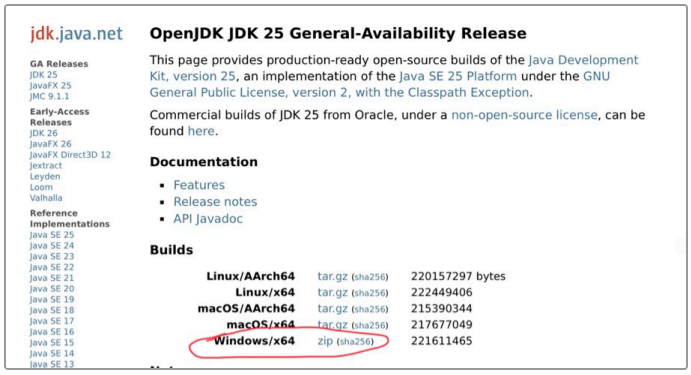


Рис. 71



Рис. 72

4. Скачайте `rars1_6.jar` со страницы выпусков: github.com/TheThirdOne/rars/releases.

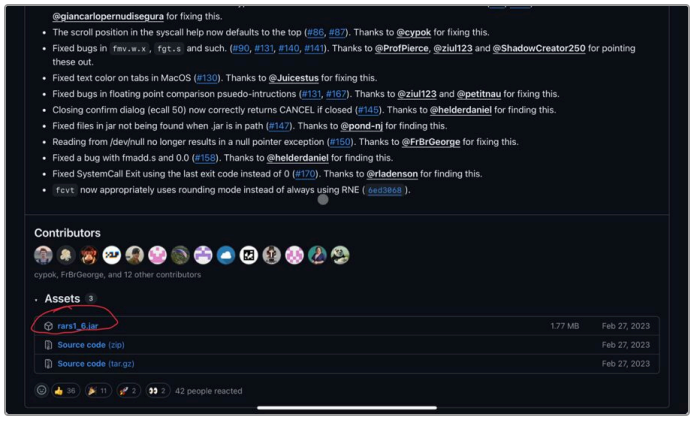


Рис. 73

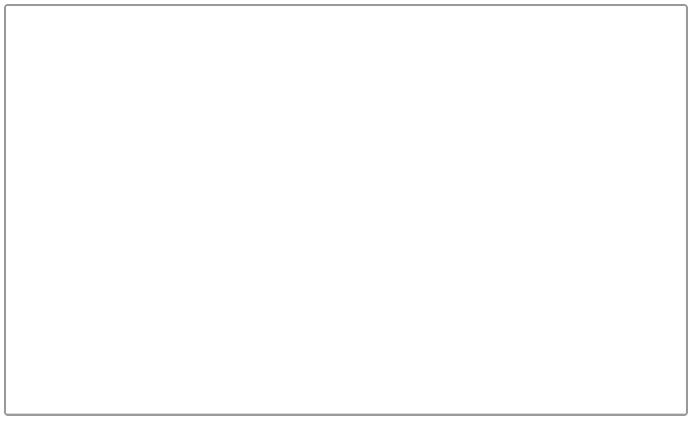


Рис. 74

Скрипт `00_setup_rars.source_bash` ищет jar-файл в домашнем каталоге и в каталоге загрузок; если найдено несколько версий, берётся самая новая.

4.1.10. Установка XPACK RISC-V GCC (для программ, исполняемых ядром внутри ПЛИС)

> НОВОЕ В ЭТОЙ РЕДАКЦИИ
>
> Этого раздела в прежней редакции не было. Он нужен для раздела курса, где в ПЛИС синтезируется процессорное ядро, а на нём исполняются программы на C и ассемблере.

В репозитории есть три процессорных ядра RISC-V, которые можно синтезировать в ПЛИС:

- `aps` — ядро из МИЭТ;
- `yrv` — компактное ядро с отладочным интерфейсом;
- `picorv32` — известное небольшое ядро.

Выбор ядра — скриптом `13_choose_another_riscv_core_for_software.bash` .

Чтобы собрать программу для такого ядра, нужен кросс-компилятор. В репозитории используется сборка **XPACK RISC-V GCC** (`riscv-none-elf-gcc`).

Установка

1. Скачайте архив со страницы выпусков: github.com/xpack-dev-tools/riscv-none-elf-gcc-xpack/releases.
2. Распакуйте его в домашний каталог. Должен получиться каталог вида `~/xpack-riscv-none-elf-gcc-14.2.0-3/` , внутри которого есть подкаталог `bin` .

Скрипт `00_setup_riscv.source_bash` ищет каталоги с именем вида `xpack-riscv-none-elf-gcc-*` на глубину до двух уровней от домашнего каталога, а также каталог `xpack/bin` . Если найдено несколько версий, выдаётся предупреждение и берётся одна из них.

Сборка и загрузка программы:

```
./11_build_software_to_run_on_cpu.bash
./12_upload_software_to_the_board_using_uart.bash
./14_run_terminal_program.bash
```

Программа загружается в плату через UART, поэтому вместе с компилятором понадобится терминальная программа — см. следующий подраздел.

4.1.11. Установка терминальной программы (для упражнений с UART)

Терминальная программа нужна не сама по себе: через неё выполняются упражнения, в которых плата разговаривает с компьютером по **UART** — последовательному каналу поверх того же кабеля USB, которым плата подключена. Без терминала эти упражнения просто нечем проверить: светодиоды и индикатор здесь ничего не показывают.

Упражнения лежат в каталоге `labs/8_unfinished/8_1_uart` и идут от простого к сложному:

УПРАЖНЕНИЕ	ЧТО ДЕЛАЕТ ПЛАТА	ЧТО НАБИРАТЬ И ГДЕ СМОТРЕТЬ
`8_1_1_uart_7segment`	Приёмник `uart_receiver` складывает последние принятые байты в сдвиговый регистр и показывает их на семисегментном индикаторе. Обратной передачи нет: `uart_tx` держится в единице	Набирайте **заглавные A…F** — на индикаторе появятся шестнадцатеричные цифры A…F. Это *не* универсальный декодер ASCII: в коде стоит `rx_buf[i] - 8'h41 + 8'hA`, то есть вычитается код буквы «A», и цифры 0…9 покажут бессмыслицу. Строка для низкого полубайта в исходнике есть, но закомментирована
`8_1_2_uart_echo`	Принятый байт сразу отправляется назад (`uart_transceiver`)	Набирайте что угодно **с выключенным локальным эхом** — ответ платы и есть проверка канала. Если каждый символ виден дважды, эхо осталось включённым, и вы смотрите на свой же ввод
`8_1_3_uart_calc`	Калькулятор с обратной польской записью: `ascii2tok` разбирает символы на числа и операции, `fifo` и `lifo` держат поток и стек, `calc` считает. Операции — `+ - * & | ^`, числа 16-битные	Результат выводится **на семисегментный индикатор** в шестнадцатеричном виде, а не в терминал: передатчик здесь тоже выключен. Самая короткая проверка — `2 3 +` и один `Enter`: на индикаторе должно появиться 5

Выражения для последнего упражнения записываются в постфиксной форме — сначала числа, потом операция; числа разделяются пробелами. Конец выражения — **один** возврат каретки или перевод строки: в `ascii2tok` проверяется `8'h0D` *или* `" "`. Если терминал отправляет CR+LF, выражение может быть обработано как два конца строки. Эти три строки лежат в самом каталоге упражнения (`mnt.txt`) и годятся как проверка посложнее:

```
3 4 + 2 - 10 3 - +          → 12   (0x000C)
3 5 + 7 2 - &               → 0    (0x0000)
100 50 * 200 - 30 20 * +    → 5400 (0x1518)
```

Все три упражнения работают на скорости **115200** бод, 8 бит данных, без контроля чётности, один стоповый бит, без управления потоком — это и есть настройки, которые нужно задать терминалу.

> ВНИМАНИЕ
>
> **Последовательный порт появляется не от любого кабеля.** У плат, где программатор построен на микросхеме FTDI (Tang и подобные), UART идёт по тому же кабелю USB. Но если плата прошивается отдельным JTAG-программатором — например USB Blaster, — последовательного порта он не даёт: нужен либо UART самой платы, либо отдельный переходник USB–UART, подключённый TX к RX, RX к TX и общей землёй. Уровни сигналов и номера выводов смотрите в документации платы.

> ПРИМЕЧАНИЕ
>
> Готовая обёртка `14_run_terminal_program.bash` лежит только в каталоге `8_1_1_uart_7segment`. Для `8_1_2_uart_echo` и `8_1_3_uart_calc` терминал после прошивки запускается вручную — командой из подраздела ниже.

> ПРИМЕЧАНИЕ
>
> Каталог называется `8_unfinished` — «незаконченные». Это значит, что упражнения ещё дорабатываются и могут измениться; собираются и запускаются они обычными скриптами, как и остальные.

Тот же терминал понадобится и дальше, для упражнений с процессорными ядрами RISC-V из каталога `labs/5_cpu`. Там плата не просто принимает байты: в ПЛИС синтезируется процессор, программа для него собирается компилятором XPACK и загружается в плату **через UART** скриптом `12_upload_software_to_the_board_using_uart.bash`, а работу программы вы наблюдаете в том же терминале. Так устроены и примеры с многопоточностью — `labs/5_cpu/program/10_rr_threads`.

Про то, как устроена эта инфраструктура и как в неё была встроена процессорная система RISC-V из МИЭТ, подробно написано в статье *«Испытание абстракций: как чужая процессорная система RISC-V переехала на десятки отладочных плат»*: habr.com/ru/articles/1091540 (есть и английская версия).

Какой терминал нужен

Скрипт `14_run_terminal_program.bash` умеет работать с тремя терминалами: **minicom**, **picocom** и **putty**. Он сам находит установленные, предлагает выбрать порт и запускает терминал с нужной скоростью. Под Windows из этих трёх ставится PuTTY — его и описываем.

1. Узнать номер COM-порта

Откройте *Диспетчер устройств* (Win+X, или через поиск у кнопки «Пуск») и раскройте раздел «**Порты (COM и LPT)**». Плата, подключённая кабелем USB, появляется там как последовательный порт с номером вида `COM3`.

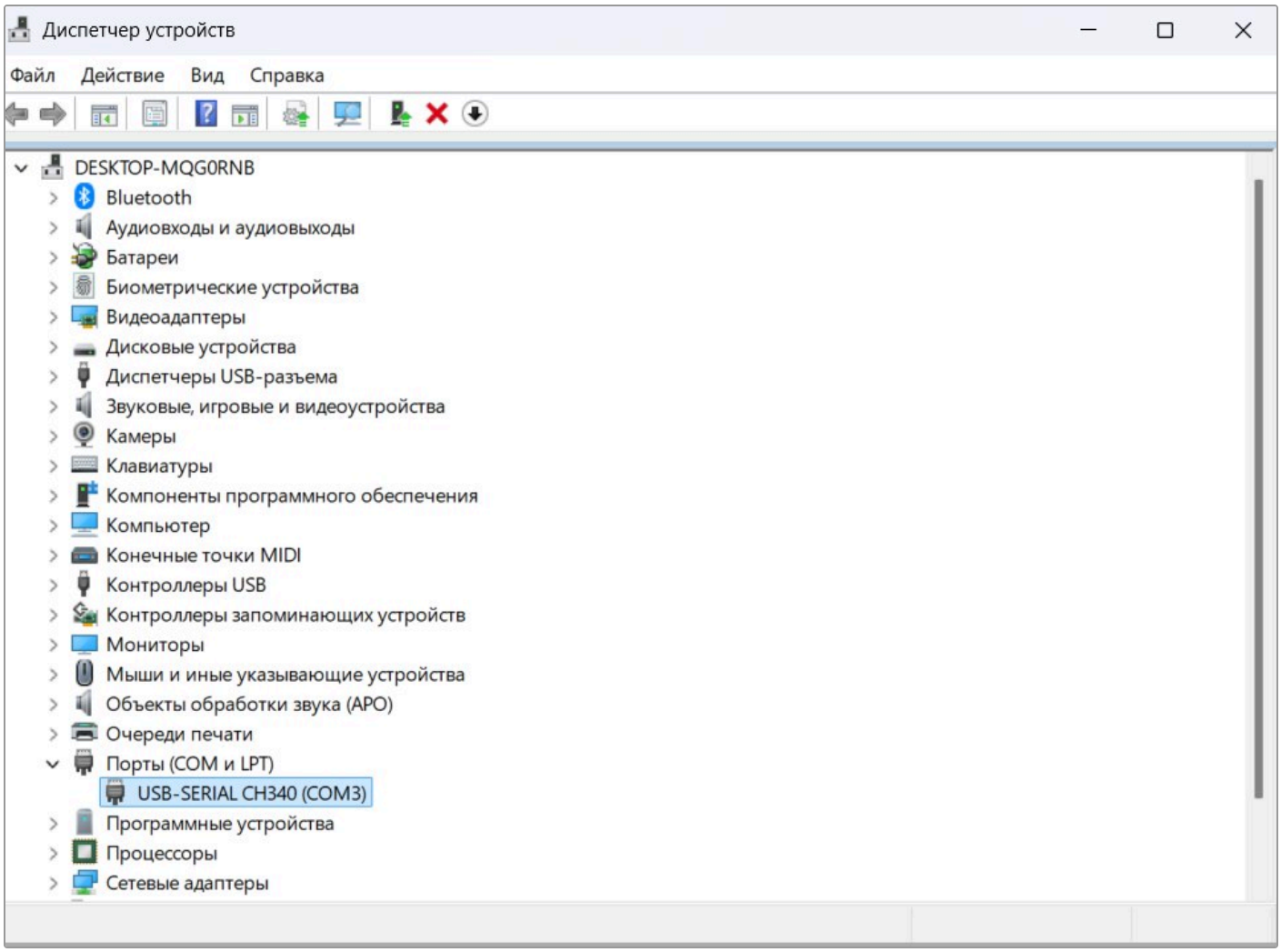


Рис. 75. Раздел «Порты (COM и LPT)» в Диспетчере устройств: здесь виден номер порта, который нужно будет указать терминалу

> **ПРИМЕЧАНИЕ**
>
> Если в разделе ничего нет, а плата подключена: проверьте, что установлен драйвер программатора (подразделы 4.1.1.2 для плат Altera и 4.1.3 для плат Gowin), и попробуйте другой разъём USB. Номер порта меняется при переподключении к другому разъёму, поэтому смотреть его стоит каждый раз.

> **НОВОЕ В ЭТОЙ РЕДАКЦИИ**
>
> Номер порта можно узнать и из командной строки, без Диспетчера устройств: команда `mode` без аргументов перечисляет порты, а `mode comN` отвечает ошибкой для несуществующего номера. В репозитории для этого лежит готовый файл `labs/8_unfinished/8_1_uart/to_integrate/check_which_port_is_used.bat`, который перебирает номера подряд.

2. Установить PuTTY

1. Скачайте установщик с официальной страницы [страницы загрузки PuTTY](#) — для современного компьютера 64-битный пакет MSI.

> БЫЛО ВЕРНО РАНЬШЕ
>
> В документе `terminal_installation.docx`, из которого взят следующий снимок экрана, указан сторонний сайт `putty.org.ru`. Снимок оставлен как источник, но качать лучше с сайта разработчика: так вы получаете подписанный установщик и текущую версию.

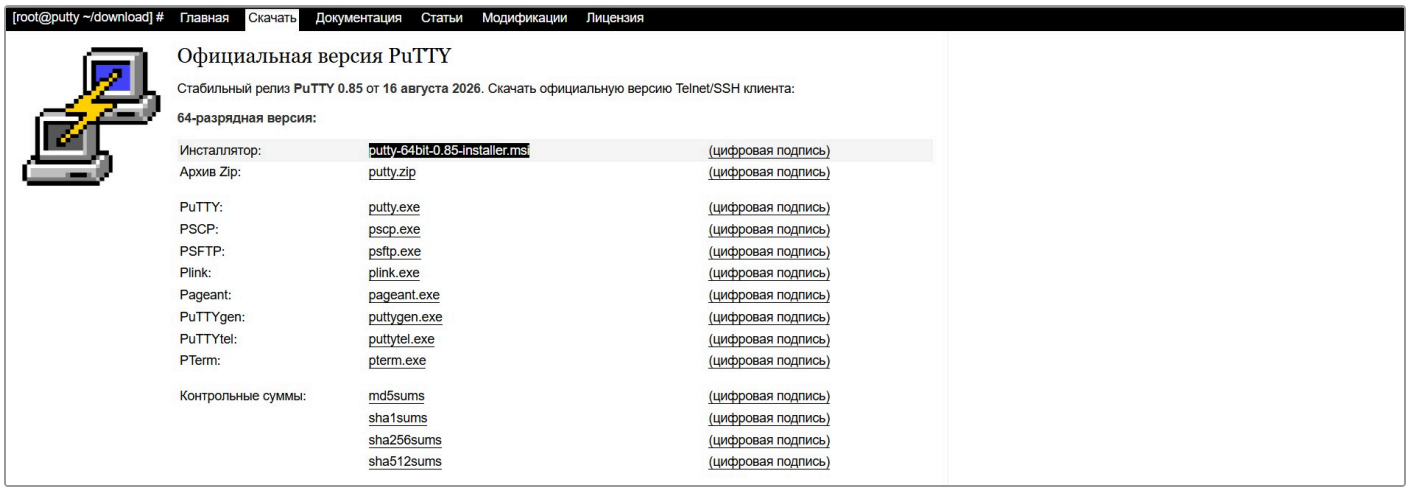


Рис. 76

2. Запустите установщик. На шаге выбора компонентов включите два пункта:

- *Add shortcut to PuTTY on the Desktop* — ярлык на рабочем столе;
- *Put install directory on the PATH for command prompts* — добавление каталога в `PATH`. Без этого пункта `14_run_terminal_program.bash` не найдёт PuTTY, и запускать его придётся вручную.

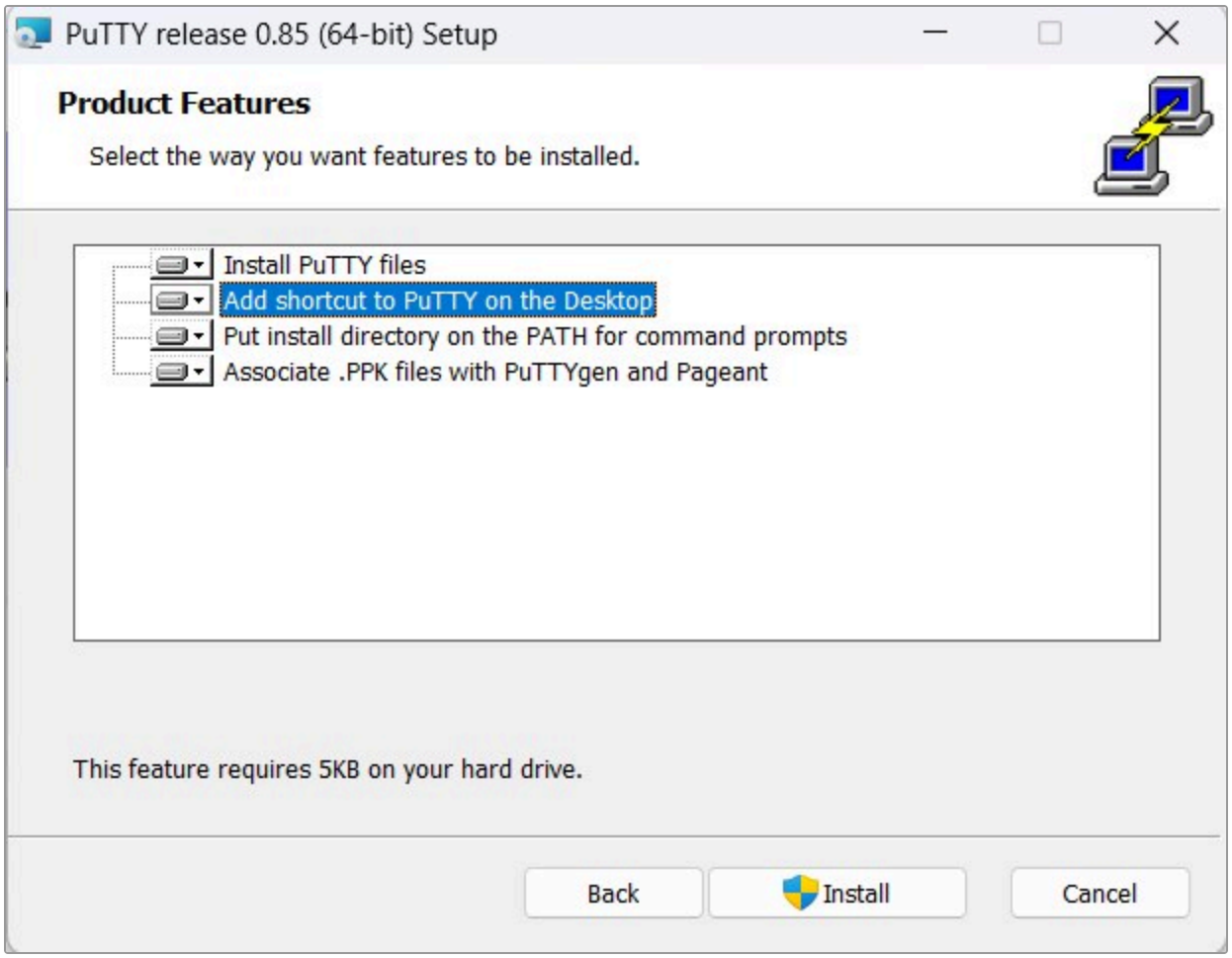


Рис. 77

3. Проверить

Проще всего — через скрипт упражнения, он сам подставит скорость:

```
cd labs/8_unfinished/8_1_uart/8_1_1_uart_7segment
./03_synthesize_for_fpga.bash
./14_run_terminal_program.bash
```

Вручную то же самое делается в графическом окне PuTTY: тип соединения *Serial*, строка *Serial line* — `COM3` (ваш номер), *Speed* — `115200`. Остальные параметры (8 бит данных, без чётности, 1 стоповый бит, без управления потоком) PuTTY использует по умолчанию.

Скорость и формат порта можно задать и командой `mode` — именно так это сделано в `to_integrate/drive_uart.bat` из каталога упражнения:

```
mode com3 baud=115200 parity=n data=8 stop=1 to=off xon=off odsr=off octs=off dtr=off
```

ВНИМАНИЕ

> Пока терминал держит порт открытым, прошить плату не получится: порт занят. Закройте терминал перед `03_synthesize_for_fpga.bash` или `04_configure_fpga.bash`.

4.1.12. Установка LibreLane (синтез в заказную микросхему)

> БЫЛО ВЕРНО РАНЬШЕ
>
> Прежняя редакция описывала **OpenLane**, устанавливаемый через Docker, и этот раздел нужно считать полностью устаревшим. В репозитории теперь используется **LibreLane** — продолжение того же проекта, которое ставится через **Nix**.

LibreLane — открытый маршрут проектирования, превращающий код на SystemVerilog в топологию микросхемы (RTL-to-GDSII). Работает только под Linux и macOS; под Windows нужен WSL или виртуальная машина с Linux (см. раздел 4.2.12 и файл docs/wsl.md репозитория).

Установка через Nix

```
# установить менеджер пакетов Nix
curl --proto '=https' --tlsv1.2 -sSf -L https://install.determinate.systems/nix | sh -

# в каталоге с LibreLane
nix develop
```

Внутри оболочки `nix develop` команда `librelane` доступна, и скрипты репозитория её найдут.

Если LibreLane стоит в другом месте

Укажите путь к каталогу с исполняемым файлом в переменной `LIBRELANE_PATH`:

```
export LIBRELANE_PATH=/путь/к/librelane/bin
```

Скрипт `00_setup_libre_lane.source_bash` добавит этот путь в `PATH`. Если команда `librelane` так и не найдена, выдаётся сообщение с обоими способами установки.

Запуск

```
./07_synthesize_for_asic.bash                  # синтез в СБИС
./08_visualize_asic_synthesis_results_1.bash   # просмотр топологии
./09_visualize_asic_synthesis_results_2.bash
```

Просмотрщик выбирается переменной `LAYOUT_VIEWER` и может быть `openroad` (по умолчанию) или `klayout`.



Рис. 78



Рис. 79

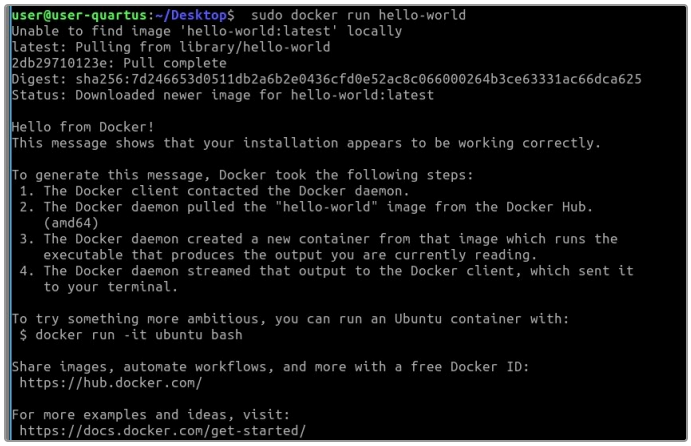


Рис. 80

Снимки экрана из прежней редакции, относящиеся к OpenLane. Оставлены для сравнения: сам маршрут заменён на LibreLane.

4.2. Linux (на примере Ubuntu 22.04)

Порядок тот же, что и для Windows, и нумерация подразделов совпадает. Там, где шаги не отличаются, они не повторяются — даётся ссылка на соответствующий подраздел 4.1.

> ПРИМЕЧАНИЕ
>
> Под Linux скрипты запускаются в обычном терминале — отдельная оболочка, как Git Bash под Windows, не нужна.

4.2.1. Установка Quartus (для плат на ПЛИС Altera)

Quartus нужен только для плат на ПЛИС Altera. Если ваша плата на Xilinx, Gowin или Lattice — переходите к подразделам 4.2.2–4.2.4.

4.2.1.1. Пошаговая установка Quartus Prime Lite 21.1

1. Скачайте установщик `QuartusLiteSetup-21.1.0.842-linux.run` и архив `.qdz` для вашей микросхемы (см. таблицу в подразделе 4.1.1.1), положив их в одну папку.

2. В терминале:

```
chmod +x QuartusLiteSetup-21.1.0.842-linux.run
./QuartusLiteSetup-21.1.0.842-linux.run
```



Рис. 81

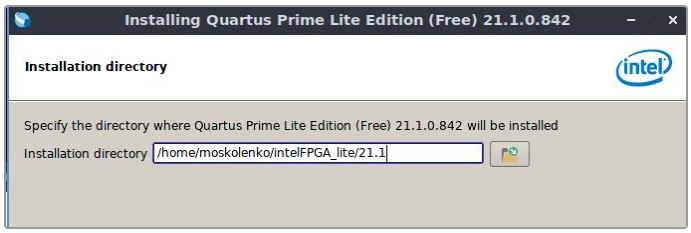


Рис. 82

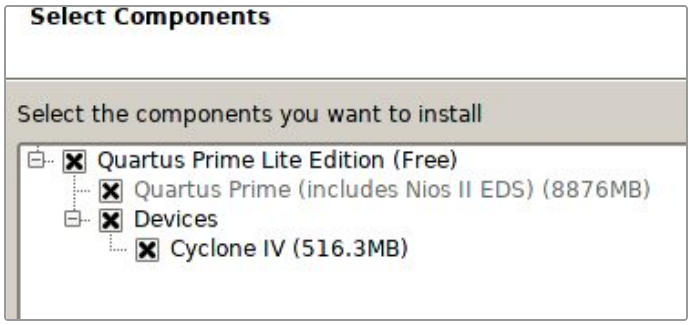


Рис. 83

> ВНИМАНИЕ
>
> Каталог установки не должен содержать пробелов и русских букв — это приводит к сбоям Quartus.

Дальше установщик ведёт себя так же, как под Windows: предлагает принять соглашение, выбрать каталог и показывает список *Devices* с микросхемами из найденных файлов `.qdz`. Если список пуст, значит файл `.qdz` не лежал рядом с установщиком; его можно добавить позже через меню *Tools → Install Devices*.

> БЫЛО ВЕРНО РАНЬШЕ
>
> **Переменные окружения задавать не нужно.** Прежняя редакция требовала дописать в `~/.bashrc` такие строки:
>
> ```
> export QSYS_ROOTDIR=…
> export QUARTUS_ROOTDIR=…
> PATH=$PATH:/home/user/intelFPGA_lite/20.1/quartus/bin:…
> ```
>
> Сейчас это необязательно — скрипты находят Quartus сами (подраздел 4.2.1.4). Заметьте также, что в прежнем примере номера версий не совпадали между строками (21.1 в двух переменных и 20.1 в `PATH`) — такая опечатка приводила к тому, что в `PATH` попадал несуществующий каталог.

4.2.1.2. Правило udev для USB Blaster

Без правила udev программатор доступен только пользователю root, и Quartus его «не видит».

1. Подключите плату и программатор и убедитесь, что система их видит:

```
lsusb
```

В списке должна появиться строка с идентификатором производителя `09fb` — это Altera.

2. Создайте файл правил:

```
sudo nano /etc/udev/rules.d/51-usbblaster.rules
```

и запишите в него:

```
# USB Blaster
SUBSYSTEM=="usb", ENV{DEVTYPE}=="usb_device", ATTRS{idVendor}=="09fb", ATTRS{idProduct
SUBSYSTEM=="usb", ENV{DEVTYPE}=="usb_device", ATTRS{idVendor}=="09fb", ATTRS{idProduct
SUBSYSTEM=="usb", ENV{DEVTYPE}=="usb_device", ATTRS{idVendor}=="09fb", ATTRS{idProduct

# USB Blaster II
SUBSYSTEM=="usb", ENV{DEVTYPE}=="usb_device", ATTRS{idVendor}=="09fb", ATTRS{idProduct
SUBSYSTEM=="usb", ENV{DEVTYPE}=="usb_device", ATTRS{idVendor}=="09fb", ATTRS{idProduct

# USB Blaster III (нужен для Terasic DE23-Lite)
SUBSYSTEM=="usb", ENV{DEVTYPE}=="usb_device", ATTRS{idVendor}=="09fb", ATTRS{idProduct
```

> НОВОЕ В ЭТОЙ РЕДАКЦИИ
>
> **Третья группа строк — новая.** USB Blaster III (`idProduct 6022`) применяется в плате Terasic DE23-Lite. В прежней редакции инструкции правил для него не было, и плата не прошивалась.

3. Примените правила и переподключите кабель программатора:

```
sudo udevadm control --reload
# затем выньте и вставьте кабель USB Blaster
```

> ПРИМЕЧАНИЕ
>
> Если в `/etc/udev/rules.d/` окажутся два файла с правилами для Blaster — например `51-usbblaster.rules` и `90-altera.rules` из комплекта Quartus, — ничего не сломается: udev читает **все** файлы и применяет все подходящие правила. Но если они задают разные права, победит файл, который идёт раньше по алфавиту, потому что оператор `:=` запрещает дальнейшие изменения. Проверить, какое правило сработало, можно командой `udevadm test` .

4.2.1.3. Проверка установки Quartus

Шаги те же, что в подразделе 4.1.1.3: запустить Quartus, открыть *Programmer*, нажать *Hardware Setup* и убедиться, что в списке есть *USB-Blaster*.

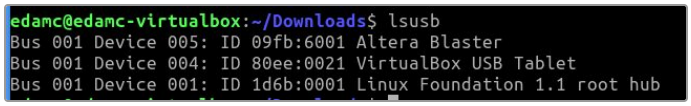


Рис. 84

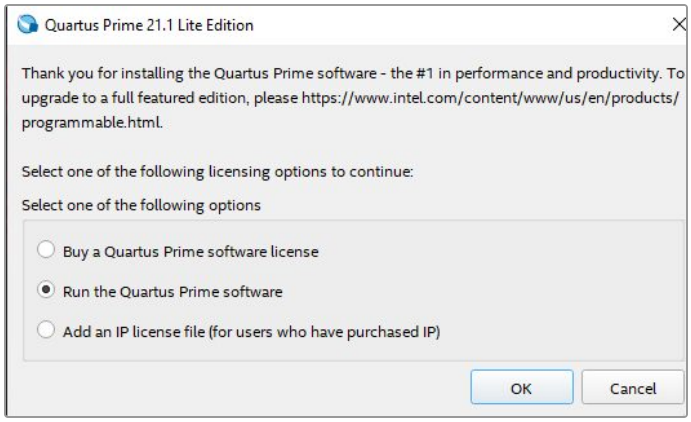


Рис. 85

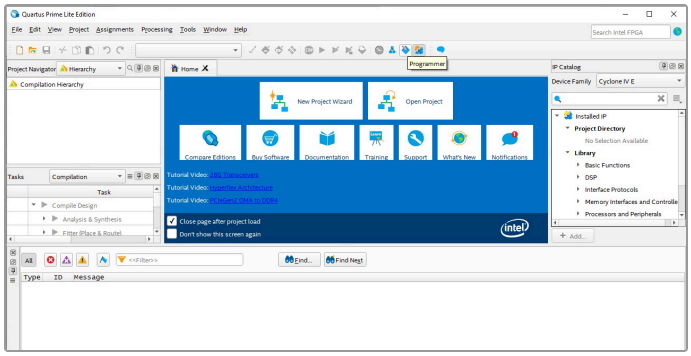


Рис. 86

> **ПРИМЕЧАНИЕ**
>
> В Lubuntu окно *Programmer* иногда не прикрепляется к краю экрана. Помогает меню *Window → Attach Window*.

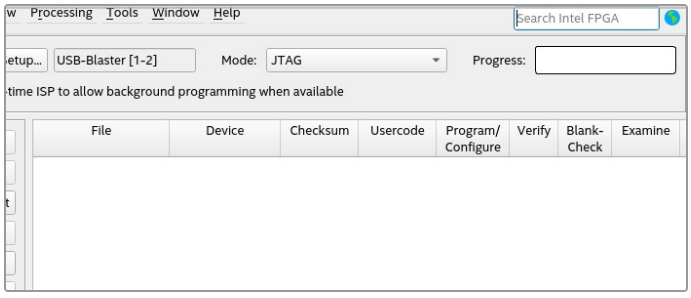


Рис. 87

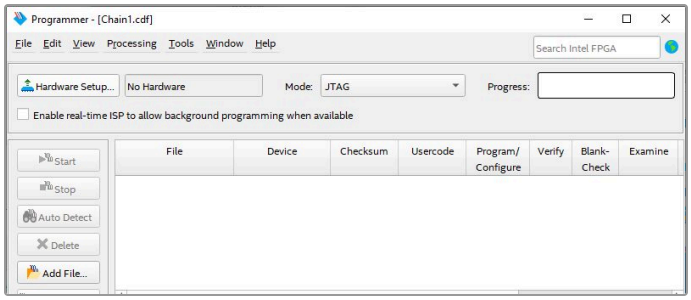


Рис. 88

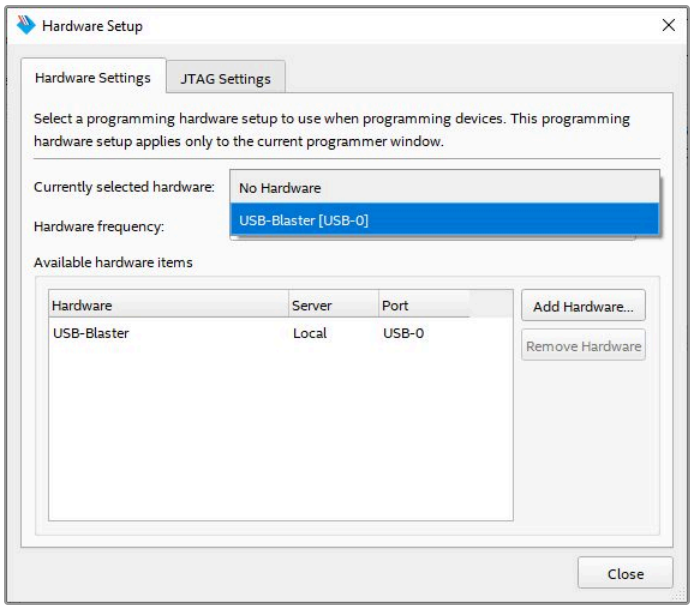


Рис. 89

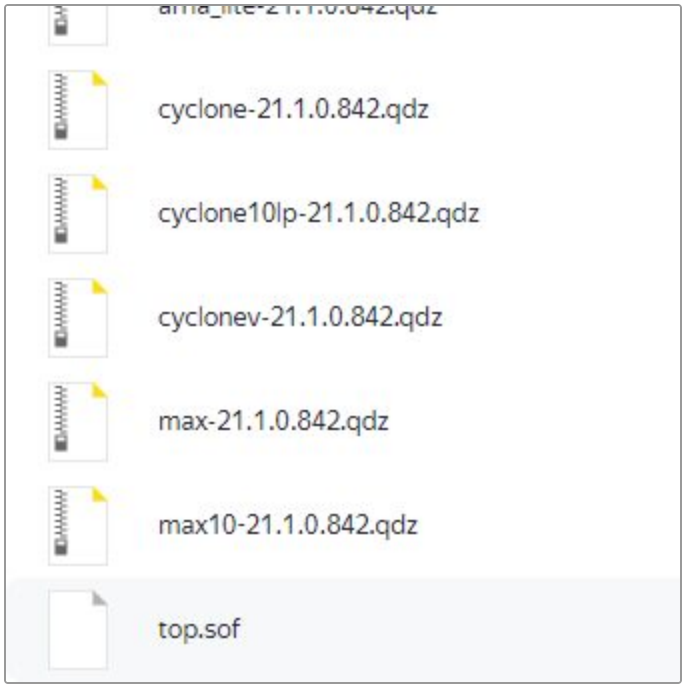


Рис. 90

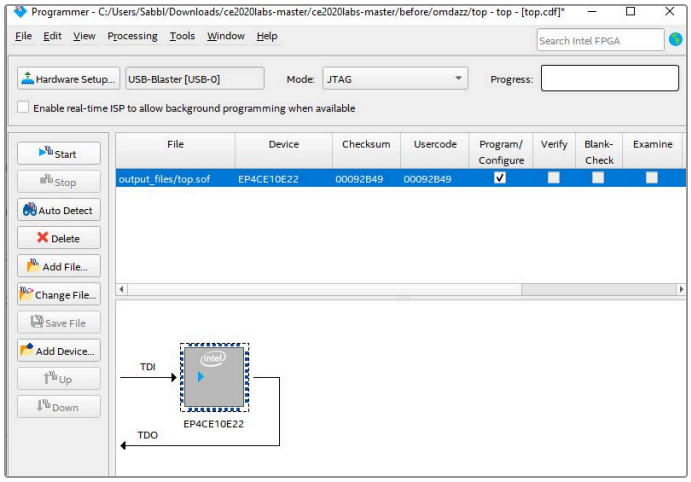


Рис. 91



Рис. 92

Надпись Successful означает, что Quartus и программатор работают

4.2.1.4. Как скрипты BGM находят Quartus

Полностью так же, как под Windows, — см. подраздел 4.1.1.4. Отличается только список стандартных мест: под Linux это домашний каталог, `/opt` и `/tools` (под Windows на их месте корни дисков `/c`, `/d`, `/e`).

4.2.2. Установка AMD Vivado (для плат на ПЛИС Xilinx)

Скачайте установщик с сайта AMD, сделайте его исполняемым и запустите:

```
chmod +x FPGAs_AdaptiveSoCs_Unified_*_Lin64.bin
./FPGAs_AdaptiveSoCs_Unified_*_Lin64.bin
```

Дальше диалоги те же, что под Windows (подраздел 4.1.2): выбрать Vivado, отметить серии Zynq-7000, Artix-7, Kintex-7 и Spartan-7, принять соглашение.

После установки запустите скрипт установки драйверов кабеля — без него программатор не будет доступен обычному пользователю:

```
cd $XILINX_VIVADO/data/xicom/cable_drivers/lnx64/install_script/install_drivers
sudo ./install_drivers
```

> ПРИМЕЧАНИЕ
>
> Под дистрибутивами, не входящими в список официально поддерживаемых, Vivado может не запуститься из-за отсутствия библиотеки `libtinfo.so.6` — сообщение вида `couldn't load file "libxv_tcltasks.so"` и затем `Failed to load feature 'core'`. Причина в том, что скрипт `ldlibpath.sh` внутри Vivado не знает вашего дистрибутива и не подставляет для него нужный путь. Решение — положить совместимую библиотеку в каталог `$XILINX_VIVADO/lib/lnx64.o/Default/`.

Как скрипты находят Vivado — подраздел 4.1.2; порядок одинаков на всех системах.

4.2.3. Установка Gowin EDA (для плат на ПЛИС Gowin)

Скачайте архив для Linux (требуется регистрация на сайте Gowin), распакуйте его в домашний каталог или в `/opt`, так чтобы получился путь вида `~/Gowin/Gowin_V1.9.9/IDE`.

Для доступа к программатору нужно правило udev — программатор плат Tang построен на микросхеме FTDI:

```
sudo nano /etc/udev/rules.d/91-sipeed.rules
```

```
SUBSYSTEM=="usb", ATTRS{idVendor}=="0403", ATTRS{idProduct}=="6010", MODE:="0666"
SUBSYSTEM=="usb", ATTRS{idVendor}=="0403", ATTRS{idProduct}=="6014", MODE:="0666"
```

```
sudo udevadm control --reload
```

Как скрипты находят Gowin EDA — подраздел 4.1.3.

4.2.4. OSS CAD Suite — открытый маршрут для Gowin и Lattice

Так же, как под Windows (подраздел 4.1.4): распаковать архив в `~/oss-cad-suite` и больше ничего не делать — скрипты подключат его сами.

```
cd ~
tar xzf oss-cad-suite-linux-x64-*.tgz
```

4.2.5. Установка Icarus Verilog и просмотрщика диаграмм (обязательно)

Для Ubuntu 22.04 и новее:

```
sudo add-apt-repository ppa:team-electronics/ppa
sudo apt-get update
sudo apt install iverilog gtkwave
```

Проверьте версию:

```
iverilog -V
```

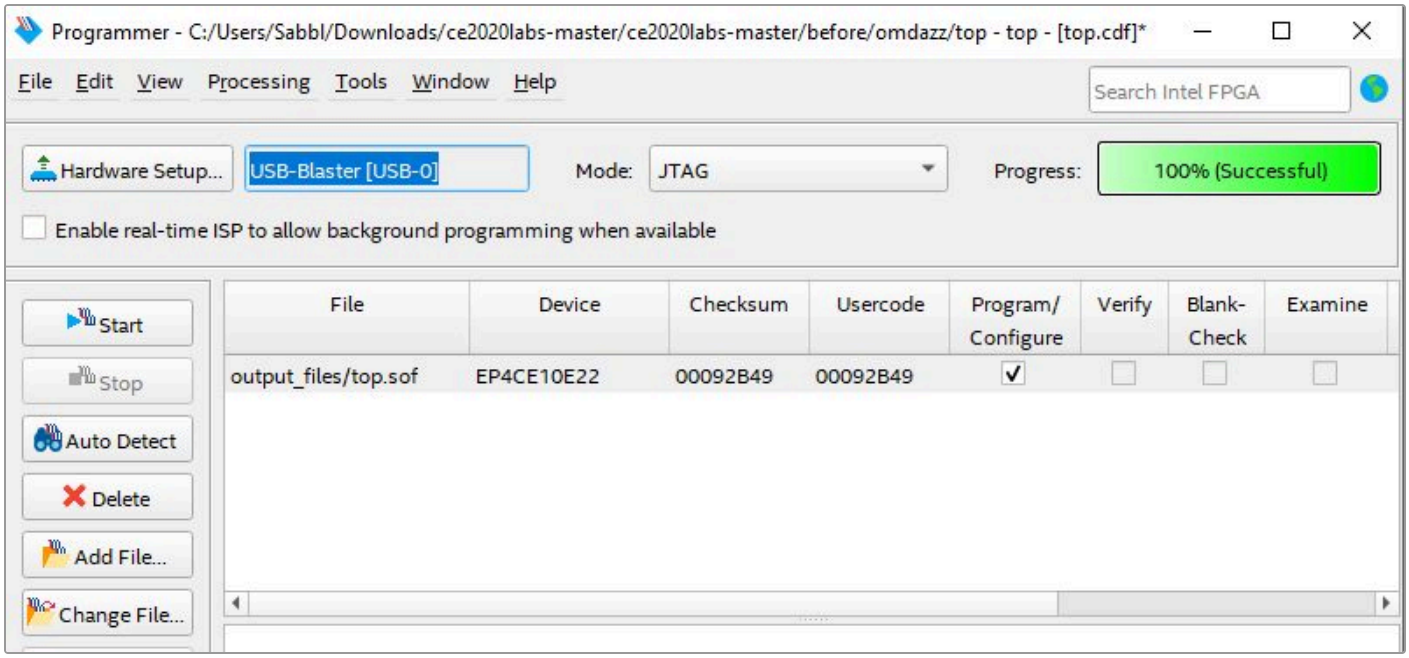


Рис. 93

> НОВОЕ В ЭТОЙ РЕДАКЦИИ
>
> Нужна версия **13**, лучше **14**. Если в репозиториях дистрибутива лежит только версия 11 или 12, есть три выхода: взять Icarus из OSS CAD Suite (подраздел 4.2.4), собрать из исходников, либо

> поставить сборку с bleyer.org/icarus (там есть и исходники). Скрипты проверяют версию и предупреждают, если она слишком старая.

Сборка из исходников в `~/install/iverilog` — путь, который скрипты знают:

```
git clone https://github.com/steveicarus/iverilog.git
cd iverilog && sh autoconf.sh
./configure --prefix=$HOME/install/iverilog
make -j$(nproc) && make install
```

4.2.6. Установка Visual Studio Code (обязательно)

Скачайте пакет `.deb` с code.visualstudio.com и установите:

```
sudo dpkg -i code_*.deb
```



Рис. 94

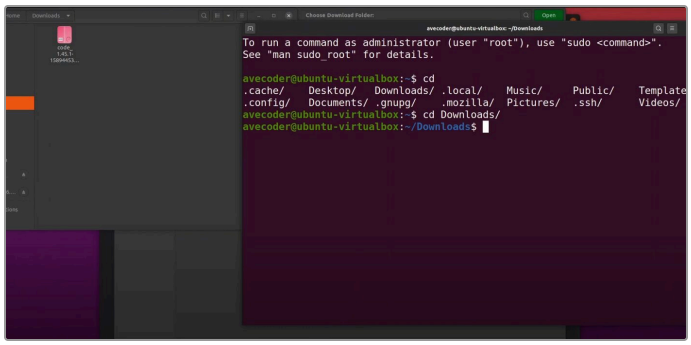


Рис. 95

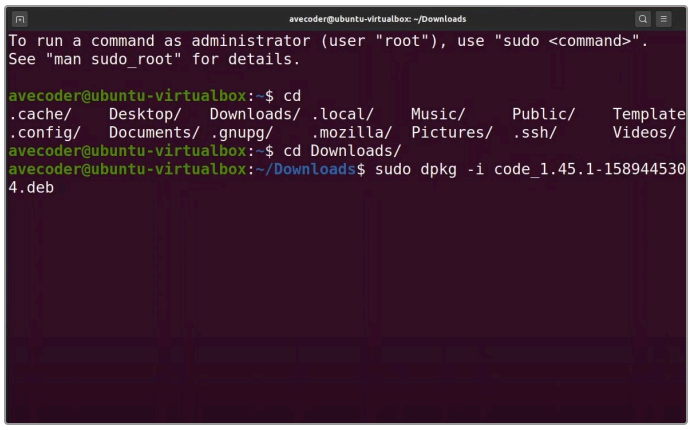


Рис. 96

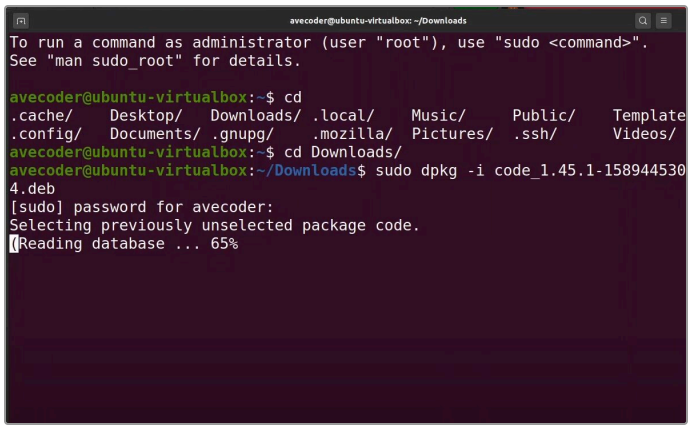


Рис. 97

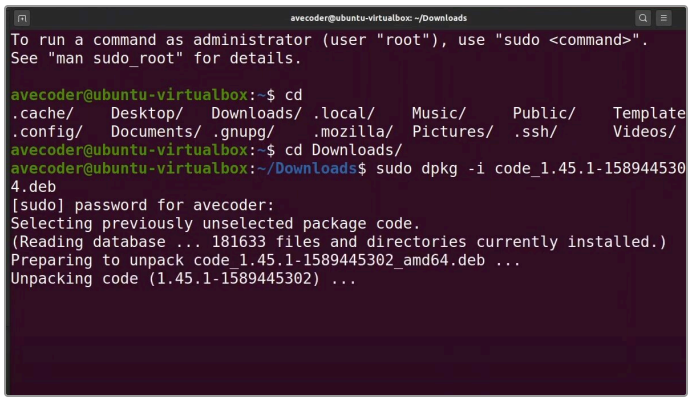


Рис. 98

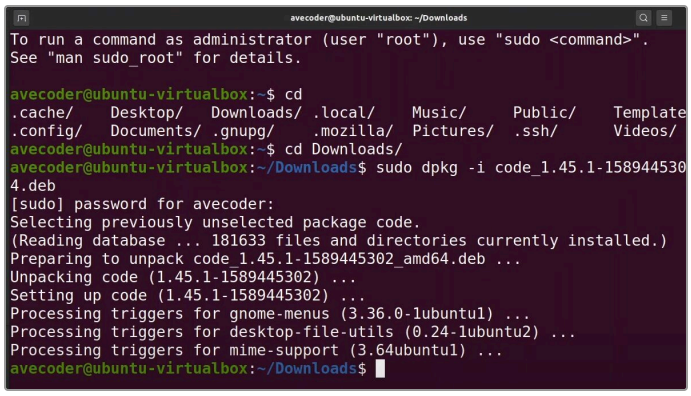


Рис. 99

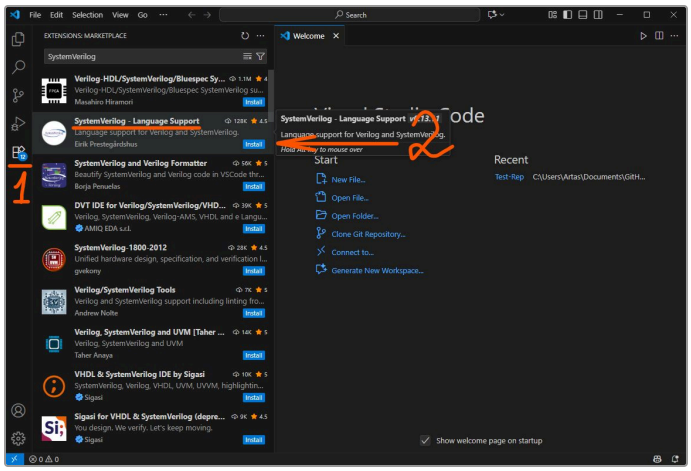


Рис. 100

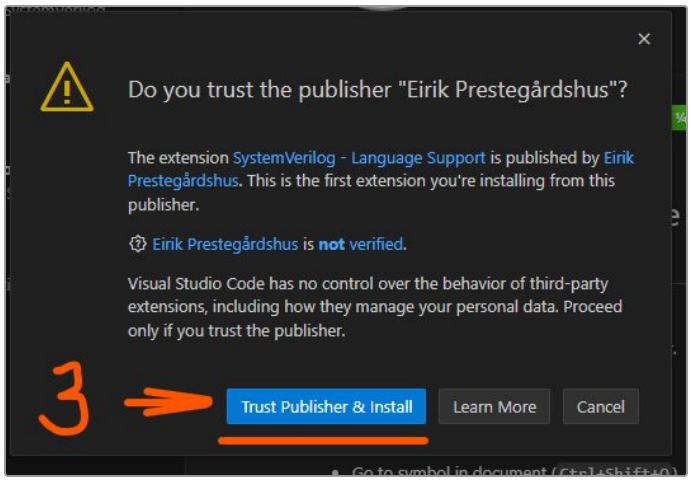


Рис. 101

Затем поставьте расширения SystemVerilog - Language Support и Surfer — так же, как под Windows (подразделы 4.1.6 и 4.1.7)

4.2.7. Установка Surfer — просмотрщика диаграмм

Скачайте сборку для Linux со страницы релизов gitlab.com/surfer-project/surfer, распакуйте и положите исполняемый файл в каталог, который есть в `PATH` — например `~/.local/bin`:

```
mkdir -p ~/.local/bin
tar xzf surfer-*-linux.tar.gz -C ~/.local/bin --strip-components=1 surfer/surfer
surfer --version
```

Чтобы скрипты открывали диаграммы в Surfer, а не в GTKWave:

```
export WAVE_VIEWER=surfer
```

Подробнее о выборе просмотрщика — подраздел 4.1.7.

4.2.8. Установка Git (обязательно)

```
sudo apt install git
```

Настройте имя и адрес — иначе Git откажется делать коммиты:

```
git config --global user.name "Имя Фамилия"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
```

Отдельная оболочка не нужна: под Linux скрипты работают в обычном терминале.

4.2.9. Установка RARS (обязательно)

Нужна Java — под Linux она ставится одной командой, переменные `JAVA_HOME` и `PATH` настраивать не требуется:

```
sudo apt install default-jre
java --version
```

Затем скачайте `rars1_6.jar` со страницы релизов (github.com/TheThirdOne/rars) в домашний каталог и запустите:

```
java -jar rars1_6.jar
```

Скрипт `10_run_instruction_set_simulator.bash` находит файл `.jar` сам — см. подраздел 4.1.9.

4.2.10. Установка XPACK RISC-V GCC (для программ, исполняемых ядром внутри ПЛИС)

Распакуйте архив XPACK в домашний каталог — скрипты найдут его сами (подраздел 4.1.10):

```
cd ~
tar xzf xpack-riscv-none-elf-gcc-*-linux-x64.tar.gz
```

Программа загружается в плату через UART, поэтому вместе с компилятором понадобится терминальная программа — следующий подраздел.

4.2.11. Установка терминальной программы (для упражнений с UART)

Терминальная программа нужна не сама по себе: через неё выполняются упражнения, в которых плата разговаривает с компьютером по **UART** — последовательному каналу поверх того же кабеля USB, которым плата подключена. Без терминала эти упражнения просто нечем проверить: светодиоды и индикатор здесь ничего не показывают.

Упражнения лежат в каталоге `labs/8_unfinished/8_1_uart` и идут от простого к сложному:

УПРАЖНЕНИЕ	ЧТО ДЕЛАЕТ ПЛАТА	ЧТО НАБИРАТЬ И ГДЕ СМОТРЕТЬ
`8_1_1_uart_7segment`	Приёмник `uart_receiver` складывает последние принятые байты в сдвиговый регистр и показывает их на семисегментном индикаторе. Обратной передачи нет: `uart_tx` держится в единице	Набирайте **заглавные A…F** — на индикаторе появятся шестнадцатеричные цифры A…F. Это *не* универсальный декодер ASCII: в коде стоит `rx_buf[i] - 8'h41 + 8'hA`, то есть вычитается код буквы «A», и цифры 0…9 покажут бессмыслицу. Строка для низкого полубайта в исходнике есть, но закомментирована
`8_1_2_uart_echo`	Принятый байт сразу отправляется назад (`uart_transceiver`)	Набирайте что угодно **с выключенным локальным эхом** — ответ платы и есть проверка канала. Если каждый символ виден дважды, эхо осталось включённым, и вы смотрите на свой же ввод
`8_1_3_uart_calc`	Калькулятор с обратной польской записью: `ascii2tok` разбирает символы на числа и операции, `fifo` и `lifo` держат поток и стек, `calc` считает. Операции — `+ - * & | ^`, числа 16-битные	Результат выводится **на семисегментный индикатор** в шестнадцатеричном виде, а не в терминал: передатчик здесь тоже выключен. Самая короткая проверка — `2 3 +` и один `Enter`: на индикаторе должно появиться 5

Выражения для последнего упражнения записываются в постфиксной форме — сначала числа, потом операция; числа разделяются пробелами. Конец выражения — **один** возврат каретки или перевод строки: в `ascii2tok` проверяется `8'h0D` *или* `" "`. Если терминал отправляет CR+LF, выражение может быть обработано как два конца строки. Эти три строки лежат в самом каталоге упражнения (`mnt.txt`) и годятся как проверка посложнее:

```
3 4 + 2 - 10 3 - +          → 12   (0x000C)
3 5 + 7 2 - &               → 0    (0x0000)
100 50 * 200 - 30 20 * +    → 5400 (0x1518)
```

Все три упражнения работают на скорости **115200** бод, 8 бит данных, без контроля чётности, один стоповый бит, без управления потоком — это и есть настройки, которые нужно задать терминалу.

> ВНИМАНИЕ
>
> **Последовательный порт появляется не от любого кабеля.** У плат, где программатор построен на микросхеме FTDI (Tang и подобные), UART идёт по тому же кабелю USB. Но если плата

прошивается отдельным JTAG-программатором — например USB Blaster, — последовательного порта он не даёт: нужен либо UART самой платы, либо отдельный переходник USB–UART, подключённый TX к RX, RX к TX и общей землёй. Уровни сигналов и номера выводов смотрите в документации платы.

ПРИМЕЧАНИЕ

Готовая обёртка `14_run_terminal_program.bash` лежит только в каталоге `8_1_1_uart_7segment`. Для `8_1_2_uart_echo` и `8_1_3_uart_calc` терминал после прошивки запускается вручную — командой из подраздела ниже.

ПРИМЕЧАНИЕ

Каталог называется `8_unfinished` — «незаконченные». Это значит, что упражнения ещё дорабатываются и могут измениться; собираются и запускаются они обычными скриптами, как и остальные.

Тот же терминал понадобится и дальше, для упражнений с процессорными ядрами RISC-V из каталога `labs/5_cpu`. Там плата не просто принимает байты: в ПЛИС синтезируется процессор, программа для него собирается компилятором XPACK и загружается в плату **через UART** скриптом `12_upload_software_to_the_board_using_uart.bash`, а работу программы вы наблюдаете в том же терминале. Так устроены и примеры с многопоточностью — `labs/5_cpu/program/10_rr_threads`.

Про то, как устроена эта инфраструктура и как в неё была встроена процессорная система RISC-V из МИЭТ, подробно написано в статье *«Испытание абстракций: как чужая процессорная система RISC-V переехала на десятки отладочных плат»*: habr.com/ru/articles/1091540 (есть и английская версия).

1. Узнать имя устройства

Подключённая плата появляется в системе как последовательное устройство:

```
ls /dev/ttyUSB* /dev/ttyACM*
```

Платы с программатором на микросхеме FTDI (Tang и другие) обычно дают `/dev/ttyUSB0`; устройства с USB CDC — `/dev/ttyACM0`. Если плата подключена, а список пуст, проверьте правило udev (подразделы 4.2.1.2 и 4.2.3).

2. Получить право читать и писать в порт

Без этого терминал отвечает `Permission denied`: устройство принадлежит группе `dialout`, а обычный пользователь в неё не входит.

```
sudo usermod -aG dialout $USER
```

> ВНИМАНИЕ
>
> Изменение вступает в силу только после нового входа в систему — перезапустить терминал недостаточно. Это самая частая причина `Permission denied` на `/dev/ttyUSB0`.

3. minicom

```
sudo apt install minicom
```

Локальное эхо показывает на экране то, что вы набираете. Нужно оно **не всегда**:

УПРАЖНЕНИЕ	ЛОКАЛЬНОЕ ЭХО	ПОЧЕМУ
`8_1_1_uart_7segment`	включить	плата не отвечает, и без эха не видно, что вы нажали
`8_1_3_uart_calc`	включить	выражение иначе приходится набирать вслепую
`8_1_2_uart_echo`	**выключить**	плата сама возвращает каждый символ; с эхом он появится дважды, и проверка потеряет смысл

Чтобы включить эхо навсегда, создайте файл в домашнем каталоге:

```
touch ~/.minirc.dfl
vim ~/.minirc.dfl        # или nano, или gedit
```

и добавьте в него одну строку:

```
pu localecho        Yes
```

Для упражнения с эхом эту настройку нужно выключить — либо поменять `Yes` на `No`, либо переключить эхо прямо в сеансе: `Ctrl`+`A`, затем `E`. Остальные параметры удобно задать один раз через `minicom -s`: в разделе *Serial port setup* — порт, 115200 8N1, *Hardware Flow Control: No*, *Software Flow Control: No*, затем *Save setup as dfl*.

> БЫЛО ВЕРНО РАНЬШЕ
>
> Документ `terminal_installation.docx`, из которого взята эта инструкция, советует включить локальное эхо один раз и навсегда. Для двух упражнений из трёх это верно, а для `8_1_2_uart_echo` — нет.

Если minicom нужен без скриптов лабораторных работ, он запускается так (`-D` — порт, `-b` — скорость):

```
minicom -D /dev/ttyUSB0 -b 115200
```

Выход — `Ctrl`+`A`, затем `X`, и подтвердить.

Подробнее: wiki.wirenboard.com/wiki/Minicom.

4. picocom

```
sudo apt install picocom
```

Запуск без скриптов (`-b` — скорость, `--echo` включает эхо):

```
picocom -b 115200 --echo /dev/ttyUSB0          # 7segment, calc
picocom -b 115200 /dev/ttyUSB0                 # echo: без --echo
```

Если хочется задать формат порта явно, а не полагаться на значения по умолчанию:

```
picocom -b 115200 -d 8 -p 1 -y n -f n /dev/ttyUSB0
```

Эхо переключается и во время работы: `Ctrl`+`A`, затем `Ctrl`+`C`. Выход — `Ctrl`+`A`, затем `Ctrl`+`X`.

Подробнее: linuxcommandlibrary.com/man/picocom.

5. putty

PuTTY есть и под Linux, и его тоже понимает `14_run_terminal_program.bash` :

```
sudo apt install putty
```

Запуск без скриптов (`-serial` — порт, `-sercfg` — параметры: скорость 115200, 8 бит данных, без чётности, 1 стоповый бит, без управления потоком; `-fn` — шрифт):

```
putty -serial /dev/ttyUSB0 -sercfg 115200,8,n,1,N -fn "Monospace 12"
```

В самом окне PuTTY, в разделе *Terminal*, стоит задать два переключателя: *Local echo — Force on* для 7segment и calc, *Force off* для echo; *Local line editing — Force off*, иначе PuTTY копит строку до `Enter` и плата не получает символы по одному.

> ПРИМЕЧАНИЕ
>
> Ставить все три не нужно: достаточно одного. **picocom** — самый простой и запускается одной строкой; **minicom** привычнее тем, кто работал с ним раньше; **putty** удобен, если вы уже пользуетесь им под Windows и хотите одинаковое окно на обеих системах. Выбрать конкретный терминал в скриптах можно переменной `TERMINAL_PROGRAM`.

6. Проверить

```
cd labs/8_unfinished/8_1_uart/8_1_1_uart_7segment
./03_synthesize_for_fpga.bash
./14_run_terminal_program.bash
```

Наберите заглавную `A` — на индикаторе платы должна появиться шестнадцатеричная цифра A.

> ПРИМЕЧАНИЕ
>
> **Что делает сам `14_run_terminal_program.bash`.** Он перечисляет найденные minicom, picocom и putty в этом порядке, предлагает выбрать порт и помнит выбор в файле `last_uart_port`. Стоит знать три его особенности: под Linux он ищет только `/dev/ttyUSB*`, поэтому устройство `/dev/ttyACM*` придётся открывать вручную; под Git Bash он перебирает `/dev/ttyS*`; picocom он запускает *всегда* с `--echo`, так что для упражнения с эхом терминал лучше открыть вручную.

> ВНИМАНИЕ
>
> Пока терминал держит порт открытым, прошить плату не получится: порт занят. Закройте терминал перед `03_synthesize_for_fpga.bash` или `04_configure_fpga.bash`.

4.2.12. Установка LibreLane (синтез в заказную микросхему)

См. подраздел 4.1.12 — установка через Nix одинакова для Linux и macOS. Под Linux это основной, штатный способ: ни WSL, ни виртуальная машина не нужны.

```
curl --proto '=https' --tlsv1.2 -sSf -L https://install.determinate.systems/nix | sh -
nix develop
./07_synthesize_for_asic.bash
```

4.3. macOS

> ВНИМАНИЕ
>
> **Под macOS нет ни Quartus, ни Vivado.** Altera и AMD не выпускают эти программы для macOS ни в каком виде. Значит, платы на ПЛИС **Altera** и **Xilinx** с macOS напрямую использовать нельзя — нужна виртуальная машина с Linux или Windows (разделы 5 и 7), либо отдельный компьютер.

Зато полностью работают и не требуют никаких ухищрений:

- **Gowin EDA** — платы Tang (с оговоркой ниже);
- **OSS CAD Suite** — платы Gowin и Lattice;
- **Icarus Verilog**, **GTKWave**, **Surfer**, **VS Code**, **Git**, **RARS**, **XPACK RISC-V GCC**, **LibreLane**.

Иными словами, на Mac удобно учиться с платой Tang Nano 9K — это ещё одна причина, по которой Школа рекомендует её как недорогой вариант.

В остальном macOS ведёт себя как Linux: те же скрипты, тот же терминал, те же команды. Но есть четыре особенности, которые заложены в сами скрипты.

1. Gowin EDA работает только на Apple Silicon

На компьютерах Mac с процессором Intel Gowin EDA не запускается, и скрипт сообщает об этом прямо:

```
Gowin IDE is not working on your platform (Mac x64?)
```

На Mac с процессорами M1 и новее всё работает.

2. Другая раскладка каталогов Gowin

Под macOS Gowin EDA поставляется как приложение, и исполняемые файлы лежат внутри него. Скрипт ищет каталоги `GowinIDE.app`, `Gowin` и `gowin`, а внутри них дополнительно проверяет подкаталог:

```
Contents/Resources/Gowin_EDA
```

Поэтому путь к установке выглядит, например, так: `/Applications/GowinIDE.app/Contents/Resources/Gowin_EDA/IDE/bin`. Переменные `DYLD_FRAMEWORK_PATH` и `DYLD_LIBRARY_PATH`, без которых Gowin EDA под macOS не находит свои библиотеки, скрипт выставляет сам — вручную их задавать не нужно.

3. Карантин: «программу нельзя открыть, так как её автор не установлен»

macOS помечает всё скачанное из интернета атрибутом `com.apple.quarantine` и отказывается запускать такие программы. Это самая частая причина, по которой на Mac «не работают» Gowin EDA, OSS CAD Suite и компилятор XPACK.

Скрипты репозитория это **обнаруживают** и подсказывают команду, но снять атрибут должны вы сами. Проверяются:

- каталог Gowin EDA, а также вложенные `IDE` и `Programmer`;
- `~/oss-cad-suite` и `~/Downloads/oss-cad-suite`;
- каталог XPACK RISC-V GCC.

Команда для любого из них:

```
xattr -rd com.apple.quarantine ~/oss-cad-suite

# если выдаёт Permission denied:
sudo xattr -rd com.apple.quarantine ~/oss-cad-suite
```

> ПРИМЕЧАНИЕ
>
> Обнаружив атрибут, скрипт задаёт вопрос `Abort, Retry, Ignore? [a/R/i]`. Это значит: откройте второе окно терминала, выполните подсказанную команду `xattr`, вернитесь и нажмите `Enter` — вариант *Retry* выбран по умолчанию, и скрипт проверит атрибут заново. Ответ `i` продолжит работу без снятия атрибута (скорее всего, с ошибкой), `a` прервёт выполнение.

4. Просмотрщик диаграмм: Surfer на Apple Silicon

На Mac с процессором M1 и новее скрипты открывают диаграммы в **Surfer**, а не в GTKWave, и требуют, чтобы он был установлен:

```
brew install surfer   # либо скачайте сборку со страницы релизов
```

На Mac с процессором Intel используется GTKWave, причём запускается исполняемый файл внутри приложения:

```
/Applications/gtkwave.app/Contents/MacOS/gtkwave-bin
```

Поэтому там достаточно перетащить `gtkwave.app` в `/Applications`. Запуск через `open -a gtkwave` не годится: в этом случае GTKWave не читает файл сценария `gtkwave.tcl`, который заранее расставляет сигналы на диаграмме, — в комментарии к скрипту это отмечено отдельно.

> ПРИМЕЧАНИЕ
>
> Ещё одна мелочь, заложенная в скрипты: при поиске компилятора XPACK под macOS каталог `Desktop` исключается из обхода. Это сделано, чтобы macOS не спрашивала разрешение на

> доступ к файлам рабочего стола при каждом запуске.

5. Терминал для упражнений с UART

Упражнения из раздела 4.1.11 работают и на Mac. Плата появляется как `/dev/tty.usbserial-*` или `/dev/tty.usbmodem*`:

```
ls /dev/tty.usb*
```

Группы `dialout` в macOS нет, права настраивать не нужно. Терминал ставится через Homebrew:

```
brew install picocom          # или minicom
picocom -b 115200 --echo /dev/cu.usbserial-0001
```

> ПРИМЕЧАНИЕ
>
> У одного и того же адаптера в macOS два имени: `/dev/tty.*` и `/dev/cu.*`. Для своих подключений берите `cu` («call-up»): `tty` ждёт сигнала DCD и может просто повиснуть при открытии. Скрипт `14_run_terminal_program.bash` при этом ищет именно `/dev/tty.usbserial-*` и пользуется GNU-вариантами `stat -c` и `stty -F`, поэтому на macOS надёжнее запускать терминал вручную.

Без установки чего-либо годится и встроенный `screen`, но выходить из него неудобно (`Ctrl`+`A`, `K`, затем `y`):

```
screen /dev/tty.usbserial-0001 115200
```

Установка остального через Homebrew:

```
brew install git icarus-verilog
brew install surfer                     # на Apple Silicon
brew install gtkwave                    # на Mac с процессором Intel
brew install picocom                    # терминал для упражнений с UART
brew install --cask visual-studio-code
```

> НЕ ПРОВЕРЕНО
>
> Раздел про macOS написан на основе кода скриптов репозитория (обработка `darwin` в `00_setup_gowin`, `00_setup_yosys`, `00_setup_icarus`, `00_setup_riscv` и

> `00_setup.source_bash`), а не проверен на живой машине с macOS. Если вы столкнётесь с расхождением, сообщите об этом — раздел будет исправлен.

5. Виртуальная машина для занятий по функциональной верификации

Для раздела курса про функциональную верификацию нужен симулятор **Questa**. Он распространяется отдельно и может требовать лицензию, поэтому Школа выдаёт готовую виртуальную машину с установленным симулятором.

1. Установите VirtualBox.
2. Скачайте и распакуйте архив с виртуальной машиной (ссылку даёт Школа).
3. Запустите VirtualBox и добавьте машину кнопкой *Добавить*, указав файл с расширением `.vbox` из распакованного каталога.
4. Запустите машину. Пароль пользователя: `1234`.
5. Откройте терминал (*Applications* → *System Tools* → *Terminal*) и введите `vsim`. Должно появиться приветственное окно симулятора.

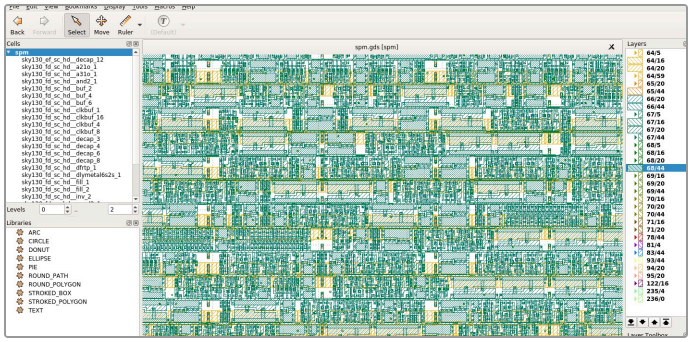


Рис. 102

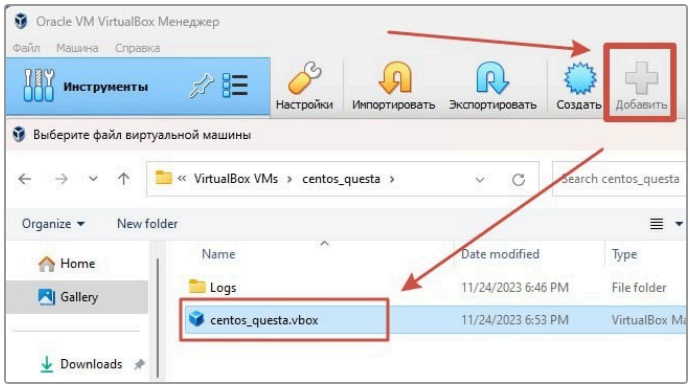


Рис. 103

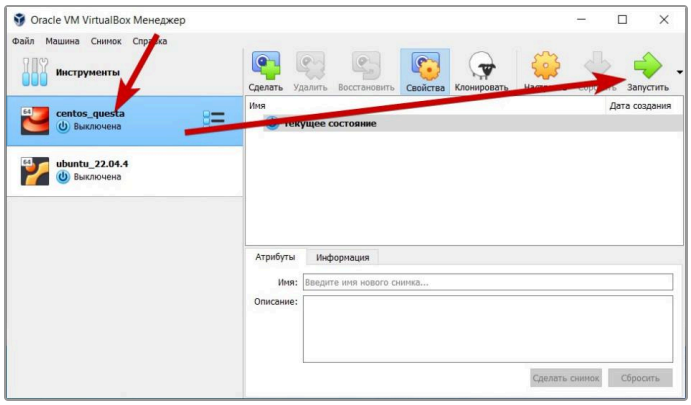


Рис. 104

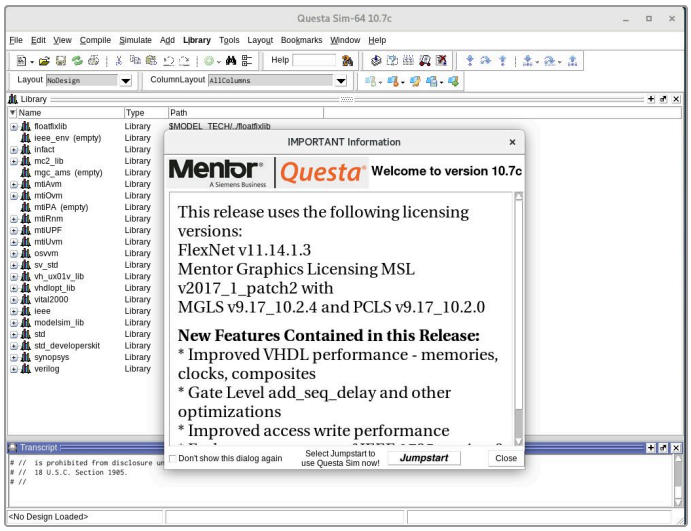


Рис. 105

При затруднениях напишите в чат Школы синтеза цифровых схем.

6. Загружаемый SSD с предустановленным ПО

Вместо установки всего перечисленного вручную можно получить готовый образ SSD с предустановленным ПО и загружаться с него. Это удобно для занятий в учебных классах.

Основа раздела — статья Юрия Панчула: habr.com/ru/articles/754262.

> ПРИМЕЧАНИЕ
>
> Образ содержит ПО для плат на ПЛИС Altera (Quartus). Vivado и Gowin EDA при необходимости ставятся отдельно. Questa в образе может требовать лицензию.

6.1. Запись образа на SSD

Скачайте файл образа `.img` (ссылку даёт Школа).

> ВНИМАНИЕ
>
> **Про GPT.** Таблиц разделов GPT на диске две — основная и резервная. Образ (около 50 ГБ в распакованном виде) меньше любого современного SSD, поэтому после записи резервная таблица остаётся нетронутой и мешает первой загрузке. Поэтому вторую таблицу нужно стереть.

Это делает скрипт из самого репозитория:

```
scripts/admin/erase_ssd_gpt_and_write_bootable_image.bash
```

Положите скрипт в каталог с образом (или образ — в каталог со скриптом) и запустите под `sudo`. Скрипт проверит, что образ один, спросит, какой диск перезаписать, убедится, что диск не смонтирован, и запишет образ. Операция занимает десятки минут.

6.2. Загрузка с SSD

Подключите SSD к выключенному компьютеру, включите его и нажимайте клавишу входа в меню загрузки:

ПРОИЗВОДИТЕЛЬ	КЛАВИША
ASUS, Acer	Del или F2
Dell	F12 или F2
HP	F10
Lenovo	F2 или Fn+F2 (ноутбуки), F1 (десктопы), Enter затем F1 (ThinkPad)
MSI	Del
Samsung	F2

Образ поддерживает и UEFI, и устаревшую Legacy-загрузку.

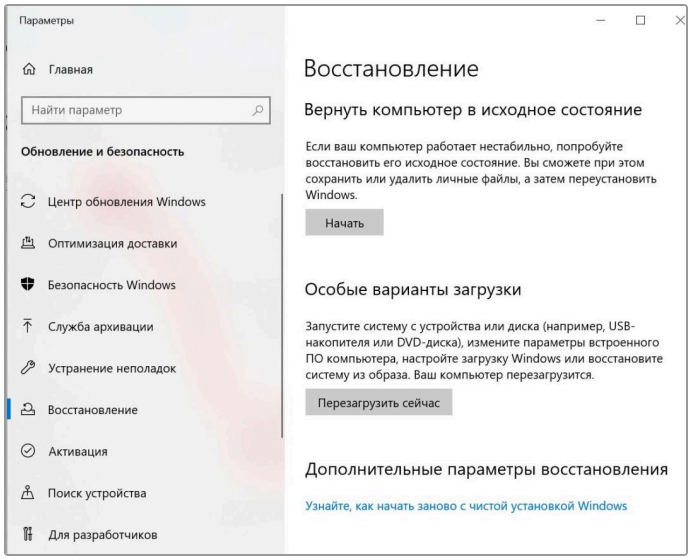


Рис. 106

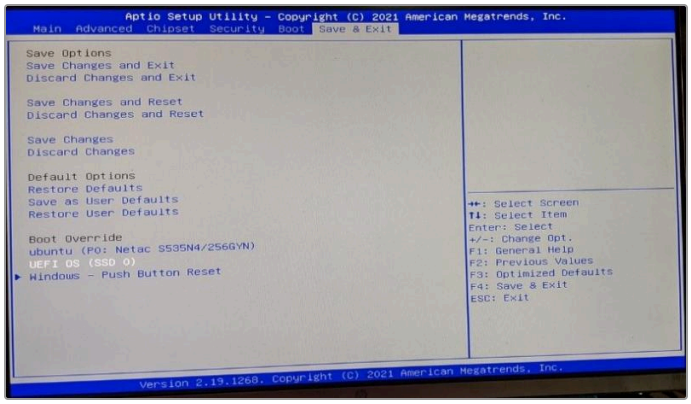


Рис. 107

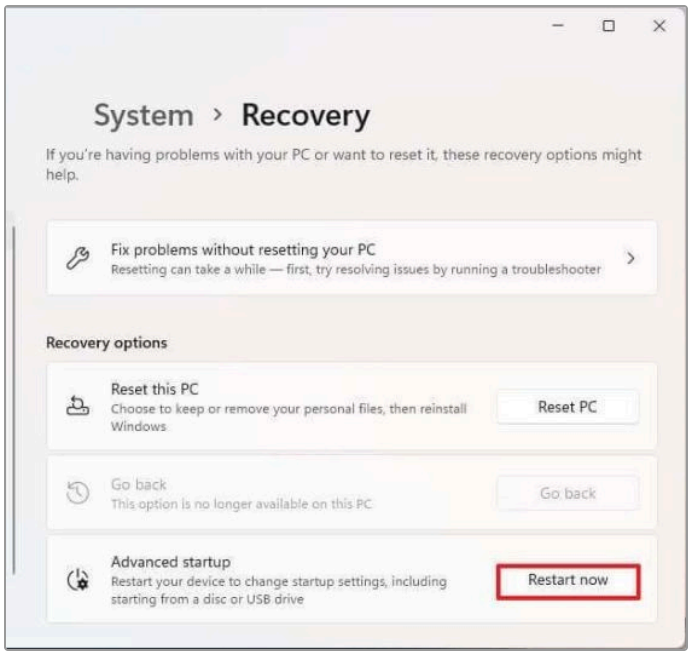


Рис. 108

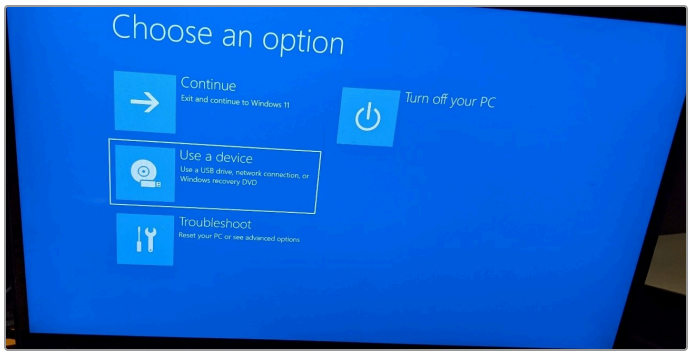


Рис. 109

Если клавиши при включении не срабатывают (так бывает из-за быстрой загрузки Windows), воспользуйтесь средствами самой Windows: *Параметры → Восстановление → Особые варианты загрузки → Перезагрузить сейчас*, затем *Use a device* и выберите пункт *Linpus lite* (вариант *EFI USB Device* работает не всегда).



Рис. 110



Рис. 111

```
Info: Quartus Prime Shell was successful. 0 errors, 65 warnings
    Info: Peak virtual memory: 496 megabytes
    Info: Processing ended: Sun Aug 13 19:07:19 2023
    Info: Elapsed time: 00:00:20
    Info: Total CPU time (on all processors): 00:00:17
03_synthesize_for_fpga.bash: using cable USB-Blaster [1-7.3]
Error (213019): Can't scan JTAG chain. Error code 87.
```

Рис. 112

```
[verilog@host-15 basics-graphics-music]$ cd ~/projects/basics-graphics-music
[verilog@host-15 basics-graphics-music]$ ./check_setup_and_choose_fpga_board.bash
check_setup_and_choose_fpga_board.bash: Please select an FPGA board amoung the following supported:
1) 000_todo          4) de10_lite        7) piswords6  10) tangprimer20k
2) c5gx              5) de2_115          8) rzrd       11) zeowaa
3) de0_cv            6) omdazz           9) saylinx    12) exit
Your choice (a number):
```

Рис. 113

Далее загрузится Simply Linux. Логин `verilog`, пароль `verilog`.

```
Would you like to create the new run directories for the synthesis of all labs in the package, based
on your FPGA board selection? We recommend to do this if you plan to work with Quartus GUI rather tha
n with the synthesis scripts. [y/n] y

check_setup_and_choose_fpga_board.bash: Setting up: "/home/verilog/projects/basics-graphics-music/lab
s/12_snail_fsm/run"
check_setup_and_choose_fpga_board.bash: Setting up: "/home/verilog/projects/basics-graphics-music/lab
s/03_decoder/run"
```

Рис. 114

```
[verilog@host-15 01_and_or_not_xor_de_morgan]$ ./04_configure_fpga.bash
04_configure_fpga.bash: using cable USB-Blaster [1-7.3]
Info: *******************************************************************
Info: Running Quartus Prime Programmer
Info: Command: quartus_pgm --no_banner -c "USB-Blaster [1-7.3]" --mode=jtag -o P;fpga_project.sof
Info (213045): Using programming cable "USB-Blaster [1-7.3]"
Info (213011): Using programming file fpga_project.sof with checksum 0x00094EB6 for device EP4CE6E22C
1
Info (209060): Started Programmer operation at Sun Aug 13 19:08:10 2023
Info (209016): Configuring device index 1
Info (209017): Device 1 contains JTAG ID code 0x020F10DD
Info (209007): Configuration succeeded -- 1 device(s) configured
Info (209011): Successfully performed operation(s)
Info (209061): Ended Programmer operation at Sun Aug 13 19:08:13 2023
Info: Quartus Prime Programmer was successful. 0 errors, 0 warnings
    Info: Peak virtual memory: 325 megabytes
    Info: Processing ended: Sun Aug 13 19:08:13 2023
    Info: Elapsed time: 00:00:04
    Info: Total CPU time (on all processors): 00:00:01
```

Рис. 115

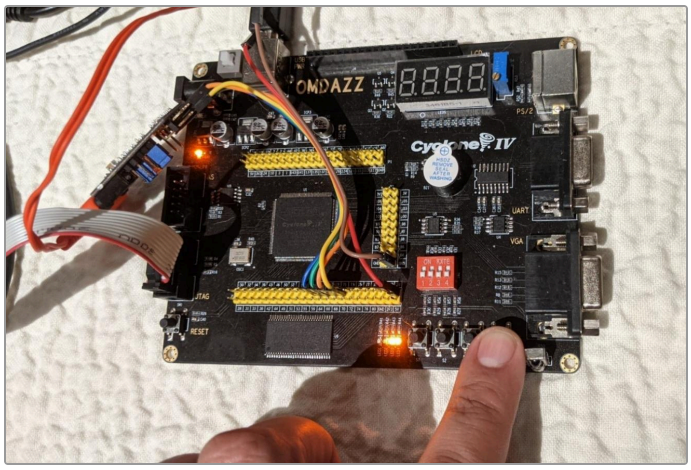


Рис. 116

6.3. Обновление репозиториев

Подключитесь к интернету и обновите примеры. Если каталог уже есть:

```
cd ~/projects/basics-graphics-music
git pull
```

Если каталога нет:

```
cd ~/projects
git clone https://github.com/chipdesignschool/basics-graphics-music.git
git clone https://github.com/chipdesignschool/systemverilog-homework.git
```

> БЫЛО ВЕРНО РАНЬШЕ
>
> Прежняя редакция предлагала клонировать с `gitflic.ru` и упоминала репозиторий `valid-ready-etc` . Он устарел; его материал вошёл в BGM. Актуальные адреса — в разделах 1.2 и 1.3.

6.4. Проверка работы с платой

Сначала выберите плату:

```
cd ~/projects/basics-graphics-music
./check_setup_and_choose_fpga_board.bash
```

Скрипт покажет список поддерживаемых плат, запомнит ваш выбор в файле `fpga_board_selection` в корне репозитория и предложит создать рабочие каталоги для всех примеров.

Затем соберите и прошейте пример:

```
cd labs/1_basics/1_01_and_or_not_xor_de_morgan
./03_synthesize_for_fpga.bash
```

> ПРИМЕЧАНИЕ
>
> Имена каталогов примеров изменились: теперь они сгруппированы по разделам курса — `labs/1_basics/1_01_and_or_not_xor_de_morgan` вместо прежнего `labs/01_and_or_not_xor_de_morgan`.

Если прошивка не удалась с первого раза, переподключите кабель USB Blaster и запустите только прошивку:

```
./04_configure_fpga.bash
```

После этого можно нажимать кнопки на плате и наблюдать за светодиодами в соответствии с кодом примера.

7. Виртуальная машина с предустановленным ПО

Если загружаться с отдельного SSD неудобно, тот же образ можно запустить в виртуальной машине.

1. Скачайте образ `.img`.
2. Преобразуйте его в формат VirtualBox при помощи `qemu-img`:

   ```
   qemu-img convert -f raw -O vdi образ.img образ.vdi
   ```

3. Создайте в VirtualBox новую машину и подключите к ней полученный файл `.vdi` как существующий жёсткий диск.
4. Логин `verilog`, пароль `verilog`.

VirtualBox binaries

By downloading, you agree to the terms and conditions of the respective license.

If you're looking for the latest VirtualBox 6.1 packages, see VirtualBox 6.1 builds. Version 6.1 will remain supported until December 2023.

VirtualBox 7.0.10 platform packages

- Windows hosts
- macOS / Intel hosts
- Linux distributions
- Solaris hosts
- Solaris 11 IPS hosts

The binaries are released under the terms of the GPL version 3.

See the changelog for what has changed.

You might want to compare the checksums to verify the integrity of downloaded packages. *The SHA256 checksums should be favored as the MD5 algorithm must be treated as insecure!*

- SHA256 checksums, MD5 checksums

Note: After upgrading VirtualBox it is recommended to upgrade the guest additions as well.

VirtualBox 7.0.10 Oracle VM VirtualBox Extension Pack

- All supported platforms

Support VirtualBox RDP, disk encryption, NVMe and PXE boot for Intel cards. See this chapter from the User Manual for an introduction to this Extension Pack. The Extension Pack binaries are released under the VirtualBox Personal Use and Evaluation License (PUEL). *Please install the same version extension pack as your installed version of VirtualBox.*

VirtualBox 7.0.10 Software Developer Kit (SDK)

- All platforms

Рис. 117

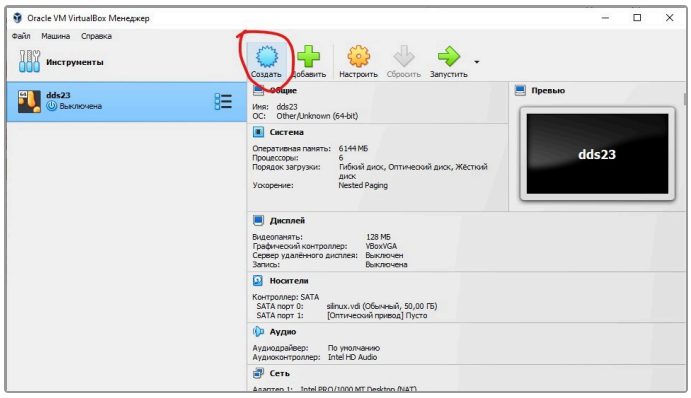


Рис. 118

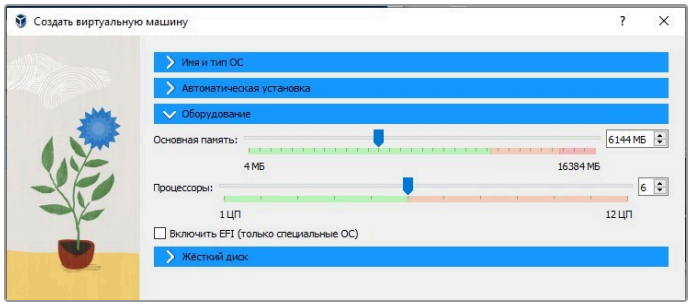


Рис. 119

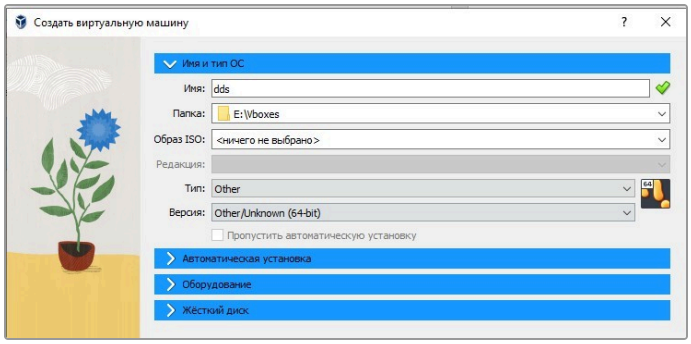


Рис. 120

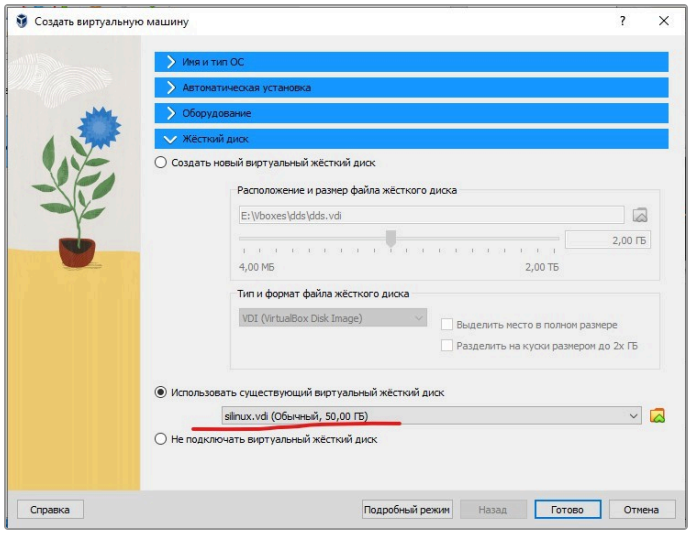


Рис. 121

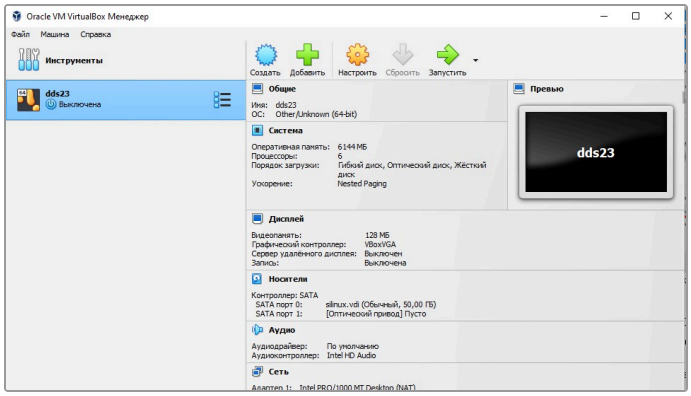


Рис. 122

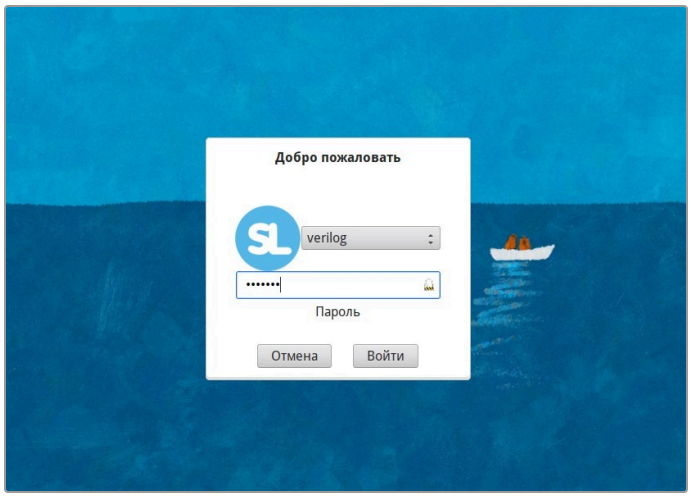


Рис. 123



Рис. 124



Рис. 125



Рис. 126

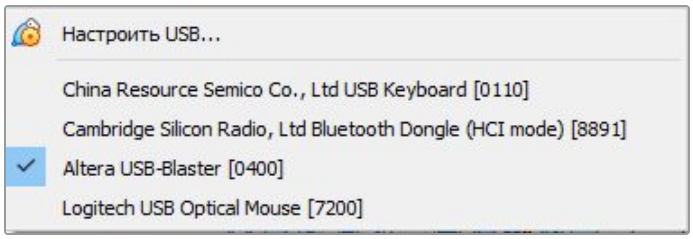


Рис. 127

> ВНИМАНИЕ
>
> Прошивка платы из виртуальной машины требует передать USB-устройство внутрь машины (*Устройства → USB*), и для VirtualBox нужен Extension Pack. Если программатор упорно не определяется, проще работать с платой из основной системы, а виртуальную машину использовать только для моделирования и верификации.

8. Подробнее о репозитории BGM

8.1. Из чего состоит курс

Примеры в репозитории сгруппированы по разделам курса. Ниже — краткая карта, по статье habr.com/ru/articles/1071736.

РАЗДЕЛ	О ЧЁМ	ЧТО ПОЛУЧАЕТСЯ НА ПЛАТЕ
`1_basics`	Логические элементы, мультиплексоры, D-триггеры, счётчики, сдвиговые регистры, конечные автоматы	Светодиоды, семисегментный индикатор, кнопки
`2_graphics`	Генерация изображения комбинационной логикой, затем с памятью	Картинка на VGA, HDMI или ЖК-экране; игры
`3_music`	Распознавание нот через микрофон, синтез звука через ЦАП	Звук из динамика, определение сыгранной ноты
`4_microarchitecture`	Конвейеры, очереди FIFO, управление потоком по кредитам	То, о чём спрашивают на собеседованиях
`8_unfinished`	Упражнения в работе, в том числе `8_1_uart` : приём байта, эхо и калькулятор с обратной польской записью поверх UART	Обмен символами с платой в терминальной программе (см. разделы 4.1.11 и 4.2.11)
`5_cpu`	Процессор schoolRISCV (около 300 строк на Verilog, около десятка команд RISC-V), а также ядра `picorv32` , `yrv` , `aps`	Собственный процессор в ПЛИС, исполняющий ваши программы

Разделы 3–5 курса Школы соответствуют этой структуре: ассемблер RISC-V в симуляторе RARS, затем микроархитектура процессорного ядра, затем интеграция ядра в систему — ввод-вывод от датчиков и параллелизм через прерывания. Пример *Round-Robin Threads* (RR Threads, RRT; прежнее название — Femto Threads) реализует переключение контекста задач на ассемблере, то есть многопоточность на синтезированном вами процессоре. Он лежит в `labs/5_cpu/program/10_rr_threads` .

8.2. Как выполнять упражнения

Каждый пример — отдельный каталог с одинаковым набором скриптов. Порядок работы обычно такой:

1. Выбрать плату один раз на весь репозиторий:

   ```
   ./check_setup_and_choose_fpga_board.bash
   ```

2. Зайти в каталог примера и посмотреть файл `lab_top.sv` — это то, что предлагается изменить.
3. Промоделировать и посмотреть диаграммы:

   ```
   ./02_simulate_rtl.bash
   ```

4. Собрать и прошить плату:

```
./03_synthesize_for_fpga.bash
```

Полный перечень скриптов:

СКРИПТ	ЧТО ДЕЛАЕТ
`01_clean`	удалить результаты предыдущих запусков
`02_simulate_rtl`	моделирование в Icarus Verilog, диаграммы в GTKWave или Surfer
`03_synthesize_for_fpga`	синтез, размещение, трассировка и прошивка — одной командой
`04_configure_fpga`	только прошивка уже собранного битстрима
`05_run_gui_for_fpga_synthesis`	открыть проект в графическом интерфейсе САПР
`06_choose_another_fpga_board`	выбрать другую плату
`07_synthesize_for_asic`	синтез в заказную микросхему через LibreLane
`08_, 09_visualize_asic_…`	просмотр топологии (OpenROAD, KLayout)
`10_run_instruction_set_simulator`	запуск RARS
`11_build_software_to_run_on_cpu`	сборка C и ассемблера компилятором XPACK
`12_upload_software_to_the_board_using_uart`	загрузка программы в плату через UART
`13_choose_another_riscv_core_for_software`	выбрать ядро: aps, yrv, picorv32
`14_run_terminal_program`	терминал: minicom, picocom или putty

Руководство для начинающих с разбором первого примера: verilog-meetup.com — Beginner's guide, а также файл docs/beginner-s-guide-to-basics-graphics-music.md в репозитории.

9. Подробнее о репозитории systemverilog-homework

SVH — это набор небольших задач с автоматической проверкой. Структура простая: в файле задачи есть незаполненный модуль, рядом — тестовое окружение, которое сообщает, прошло решение или нет.

Что нужно: только **Icarus Verilog** и **Git**. Плата и САПР не требуются, поэтому SVH удобно начинать сразу, не дожидаясь получения платы.

```
git clone https://github.com/chipdesignschool/systemverilog-homework.git
cd systemverilog-homework
# внутри каталога задачи:
./run.bash
```

Темы задач идут от простых комбинационных схем к арифметике, конечным автоматам, конвейерам и интерфейсам с квитированием — то есть к тем же микроархитектурным приёмам, что и раздел `4_microarchitecture` в BGM, но без привязки к плате.

Три копии репозитория — как и у BGM; для занятий Школы берите версию `chipdesignschool`:

Школа синтеза	github.com/chipdesignschool/systemverilog-homework
Разработка	github.com/yuri-panchul/systemverilog-homework
Международные семинары	github.com/verilog-meetup/systemverilog-homework

10. Синтез примеров в заказную микросхему через Tiny Tapeout

Tiny Tapeout — сервис, который собирает проекты многих участников на один кристалл и отправляет его на фабрику. Такой способ изготовления называется MPW (multi-project wafer) или «шаттл»: стоимость маски делится между всеми участниками, и поэтому изготовление одной небольшой схемы становится доступным для университета, а иногда и для отдельного человека.

Для BGM есть готовый шаблон, который берёт из примера модуль `lab_top` — то есть ровно тот файл, который учащийся менял, отлаживая пример на плате. Изготовление идёт на фабрике института IHP (Leibniz-Institut für innovative Mikroelektronik) в Германии.

Порядок действий

1. Отладить пример на ПЛИС обычным путём (разделы 8.2 и 4).
2. Локально проверить, что пример проходит синтез в микросхему:

   ```
   ./07_synthesize_for_asic.bash
   ./08_visualize_asic_synthesis_results_1.bash
   ```

   Это тот же маршрут RTL-to-GDSII, который применяется и при настоящем изготовлении, поэтому ошибки видны заранее.
3. Создать репозиторий из шаблона Tiny Tapeout и перенести в него `lab_top`.

4. Отправить проект на очередной шаттл через сайт Tiny Tapeout.

Подробнее:

- habr.com/ru/articles/1084844 — статья о превращении примеров BGM в микросхему;
- verilog-meetup.com — From FPGA to ASIC using template;
- tinytapeout.com — сроки очередных шаттлов и стоимость участия.

> НЕ ПРОВЕРЕНО
>
> Стоимость участия и даты шаттлов меняются, и в источниках, доступных при подготовке этой редакции, конкретных цифр не было. Смотрите сайт Tiny Tapeout.

11. Дополнительная информация

Документация внутри репозитория

В каталоге docs/ лежат отдельные файлы, которые полезно читать вместе с этой инструкцией:

ФАЙЛ	О ЧЁМ
`IntelQuartus.md`	замечания по Quartus
`GowinEDA.md`	замечания по Gowin EDA
`Yosys.md`	открытый маршрут
`vivado_installation_guide/`	установка Vivado с иллюстрациями
`wsl.md`	работа под Windows Subsystem for Linux
`qemu.md`	запуск в QEMU
`beginner-s-guide-to-basics-graphics-music.md`	руководство для начинающих
`boards/README.md`	таблица плат (см. раздел 3)

Статьи и материалы

- Обзор курса Школы синтеза (Хабр)
- Загружаемый SSD с ПО (Хабр)
- Из ПЛИС в микросхему через Tiny Tapeout (Хабр)

- Испытание абстракций: как чужая процессорная система RISC-V переехала на десятки отладочных плат (Хабр) — про инфраструктуру упражнений с процессорными ядрами и про то, как устроены уровни `board_specific_top` и `lab_top`; есть английская версия
- verilog-meetup.com — заметки, руководства по платам Tang на пяти языках, сравнения САПР
- edu.yadro.com/chip-design-school — сайт Школы

Чего в этой инструкции нет

В скриптах репозитория есть поддержка ещё одного САПР — **Efinity** для ПЛИС Efinix (`scripts/steps/00_setup_efinity.source_bash`, только под Linux). Установка здесь не описана сознательно: единственная плата на Efinix (Trion T20) лежит в каталоге `boards/zzz_postponed_and_retired/`, то есть в число действующих плат не входит и в таблицу раздела 3 не попадает. Если такая плата у вас появится, смотрите скрипт: он ищет Efinity по переменной `EFINITY_HOME` и по команде `efinity_sh.sh` в `PATH`.

Где спросить

Чат Школы синтеза цифровых схем — для вопросов по занятиям и по установке. Ошибки и несоответствия в самих примерах лучше оформлять как issue в репозитории: это самый надёжный способ, чтобы о проблеме узнали разработчики.

12. Частые вопросы

Нужно ли ставить все САПР?

Нет. Только тот, который соответствует вашей плате: Quartus для Altera, Vivado для Xilinx, Gowin EDA или OSS CAD Suite для Gowin, OSS CAD Suite для Lattice. Если платы нет, не нужен ни один: примеры моделируются в Icarus Verilog, а задачи SVH выполняются целиком без платы.

У меня нет платы. Можно учиться?

Да. Моделирование в Icarus Verilog с просмотром диаграмм покрывает значительную часть курса, а весь репозиторий SVH вообще не требует платы. Для графики есть возможность посмотреть результат в виде изображения, не имея монитора на плате.

Нужно ли прописывать PATH и переменные окружения?

Нет, если инструменты установлены в стандартные места. Скрипты находят их сами. Переменные (`QUARTUS_ROOTDIR`, `XILINX_VIVADO`, `GOWIN_VERSION_DIR` и остальные) нужны в двух случаях: установка в нестандартном каталоге, либо несколько версий и необходимость выбрать конкретную.

Можно ли держать несколько версий Quartus одновременно?

Да, и для старых плат это обязательно: Cyclone II требует Quartus II 13.0sp1, Cyclone III — 13.1, а современные платы — Quartus Lite 21.1 или новее. Все они спокойно живут рядом, а скрипты выбирают подходящую по микросхеме вашей платы.

Почему скрипт говорит, что ни одна версия Quartus не подходит?

Значит, ни одна из установленных версий не умеет собирать проект под микросхему выбранной платы. В сообщении перечислено, что найдено. Чаще всего это старая плата (нужен Quartus 13.x) или DE23-Lite (нужен Quartus Pro). См. раздел 3.1.

Под Windows скрипты не запускаются

Запускайте их из **Git Bash**, а не из `cmd.exe` и не из PowerShell. Git Bash устанавливается вместе с Git (раздел 4.1.8).

Icarus выдаёт ошибки синтаксиса в примерах

Скорее всего, у вас версия 11 или 12. Нужна 13, лучше 14 — часть примеров использует конструкции, которых старые версии не понимают. См. раздел 4.1.5.

Плата не прошивается с первого раза

Переподключите кабель программатора и запустите `./04_configure_fpga.bash` — пересобирать проект заново не нужно. Под Linux проверьте правило udev (подраздел 4.2.1.2), в том числе строку для USB Blaster III, если у вас DE23-Lite.

На Mac программа не запускается: «автор не установлен»

Снимите атрибут карантина: `xattr -rd com.apple.quarantine ~/oss-cad-suite`. См. раздел 4.3.

Какую плату купить, если её ещё нет?

Tang Nano 9K с модулем ЖК-экрана и звуковыми модулями: примерно 60 долларов за комплект, быстрый синтез, работает под всеми тремя операционными системами, включая macOS. Для плат Altera и Xilinx под macOS нужна виртуальная машина.

Зачем мне терминальная программа?

Для упражнений, в которых плата обменивается символами с компьютером по UART: `labs/8_unfinished/8_1_uart` и, дальше, упражнения с процессорными ядрами из `labs/5_cpu`, куда программа загружается через UART. Во всём остальном терминал не нужен. Установка — разделы 4.1.11 (Windows) и 4.2.11 (Linux), на macOS см. 4.3.

Терминал открывается, но плата не прошивается

Порт занят терминалом. Закройте терминал и запустите `04_configure_fpga.bash` снова.

В терминале не видно того, что я набираю

Выключено локальное эхо. В minicom добавьте строку `pu localecho Yes` в `~/.minirc.dfl`, в picocom запускайте с `--echo`. В упражнении `8_1_1_uart_7segment` эхо и не ожидается: плата там только принимает, передатчик выключен.

Где полный список плат?

В разделе 3 этой инструкции и в файле boards/README.md репозитория — он обновляется вместе с кодом.

Иллюстрации взяты из предыдущей редакции `Инструкция_по_установке_ПО.pdf` и из репозитория basics-graphics-music. Технические сведения о том, как скрипты находят инструменты, проверены по коду ветки `main` репозитория. Полный перечень расхождений с предыдущей редакцией — в отдельном файле.